You said:

I want the list of questions which are asked during the interview in application security or penetration testing for an 4 years expeirence candidate from basic questions to advance. Please me the list of 100 questions which are been asked. Please separate for API, Web and Mobile pentesting each. Each 100 and from basic to advances .

ChatGPT said:

300 Application Security & Penetration Testing Interview Questions

4 Years Experience Basic to Advanced Interview Preparation

I've structured this as an interview question bank for an **Application Security / VAPT candidate with approximately 4 years of experience**.

The questions are designed around what interviewers generally expect from someone who has worked on real-world Web, API and Mobile penetration testing projects, rather than someone who has only studied theoretical concepts.

You should prepare to:

- Explain the concept clearly.
- Describe how you identify and exploit the vulnerability.
- Explain how you validate its impact.
- Demonstrate practical knowledge of Burp Suite, Postman, Frida and other testing tools.
- Explain remediation and provide real-world examples.

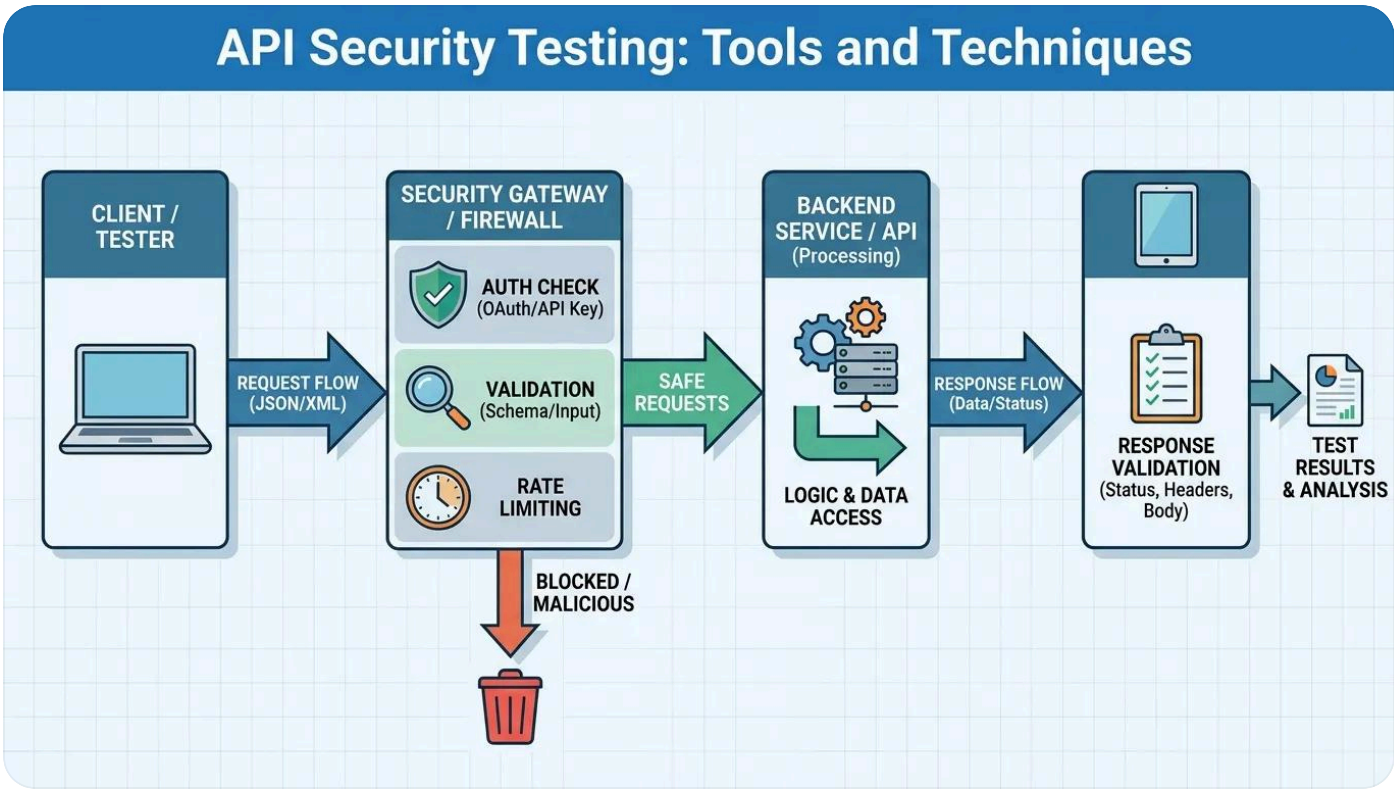
There are 300 questions in total:

Domain	Questions
API Penetration Testing	100
Web Application Penetration Testing	100
Mobile Application Penetration Testing	100
Total	**300**

Each section progresses through four levels:

1. Basic: Fundamentals and terminology.
2. Intermediate: Vulnerability identification and testing.
3. Advanced: Exploitation, bypasses and attack scenarios.
4. Expert: Real-world scenarios, architecture and complex security issues.

Part 1: API Penetration Testing (100 Questions)



Questions 1–25: Basic

Level 1: API Fundamentals

1. What is an API, and why is it used in modern applications?
2. What is the difference between REST, SOAP and GraphQL APIs?
3. What is the difference between RESTful APIs and traditional web services?
4. What are the different HTTP methods used in API communication?
5. Explain the difference between GET, POST, PUT, PATCH and DELETE.
6. What is the difference between PUT and PATCH?
7. What are HTTP request headers and response headers?
8. What is the difference between request parameters, path parameters and query parameters?
9. What is the difference between authentication and authorization in APIs?
10. What are the different HTTP status codes commonly encountered during API testing?

11. What is JSON, and why is it commonly used in APIs?
12. What is XML, and where is it used?
13. What is the difference between JSON and XML?
14. What is an API endpoint?
15. What is an API gateway, and what is its purpose?
16. What is API versioning, and why is it important?
17. What is the difference between stateless and stateful APIs?
18. What is CORS, and why is it implemented?
19. What is the difference between cookies and bearer tokens in API authentication?
20. What is the purpose of the Authorization header?
21. What is a Bearer token?
22. What is Basic Authentication, and how does it work?
23. What is OAuth 2.0?
24. What is JWT, and what are its three components?
25. What is the difference between access tokens and refresh tokens?

Questions 26–50: Intermediate

Level 2: API Vulnerabilities and Testing Methodology

26. What is BOLA (Broken Object Level Authorization)?
27. What is BFLA (Broken Function Level Authorization)?
28. What is Broken Object Property Level Authorization?
29. What is the difference between BOLA and BFLA?
30. How do you test for IDOR vulnerabilities in APIs?
31. How would you test whether one user can access another user's information?
32. What is mass assignment in API security?
33. How do you identify mass assignment vulnerabilities?
34. What is excessive data exposure?
35. How do you identify sensitive information disclosure in API responses?
36. What is rate limiting, and why is it important?
37. How do you test for API rate-limit bypass?
38. What is a brute-force attack against an API?
39. How do you test authentication bypass in APIs?
40. What is an insecure direct API endpoint?
41. What is improper input validation in APIs?
42. How do you test SQL injection in API endpoints?
43. How do you identify NoSQL injection in APIs?

44. What is command injection through an API?
45. How do you test for server-side request forgery (SSRF) through an API?
46. What is XML External Entity (XXE) injection?
47. How do you test XXE in SOAP APIs?
48. What is an insecure API error message?
49. How do you identify business logic vulnerabilities in APIs?
50. How do you perform API penetration testing using Burp Suite and Postman?

Questions 51–75: Advanced

Level 3: Advanced API Security

51. Explain the complete JWT authentication mechanism.
52. What is the JWT `alg: none` vulnerability?
53. What is JWT algorithm confusion?
54. How can an attacker exploit weak JWT signing secrets?
55. How do you test JWT signature validation?
56. What is the difference between HS256 and RS256?
57. How can JWT key confusion occur?
58. What happens if an API does not validate JWT expiration?
59. How do you test JWT token reuse?
60. How can refresh token rotation be bypassed or misconfigured?
61. How do you test OAuth 2.0 authorization code flow?
62. What is the OAuth authorization code interception attack?
63. What is PKCE, and how does it prevent authorization code interception?
64. What is an OAuth redirect URI manipulation vulnerability?
65. What is the OAuth state parameter, and why is it important?
66. What is OAuth scope abuse?
67. How can an attacker exploit weak API authorization across multiple roles?
68. How do you test horizontal and vertical privilege escalation in APIs?
69. How do you identify race conditions in API endpoints?
70. How do you test HTTP parameter pollution in APIs?
71. What is request smuggling, and how can it affect API gateways?
72. How do you identify API cache poisoning?
73. What is GraphQL introspection, and why can it be dangerous?
74. What are GraphQL query depth and query complexity attacks?
75. How do you test GraphQL authorization vulnerabilities?

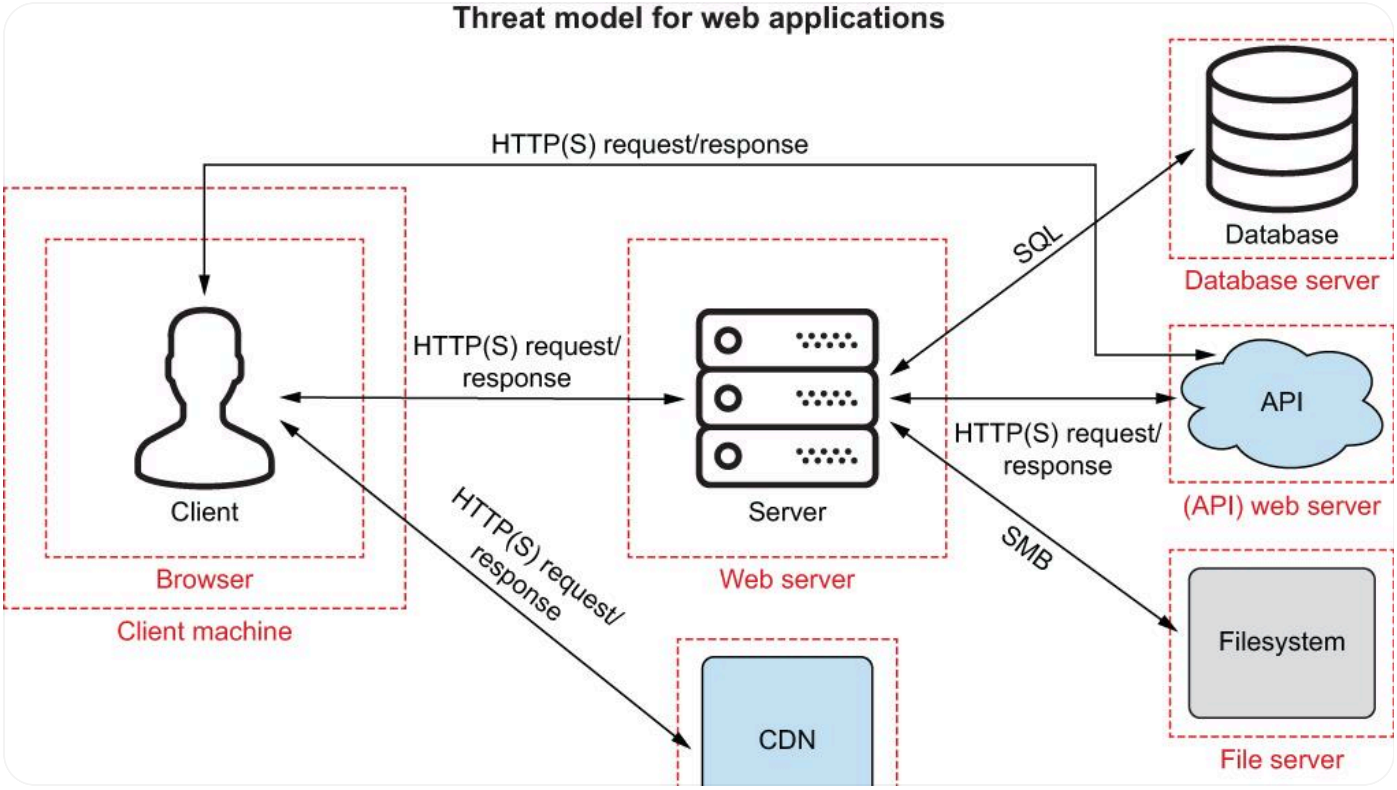
Questions 76–100: Expert / Scenario-Based

Level 4: Real-World API Security Scenarios

76. An API returns another user's account information when you change the user ID. How would you investigate and exploit this?
77. An API allows you to change the HTTP method from GET to PATCH. How would you test whether this creates an authorization issue?
78. You discover that an API accepts additional JSON parameters that are not visible in the application. How would you investigate mass assignment?
79. An API returns a valid JWT even after the user logs out. Is this a vulnerability? Explain.
80. An application uses JWT authentication. How would you test whether tokens can be forged?
81. An API has rate limiting, but changing the `X-Forwarded-For` header bypasses it. How would you assess this?
82. An API allows users to update their role through a JSON parameter. How would you validate the impact?
83. An API exposes internal server IP addresses and database errors. How would you assess the severity?
84. How would you test an API when you have only a Swagger/OpenAPI specification?
85. How would you identify undocumented API endpoints?
86. How would you test an API when you have no frontend access?
87. How would you identify hidden administrative API endpoints?
88. An API returns sensitive information in a 200 OK response. How would you determine whether it is an actual vulnerability?
89. How would you test API authorization when you have multiple user roles?
90. An API allows a user to purchase an item with a negative price. How would you investigate the business logic vulnerability?
91. How would you test duplicate transactions and payment race conditions?
92. How would you test whether a password reset API can be abused to take over accounts?
93. How would you test OTP verification APIs for brute force and bypass vulnerabilities?
94. How would you test API authentication when access tokens are stored in mobile applications?
95. How would you assess the security of APIs communicating with third-party services?
96. How would you test an API gateway for security misconfigurations?
97. How would you identify and exploit insecure webhook implementations?
98. How would you test an API that uses multiple authentication mechanisms?
99. How would you perform a complete API security assessment from reconnaissance to reporting?

100. You discover a critical API vulnerability in production. How would you validate, demonstrate impact and recommend remediation without affecting business operations?

Part 2: Web Application Penetration Testing (100 Questions)



Questions 1–25: Basic

Level 1: Web Fundamentals

1. What is the difference between HTTP and HTTPS?
2. Explain the complete HTTP request-response lifecycle.
3. What happens when you enter a URL in a browser?
4. What is DNS, and how does DNS resolution work?
5. What is the difference between HTTP/1.1, HTTP/2 and HTTP/3?
6. What are HTTP request and response headers?
7. What are cookies, and how do they work?
8. What is a session, and how is it maintained?
9. What is the difference between session-based and token-based authentication?
10. What are Secure, HttpOnly and SameSite cookie attributes?
11. What is the difference between GET and POST requests?
12. What is the same-origin policy?

13. What is CORS, and how does it work?
14. What is a reverse proxy?
15. What is a web application firewall (WAF)?
16. What is input validation?
17. What is output encoding?
18. What is the difference between client-side and server-side validation?
19. What is the OWASP Top 10?
20. What is the difference between vulnerability scanning and penetration testing?
21. What is reconnaissance in web penetration testing?
22. What is directory and file enumeration?
23. What is content discovery, and which tools are used?
24. What is Burp Suite, and how do you use it during web penetration testing?
25. Explain the complete methodology you follow while conducting a web application penetration test.

Questions 26–50: Intermediate

Level 2: Common Web Vulnerabilities

26. What is Cross-Site Scripting (XSS)?
27. Explain reflected, stored and DOM-based XSS.
28. What is the difference between XSS and HTML injection?
29. How do you identify and exploit XSS?
30. What is Content Security Policy (CSP)?
31. How can CSP be bypassed?
32. What is Cross-Site Request Forgery (CSRF)?
33. How does CSRF work?
34. What are the different techniques to prevent CSRF?
35. Can CSRF occur if an application uses JSON APIs?
36. What is SQL injection?
37. Explain error-based, union-based, blind and time-based SQL injection.
38. What is the difference between SQL injection and NoSQL injection?
39. How do you exploit SQL injection using Burp Suite?
40. What is command injection?
41. What is Local File Inclusion (LFI)?
42. What is Remote File Inclusion (RFI)?
43. What is Directory Traversal?
44. What is Server-Side Request Forgery (SSRF)?

45. What is Server-Side Template Injection (SSTI)?
46. What is XML External Entity (XXE) injection?
47. What is Open Redirect?
48. What is Clickjacking, and how can it be prevented?
49. What is security misconfiguration?
50. What is sensitive information disclosure?

Questions 51–75: Advanced

Level 3: Advanced Web Application Security

51. Explain authentication bypass techniques in web applications.
52. How do you test for weak password policies?
53. What is session fixation?
54. What is session hijacking?
55. How do you test session management vulnerabilities?
56. What is an insecure password reset mechanism?
57. How do you test account takeover vulnerabilities?
58. What is privilege escalation in web applications?
59. Explain horizontal and vertical privilege escalation.
60. What is an IDOR vulnerability in a web application?
61. What is a race condition vulnerability?
62. How do you identify race conditions using Burp Suite?
63. What is HTTP request smuggling?
64. Explain CL.TE, TE.CL and TE.TE request smuggling.
65. What is HTTP Host Header injection?
66. What is Web Cache Poisoning?
67. What is Web Cache Deception?
68. What is Prototype Pollution?
69. How can Prototype Pollution lead to XSS or privilege escalation?
70. What is DOM Clobbering?
71. What is WebSocket security, and how do you test WebSocket connections?
72. What is WebSocket Cross-Site Hijacking?
73. What is CORS misconfiguration, and how can it lead to sensitive data exposure?
74. How do you identify and exploit SSTI in different template engines?
75. How do you test file upload functionality for security vulnerabilities?

Questions 76–100: Expert / Scenario-Based

Level 4: Real-World Web Security Scenarios

76. You discover that a user can access another user's profile by modifying an ID in the URL. How would you assess this?
77. An application implements CSRF tokens, but the application still appears vulnerable. What would you investigate?
78. You discover XSS in an application with a strict CSP. How would you investigate possible bypasses?
79. A login page has rate limiting. How would you test whether the restriction can be bypassed?
80. An application uses JWT stored in local storage. What security risks would you assess?
81. An application uses session cookies without HttpOnly. What is the potential impact?
82. A web application allows file uploads but restricts extensions. How would you test the implementation?
83. How would you test file upload functionality for web shells and server-side execution risks?
84. An application redirects users based on a parameter. How would you investigate Open Redirect?
85. How would you test an application for LFI when direct file paths are blocked?
86. You discover SSRF in a cloud-hosted application. How would you assess the impact while avoiding access to unauthorized sensitive data?
87. How would you identify SQL injection when the application does not return database errors?
88. An application uses a WAF. How would you approach testing input validation and filtering?
89. How would you investigate an authentication bypass caused by inconsistent URL normalization?
90. How would you test business logic vulnerabilities in an e-commerce application?
91. How would you test a payment gateway integration?
92. How would you identify privilege escalation in an application with multiple user roles?
93. How would you test a web application that uses microservices architecture?
94. How would you test GraphQL endpoints integrated into a web application?
95. How would you identify sensitive information exposed through JavaScript files?
96. How would you test an application for cache-related security vulnerabilities?
97. How would you identify vulnerabilities caused by insecure deserialization?
98. How would you assess the security of a web application using OAuth and SAML SSO?
99. How would you conduct a complete web application VAPT, from reconnaissance to final reporting?
100. You identify a critical vulnerability that requires multiple steps to exploit. How would you create a realistic attack chain and demonstrate its business impact?

Part 3: Mobile Application Penetration Testing (100 Questions)



This section gives extra attention to **Android penetration testing**, including APK analysis, JADX, MobSF, Frida, Objection, ADB, SSL pinning and Android application security architecture.

Questions 1–25: Basic

Level 1: Mobile Application Fundamentals

1. What is mobile application penetration testing?
2. What is the difference between Android and iOS application security testing?
3. Explain the complete Android application architecture.
4. What is an APK file?
5. What are the important components of an Android application?
6. What is AndroidManifest.xml?
7. What is the purpose of the `minSdkVersion` and `targetSdkVersion` ?
8. What are Activities, Services, Broadcast Receivers and Content Providers?
9. What is an Intent in Android?
10. What is the difference between explicit and implicit intents?
11. What is an exported Android component?
12. What is ADB, and how do you use it during mobile penetration testing?

13. What is the difference between static and dynamic analysis?
14. What is reverse engineering in mobile application security?
15. What is JADX, and why is it used?
16. What is APKTool?
17. What is MobSF, and how does it help in mobile application security testing?
18. What is the difference between APK and AAB?
19. What is application signing in Android?
20. What is Android Keystore?
21. What is the difference between internal and external storage in Android?
22. What is SharedPreferences?
23. What is SQLite, and how is it used in Android applications?
24. What is WebView in Android?
25. Explain the complete Android application penetration testing methodology.

Questions 26–50: Intermediate

Level 2: Mobile Vulnerabilities and Testing

26. What is insecure local data storage?
27. How do you identify sensitive data stored in SharedPreferences?
28. How do you identify sensitive data stored in SQLite databases?
29. What is insecure logging in Android?
30. How do you identify hardcoded API keys and credentials inside an APK?
31. What is insecure communication in mobile applications?
32. How do you intercept Android application traffic using Burp Suite?
33. What is SSL/TLS certificate validation?
34. What is SSL pinning?
35. What is the difference between SSL pinning and certificate pinning?
36. How do you identify whether an Android application implements certificate pinning?
37. How do you bypass SSL pinning using Frida?
38. What is Objection, and how is it used in Android testing?
39. What is Frida, and how does its instrumentation mechanism work?
40. What is root detection in Android applications?
41. How do you identify root detection mechanisms?
42. How do you bypass root detection using Frida?
43. What is emulator detection?
44. How do you identify and bypass emulator detection?
45. What are deep links in Android?

46. What are Android App Links?
47. How do you test deep link vulnerabilities?
48. What is an insecure Content Provider?
49. How do you test exported Activities and Services?
50. What is an insecure Broadcast Receiver?

Questions 51–75: Advanced

Level 3: Advanced Android Security

51. Explain Android application sandboxing.
52. What is the Android permission model?
53. What is the difference between normal, dangerous and signature permissions?
54. What is the difference between `android:exported="true"` and `android:exported="false"` ?
55. How can an exported Activity lead to unauthorized access?
56. How do you exploit an insecure Content Provider?
57. What is an Intent redirection vulnerability?
58. What is PendingIntent hijacking?
59. What is an Android task hijacking vulnerability?
60. What is tapjacking in Android?
61. What is an insecure WebView configuration?
62. What is JavaScript interface exposure in Android WebView?
63. How can `addJavascriptInterface()` introduce security risks?
64. What is the risk of allowing cleartext traffic in Android?
65. What is Network Security Configuration?
66. How do you test Android applications for weak TLS configurations?
67. What is cryptographic key management in Android?
68. What are common cryptographic implementation mistakes in mobile applications?
69. What is the risk of storing encryption keys in SharedPreferences?
70. What is biometric authentication bypass?
71. How do you test biometric authentication implementation?
72. What is Play Integrity API, and how does it help protect Android applications?
73. What is the difference between root detection and runtime integrity verification?
74. How do you perform runtime method hooking using Frida?
75. How do you identify and bypass anti-debugging and anti-tampering mechanisms?

Questions 76–100: Expert / Scenario-Based

Level 4: Real-World Mobile Security Scenarios

76. You receive an APK file without source code. How would you start the penetration testing process?
77. An Android application does not allow proxy interception. How would you investigate the issue?
78. An application implements SSL pinning. Explain your complete approach to bypassing it in an authorized test environment.
79. You discover that an application stores access tokens in SharedPreferences. How would you assess the impact?
80. An application detects rooted devices and closes automatically. How would you investigate and test this control?
81. An application works on a real device but refuses to run on an emulator. How would you investigate emulator detection?
82. You discover hardcoded credentials inside an APK. How would you validate their security impact?
83. An application uses encrypted SQLite databases. How would you investigate the encryption and key management?
84. How would you test whether sensitive data is exposed through Android backups?
85. How would you test screenshots, clipboard and keyboard caching for sensitive information exposure?
86. How would you test an application for insecure deep link handling?
87. An application uses an exported Content Provider. How would you identify unauthorized data access?
88. How would you test an Android application for intent spoofing?
89. How would you investigate an insecure WebView that loads remote URLs?
90. How would you identify and exploit a JavaScript bridge vulnerability?
91. How would you test biometric authentication if the application also supports PIN or password fallback?
92. An application uses certificate pinning, root detection and anti-debugging together. How would you approach the assessment?
93. How would you identify secrets and API endpoints hidden in native `.so` libraries?
94. How would you analyze an Android application that uses native code instead of Java/Kotlin?
95. How would you perform dynamic instrumentation of native Android functions?
96. How would you test an Android application for insecure inter-process communication (IPC)?

97. How would you investigate an application that uses Firebase or other third-party mobile backend services?
98. How would you test a mobile application when its backend APIs are also in scope?
99. How would you perform a complete Android penetration test using MobSF, JADX, ADB, Burp Suite, Frida and Objection?
100. You identify a vulnerability that requires a rooted device to exploit. How would you determine whether it represents a genuine security risk and explain its business impact?

Part 4: Questions Beyond These Three Domains

For a candidate with four years of experience, interviewers may also test your broader AppSec knowledge. These are particularly relevant for Application Security Consultant and VAPT Consultant positions.

I recommend preparing these additional topics:

Topic	What you should know
OWASP	OWASP Top 10, API Top 10, Mobile Top 10
SDLC	Secure SDLC, security requirements, security gates
Threat Modeling	STRIDE, DREAD, attack trees, trust boundaries
DevSecOps	SAST, DAST, SCA, secrets scanning, CI/CD
Cloud Security	AWS IAM, S3, security groups, SSRF, metadata services
Source Code Review	Java, JavaScript, Python, Android code
Vulnerability Management	CVSS, CWE, remediation, retesting, SLA
Authentication	OAuth 2.0, OIDC, SAML, MFA
Reporting	Risk assessment, business impact, remediation recommendations

How I recommend preparing for these 300 questions

At four years of experience, simply memorizing definitions will not be sufficient. Interviewers will often ask three or four follow-up questions after your initial answer.

For every vulnerability, prepare these six things:

1. **Definition:** What is the vulnerability?
2. **Root cause:** Why does it happen?
3. **Identification:** How do you identify it?
4. **Exploitation:** How do you practically exploit it?
5. **Impact:** What can an attacker achieve?
6. **Remediation:** How should developers fix it?

For example, if the interviewer asks about IDOR:

- What is IDOR?
- How do you identify IDOR?
- How do you differentiate IDOR from BOLA?
- Can IDOR exist without changing an ID parameter?
- How do you test horizontal privilege escalation?
- How do you test vertical privilege escalation?
- What is the business impact?
- How should developers fix it?

This is the depth you should target.

Suggested preparation priority

Considering the three domains, I would follow this sequence:

1. API Penetration Testing — Highest priority

JWT, OAuth, BOLA, BFLA, mass assignment, business logic, authorization and race conditions.

2. Web Application Pentesting — High priority

XSS, SQLi, SSRF, CSRF, CORS, SSTI, request smuggling, authentication and business logic.

3. Android Pentesting — High priority

APK analysis, Android components, SSL pinning, Frida, root detection, IPC, WebView and local storage.

How we can use this question bank

My recommendation is to work through it in batches of 10 questions.

For each batch, the preparation should include:

- Interview-ready answers.
- Detailed technical explanations.
- Practical examples and attack scenarios.
- Burp Suite / Postman / Frida demonstrations where relevant.
- Common interviewer follow-up questions.
- What a four-year-experience candidate is expected to say.

The goal should be to reach a stage where you can explain the vulnerability, demonstrate it practically, defend your findings and recommend a proper fix, rather than simply remember 300 answers.

I can take you through the complete 300-question bank as a structured mock-interview and training program, starting with API questions 1–10.

You said:

Please answer me all the questions thoroughly like what exactly I need to answer. I need complete answer for each questions not just 1 or 2 lines answer. I really need to clear interview.

ChatGPT said:

Absolutely. Let's prepare you to **clear a four-year-experience Application Security / VAPT interview**, not just memorize a question bank.

I understand what you're looking for. You don't want textbook definitions or two-line answers. You want to know exactly what to say when an interviewer asks you a technical question, how to explain it confidently, what practical examples to give, and how to handle follow-up questions.

There are 300 questions in our question bank. I'll turn them into a detailed interview preparation guide.

How I'll structure every answer

For each question, I'll cover the following wherever applicable:

1. **Interview-ready answer:** A complete answer that you can actually speak during an interview.
2. **Technical explanation:** How the technology or vulnerability works internally.
3. **Practical demonstration:** How you would test it using tools such as Burp Suite, Postman, Frida, ADB, etc.
4. **Real-world example:** A realistic application scenario.
5. **Follow-up questions:** What the interviewer might ask next, with answers.
6. **Remediation:** How developers should fix the issue.

I'll also distinguish between what you should know theoretically and what you should be able to demonstrate practically.

Complete preparation roadmap

300 detailed interview answers

Part 1: API Pentesting — 100 questions

REST, SOAP, GraphQL, JWT, OAuth, BOLA, BFLA, injection, business logic, race conditions and API security architecture.

Starting now

Part 2: Web Pentesting — 100 questions

HTTP, XSS, SQLi, SSRF, CSRF, CORS, SSTI, request smuggling, authentication, authorization and advanced exploitation.

Part 3: Android Mobile Pentesting — 100 questions

APK analysis, JADX, MobSF, ADB, Frida, SSL pinning, root detection, Android components, IPC and reverse engineering.

Important: I'll deliver this in manageable batches, normally 10 questions at a time. That allows me to explain each answer properly without cutting corners or turning the material into a massive collection of shallow definitions.

PART 1: API PENETRATION TESTING

Questions 1–10 of 100 **Basic level**

1. What is an API, and why is it used in modern applications?

INTERVIEW-READY ANSWER

An API, or Application Programming Interface, is a mechanism that allows two different software applications or components to communicate with each other through a predefined set of rules, methods and data formats.

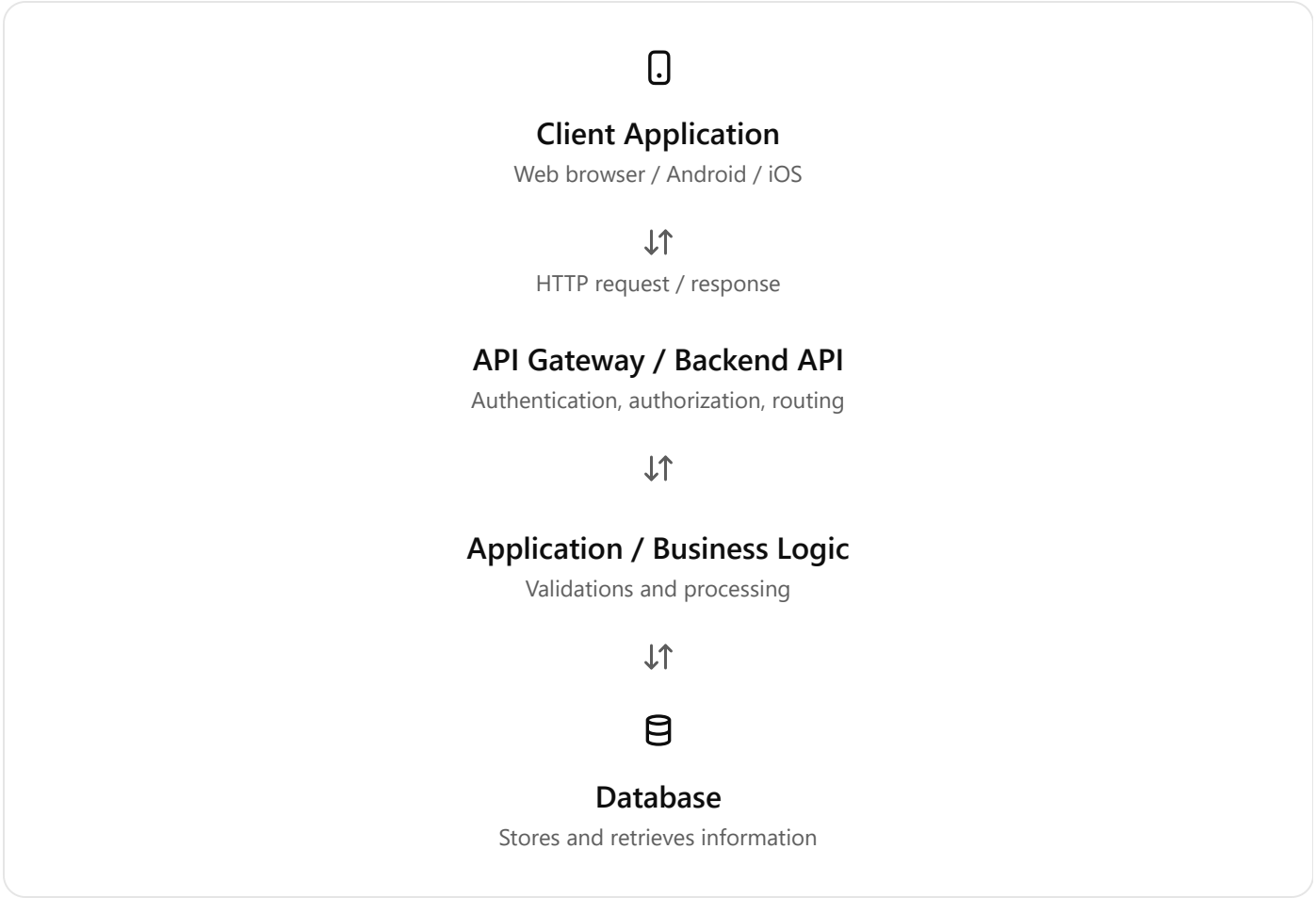
In simple terms, an API acts as an intermediary between a client and a server. It receives requests from the client, processes them on the server side and returns the appropriate response.

For example, consider a banking mobile application.

When a user logs in, checks their account balance or transfers money, the mobile application communicates with backend services through APIs.

The mobile application does not directly access the database. Instead, it sends an API request to the backend server. The backend validates the request, performs the required business logic, interacts with the database and returns a response.

A typical API communication flow looks like this:



Why are APIs used?

1. Communication: They allow frontend applications and backend systems to communicate.
2. Reusability: The same API can be consumed by web, Android and iOS applications.
3. Integration: APIs allow organizations to integrate third-party services such as payment gateways, SMS providers and identity verification systems.
4. Scalability: Backend services can be developed and maintained independently.
5. Automation: APIs allow applications, scripts and other services to perform operations programmatically.

From an application security perspective:

APIs are an important attack surface because they expose backend functionality and business logic.

During API penetration testing, I focus on authentication, authorization, input validation, data exposure, rate limiting, business logic and improper security configurations.

I also verify whether users can access or modify resources belonging to other users.

Practical example:

Suppose an application has this endpoint:

```
GET /api/v1/accounts/1001
Host: bank.example.com
Authorization: Bearer <access_token>
```

It returns the account information of user 1001.

As a penetration tester, I would check whether changing the account ID to 1002 allows me to access another user's information.

If the backend does not properly validate object-level authorization, this could result in a BOLA vulnerability.

Likely follow-up:

Q: Does an API always communicate over HTTP?

No. APIs can use different communication protocols and mechanisms, including HTTP/HTTPS, WebSockets, gRPC, message queues and other protocols. REST APIs commonly use HTTP, but APIs are not limited to REST or HTTP.

2. What is the difference between REST, SOAP and GraphQL APIs?

INTERVIEW-READY ANSWER

REST, SOAP and GraphQL are different approaches to designing and exposing APIs. They differ in their communication structure, data formats, request handling and flexibility.

1. REST (Representational State Transfer)

REST is an architectural style commonly used to develop web APIs.

REST APIs typically communicate over HTTP and use HTTP methods such as GET, POST, PUT, PATCH and DELETE to perform operations on resources.

They commonly exchange data in JSON format, although XML and other formats are also possible.

Example:

```
GET /api/users/101
```

Response:

```
{
  "id": 101,
  "name": "John",
  "email": "john@example.com"
}
```

REST is generally resource-oriented. Each endpoint represents a resource or collection of resources.

2. SOAP (Simple Object Access Protocol)

SOAP is a messaging protocol used for communication between applications.

It typically uses XML-based messages and defines a structured envelope containing the request or response.

SOAP services often use WSDL (Web Services Description Language) to describe available operations, message structures and service endpoints.

Example SOAP request:

```
<soap:Envelope>
  <soap:Body>
    <GetUser>
      <UserId>101</UserId>
    </GetUser>
  </soap:Body>
</soap:Envelope>
```

SOAP supports security-related standards such as WS-Security and is commonly encountered in enterprise and legacy systems.

3. GraphQL

GraphQL is a query language and runtime for APIs.

Unlike conventional REST APIs, GraphQL commonly exposes a single endpoint through which clients request specific fields and related objects.

Example:

```
query {
  user(id: 101) {
    name
    email
  }
}
```

The server returns only the requested fields, assuming the authorization and schema permit access.

Comparison

Feature	REST	SOAP	GraphQL
Type	Architectural style	Protocol	Query language/runtime
Data format	Usually JSON	XML	Usually JSON response
Communication	Commonly HTTP	HTTP and other bindings	Commonly HTTP
Endpoints	Multiple resources	Service operations	Often one endpoint
Flexibility	Resource-oriented	Contract-oriented	Client selects fields
Security testing focus	BOLA, BFLA, injection	XXE, XML validation, authorization	Introspection, depth, authorization

From a penetration testing perspective:

- REST: I focus on IDOR/BOLA, mass assignment, broken authorization, excessive data exposure and rate limiting.
- SOAP: I focus on XXE, XML injection, weak authentication, WS-Security misconfiguration and improper XML processing.
- GraphQL: I focus on introspection exposure, unauthorized field access, query depth attacks, alias abuse and expensive queries.

Likely follow-up questions

Q: Which API type is more secure?

None is inherently more secure than the others. Security depends on implementation, authentication, authorization, input validation, configuration and the controls implemented around the API.

Q: Can GraphQL have IDOR vulnerabilities?

Yes. If a GraphQL resolver retrieves objects based on user-supplied identifiers without checking whether the authenticated user is authorized to access those objects, it can result in BOLA/IDOR.

3. What is the difference between RESTful APIs and traditional web services?

INTERVIEW-READY ANSWER

A web service is a software system that allows applications to communicate over a network using standardized communication mechanisms.

A RESTful API is a particular type of API that follows REST architectural principles.

The important distinction is that a web service is a broader concept, whereas REST is one architectural approach used to implement web services.

Traditional web services may use SOAP, XML-RPC or other protocols and messaging formats.

RESTful APIs generally use HTTP methods and resource-based URLs to perform operations.

For example:

A SOAP-based service may expose an operation such as:

```
https://example.com/UserService
```

The client sends an XML request specifying the operation to perform.

A RESTful service may expose:

```
GET /users/101
POST /users
PUT /users/101
DELETE /users/101
```

Each endpoint represents a resource, and HTTP methods indicate the intended operation.

Key REST principles:

1. Client-server architecture: Client and server responsibilities are separated.
2. Statelessness: Each request contains the information required to process it. The server does not depend on client session state stored between requests.
3. Cacheability: Responses should indicate whether they can be cached.
4. Uniform interface: Resources are accessed through a consistent interface.
5. Layered system: The client may communicate through intermediaries such as proxies and gateways.

6. Code on demand: An optional constraint allowing servers to provide executable code to clients.

Security perspective:

When testing RESTful APIs, I examine HTTP methods, authentication, authorization, input validation, CORS, rate limiting and resource-level access control.

For traditional SOAP services, I additionally examine XML processing, XXE, SOAP message security and WSDL exposure.

Likely follow-up:

Q: Is every REST API a web service?

REST APIs exposed over a network are generally considered web services. However, not every web service is RESTful. SOAP is another example of a web-service approach.

4. What are the different HTTP methods used in API communication?

INTERVIEW-READY ANSWER

HTTP methods define the intended action that a client wants to perform on a resource exposed by a server.

The most commonly used HTTP methods during API communication are GET, POST, PUT, PATCH and DELETE.

There are also other methods such as HEAD, OPTIONS, TRACE and CONNECT.

1. GET

GET is used to retrieve information from the server.

Example:

```
GET /api/users/101 HTTP/1.1
Host: example.com
Authorization: Bearer <token>
```

It should not cause a requested application-state change on the server.

2. POST

POST is generally used to submit data or create resources.

Example:

```
POST /api/users HTTP/1.1
Content-Type: application/json

{
  "name": "John",
  "email": "john@example.com"
}
```

3. PUT

PUT is used to create or replace the state of a resource at a specified URI.

Example:

```
PUT /api/users/101
Content-Type: application/json

{
  "name": "John Smith",
  "email": "john@example.com"
}
```

4. PATCH

PATCH is used to apply partial modifications to a resource.

Example:

```
PATCH /api/users/101
Content-Type: application/json

{
```

```
  "email": "new@example.com"
}
```

5. DELETE

DELETE is used to request deletion of a resource.

Example:

```
DELETE /api/users/101
```

6. HEAD

HEAD is similar to GET, but the server returns response headers without the response body.

7. OPTIONS

OPTIONS is used to identify communication options supported by a resource, including supported methods. It is also used in CORS preflight requests.

8. TRACE

TRACE is intended for diagnostic purposes and can reveal how a request is processed through intermediaries. It is commonly disabled because of security concerns, including potential Cross-Site Tracing risks in vulnerable environments.

9. CONNECT

CONNECT establishes a tunnel to the server identified by the target resource, commonly used for HTTPS tunneling through proxies.

Important security testing point:

During API penetration testing, I don't assume that the HTTP method is properly restricted.

For example, if an endpoint only exposes GET functionality in the frontend, I would test whether the backend also accepts POST, PUT, PATCH or DELETE.

I would verify whether changing the HTTP method exposes unauthorized functionality or bypasses authorization controls.

Likely follow-up:

Q: Are GET and PUT idempotent?

Yes. GET and PUT are defined as idempotent methods.

Idempotency means that making the same request multiple times has the same intended effect on the server as making it once.

DELETE is also idempotent in HTTP semantics, even though the first request may delete the resource and subsequent requests may return a different status code.

POST is not inherently idempotent.

PATCH is not inherently idempotent, although a particular PATCH operation can be designed to be idempotent.

5. What is the difference between GET, POST, PUT, PATCH and DELETE?

INTERVIEW-READY ANSWER

These HTTP methods differ in their intended operation on server resources.

Method	Purpose	Typical use
GET	Retrieve data	View user profile
POST	Submit data or create a resource	Register user
PUT	Create or replace a resource	Replace user profile
PATCH	Partially modify a resource	Update email only
DELETE	Delete a resource	Delete user account

Let's consider a banking application.

Suppose the API exposes:

```
/api/accounts/1001
```

- GET retrieves account details.
- POST may create a new account.
- PUT replaces the account resource.
- PATCH updates selected account attributes.
- DELETE requests deletion of the account resource.

Important technical differences:

GET:

- Safe method.
- Idempotent.
- Should not modify application state as its intended operation.
- Can be cached depending on response headers and cache rules.

POST:

- Generally used for resource creation or processing.
- Not inherently idempotent.
- Repeated requests may create duplicate transactions if the application does not handle them safely.

PUT:

- Idempotent.
- Usually replaces the resource representation.
- Repeated identical requests should produce the same intended resource state.

PATCH:

- Used for partial updates.
- Not inherently idempotent.
- Can use different patch formats, including JSON Patch and JSON Merge Patch.

DELETE:

- Idempotent by HTTP semantics.
- Requests resource deletion.
- Repeated requests should have the same intended effect.

Security perspective:

One important test is checking whether the backend properly validates the operation regardless of which method is used.

For example, suppose a normal user can retrieve their profile using GET.

I would investigate whether the same user can modify restricted attributes by sending:

```
PATCH /api/users/101

{
  "role": "admin"
}
```

If the backend accepts this modification without proper authorization, it may indicate a mass assignment or broken property-level authorization vulnerability.

Likely follow-up:

Q: If an API returns 200 OK for DELETE, does that mean the resource was deleted?

Not necessarily. HTTP status alone is insufficient. I would verify the resource state through a subsequent authorized GET request or another reliable application-level check.

6. What are HTTP request headers and response headers?

INTERVIEW-READY ANSWER

HTTP headers are metadata fields exchanged between the client and server during HTTP communication.

They provide additional information about the request or response, including content type, authentication credentials, caching instructions, accepted formats and security policies.

There are two broad categories:

1. Request headers

Request headers are sent by the client to provide information about the request.

Example:

```
GET /api/profile HTTP/1.1
Host: example.com
Authorization: Bearer eyJhbGciOi...
Content-Type: application/json
Accept: application/json
User-Agent: Mozilla/5.0
Origin: https://example.com
```

Important request headers:

- Host: Identifies the target host.
- Authorization: Carries authentication credentials.
- Content-Type: Specifies the media type of the request body.
- Accept: Specifies acceptable response media types.
- Origin: Identifies the origin initiating a request.
- User-Agent: Identifies the client software.
- Cookie: Sends applicable cookies to the server.

2. Response headers

Response headers are returned by the server and provide information about the response and how the client should process it.

Example:

```
HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: no-store
X-Content-Type-Options: nosniff
Strict-Transport-Security: max-age=31536000

{
  "name": "John"
}
```

Important response headers:

- Content-Type: Specifies the response media type.
- Content-Length: Indicates the response body size.

- Cache-Control: Defines caching behavior.
- Set-Cookie: Sets cookies in the client.
- Strict-Transport-Security: Instructs browsers to use HTTPS for the applicable host.
- X-Content-Type-Options: Helps prevent MIME-type sniffing.
- Content-Security-Policy: Defines browser resource-loading policies.
- Access-Control-Allow-Origin: Specifies allowed origins for CORS.

How do I test HTTP headers?

During penetration testing, I inspect request and response headers using Burp Suite.

I check for:

1. Missing security headers.
2. Insecure cookie attributes.
3. Sensitive information exposed in headers.
4. Incorrect CORS configuration.
5. Host header injection.
6. Cache-control misconfigurations.
7. Trust placed in attacker-controlled headers such as X-Forwarded-For.

Important clarification:

Not every missing security header automatically represents a vulnerability. I assess the application architecture, browser behavior and actual security impact before reporting a finding.

Likely follow-up:

Q: Can an attacker manipulate HTTP headers?

Yes. A client generally controls many request headers. The server must not blindly trust security-sensitive values supplied by the client, such as forwarded IP headers, host-related values or custom role headers.

7. What is the difference between request parameters, path parameters and query parameters?

INTERVIEW-READY ANSWER

Parameters are values passed to an API to identify resources, provide input or control how the server processes a request.

They are commonly passed through the URL path, URL query string or request body.

1. Path parameters

Path parameters are included directly in the URL path and generally identify a specific resource.

Example:

```
GET /api/users/101
```

Here, `101` is the path parameter representing the user ID.

2. Query parameters

Query parameters are supplied after the question mark in the URL.

Example:

```
GET /api/users?role=admin&status=active
```

Here:

- `role=admin`
- `status=active`

These are query parameters that may be used for filtering or searching.

3. Body parameters

Body parameters are included in the HTTP request body, commonly in JSON or XML format.

Example:

```
POST /api/users

{
  "username": "john",
```

```
  "email": "john@example.com",
  "role": "user"
}
```

Here, username, email and role are body parameters.

4. Header parameters

Some API inputs are passed through HTTP headers.

Example:

```
Authorization: Bearer <token>
X-Request-ID: 12345
```

Security testing perspective:

I test all input locations because developers sometimes validate one input location but overlook another.

For example:

- Path parameters: Test IDOR/BOLA by changing object identifiers.
- Query parameters: Test SQL injection, parameter pollution and unauthorized filtering.
- Body parameters: Test mass assignment, injection and improper validation.
- Headers: Test authentication weaknesses, trust boundary issues and header injection.

Practical Burp Suite example:

Original request:

```
GET /api/users/101?include=profile HTTP/1.1
```

I would investigate:

- Changing user ID 101 to another user's ID.
- Modifying the include parameter.
- Adding unexpected parameters.
- Testing duplicate parameters.

- Testing parameter type mismatches.

I would then validate whether the server enforces authorization and input validation correctly.

Likely follow-up:

Q: Which parameter type is more dangerous?

None is inherently more dangerous. The risk depends on how the backend processes the parameter and whether it is properly validated and authorized.

8. What is the difference between authentication and authorization in APIs?

INTERVIEW-READY ANSWER

Authentication and authorization are two separate security mechanisms that work together to protect APIs.

Authentication verifies the identity of the user or system making the API request.

Authorization determines whether that authenticated identity has permission to perform the requested operation or access the requested resource.

Let's take an example of a banking application.

A user logs in using their username and password. The backend verifies the credentials and issues an access token.

This is authentication.

Now the user requests:

```
GET /api/accounts/1001
Authorization: Bearer <token>
```

The backend verifies the token and identifies the user.

Then it must check whether that user is authorized to access account 1001.

That is authorization.

Common authentication mechanisms:

1. Username and password.
2. Basic Authentication.
3. API keys.
4. Session cookies.
5. Bearer tokens.
6. JWT.
7. OAuth 2.0 and OpenID Connect.
8. Mutual TLS.

Common authorization models:

1. RBAC: Role-Based Access Control.
2. ABAC: Attribute-Based Access Control.
3. ACL: Access Control Lists.
4. ReBAC: Relationship-Based Access Control.

From a penetration testing perspective:

I test both independently.

For authentication, I check:

- Authentication bypass.
- Weak password policies.
- Brute-force protection.
- Token expiration.
- Token validation.
- Session management.
- Password reset mechanisms.

For authorization, I check:

- BOLA/IDOR.
- BFLA.
- Horizontal privilege escalation.
- Vertical privilege escalation.
- Unauthorized access to administrative functions.
- Unauthorized access to sensitive object properties.

Practical example:

Suppose two users have valid accounts:

- User A: ID 101
- User B: ID 102

Both successfully authenticate.

If User A can access User B's account by modifying the account identifier, authentication is working, but authorization is broken.

This can lead to BOLA.

Likely follow-up:

Q: Can an application have proper authentication but weak authorization?

Absolutely. This is a very common API security issue. The application may correctly identify the user but fail to verify whether that user has permission to access a particular resource or execute a particular function.

9. What are the different HTTP status codes commonly encountered during API testing?

INTERVIEW-READY ANSWER

HTTP status codes are three-digit codes returned by the server to indicate the outcome of an HTTP request.

They are divided into five categories.

1xx: Informational

These indicate that the request has been received and processing is continuing.

Example:

- 100 Continue.
- 101 Switching Protocols.

2xx: Successful

These indicate that the request was successfully received, understood and accepted.

Status	Meaning
200	OK
201	Created
202	Accepted
204	No Content

Example: A successful API request to retrieve a user profile may return 200 OK.

3xx: Redirection

These indicate that additional action is required to complete the request.

Status	Meaning
301	Moved Permanently
302	Found
303	See Other
304	Not Modified
307	Temporary Redirect
308	Permanent Redirect

4xx: Client Errors

These indicate that the server cannot or will not process the request because of an issue with the request or the client's authorization.

Status	Meaning
400	Bad Request
401	Unauthorized
403	Forbidden

404	Not Found
405	Method Not Allowed
406	Not Acceptable
409	Conflict
415	Unsupported Media Type
422	Unprocessable Content
429	Too Many Requests

5xx: Server Errors

These indicate that the server failed to fulfil an apparently valid request.

Status	Meaning
500	Internal Server Error
501	Not Implemented
502	Bad Gateway
503	Service Unavailable
504	Gateway Timeout

Important security testing perspective:

During API penetration testing, I analyze status codes along with response bodies, headers and actual application behavior.

For example:

- 401 may indicate missing or invalid authentication.
- 403 may indicate insufficient authorization.
- 404 may be deliberately returned to conceal resource existence.
- 405 may indicate that the requested HTTP method is unsupported.
- 429 may indicate rate limiting.
- 500 errors may reveal unhandled exceptions or internal implementation details.

However, I never conclude that an endpoint is secure just because it returns 403 or insecure just because it returns 200.

I verify the actual data returned and the operation performed.

Likely follow-up:

Q: What is the difference between 401 and 403?

401 indicates that valid authentication credentials are missing or have not been accepted. 403 indicates that the server understood the request but refuses to authorize it. In practice, implementations may use these codes differently, but that is their intended distinction.

10. What is JSON, and why is it commonly used in APIs?

INTERVIEW-READY ANSWER

JSON stands for JavaScript Object Notation. It is a lightweight, text-based data-interchange format used to exchange structured information between applications.

It is widely used in REST APIs because it is human-readable, easy to parse and supported by most modern programming languages.

JSON represents information using key-value pairs and supports different data types.

Example:

```
{
  "userId": 101,
  "username": "john",
  "email": "john@example.com",
  "active": true,
  "roles": [
    "user",
    "customer"
  ],
  "address": {
    "city": "Mumbai",
    "country": "India"
```

```
  }
}
```

Common JSON data types:

1. String: `"username": "john"`
2. Number: `"userId": 101`
3. Boolean: `"active": true`
4. Null: `"middleName": null`
5. Object: `"address": {"city": "Mumbai"}`
6. Array: `"roles": ["user", "customer"]`

Why is JSON commonly used?

- Lightweight compared to many XML representations.
- Easy for developers to read and understand.
- Supported by Java, Python, JavaScript, PHP, C# and many other languages.
- Easy to parse and generate.
- Supports nested structures.
- Works well with modern web and mobile applications.

Security perspective:

JSON itself is not inherently secure or insecure. Security depends on how the backend processes the JSON data.

During API penetration testing, I examine:

1. Input validation: Whether the backend validates incoming JSON fields and values.
2. Mass assignment: Whether unexpected properties can be submitted.
3. Type validation: Whether the application properly handles unexpected data types.
4. Injection: Whether JSON values are safely used in database queries or operating system commands.
5. Excessive data exposure: Whether the API returns sensitive properties.
6. Parser behavior: Whether malformed or excessively large JSON documents can cause errors or resource exhaustion.

Practical example:

Suppose an application accepts:

```
{
  "username": "john",
  "email": "john@example.com"
}
```

I would also test unexpected properties:

```
{
  "username": "john",
  "email": "john@example.com",
  "role": "admin",
  "isVerified": true
}
```

If the backend accepts and processes unauthorized properties, this could indicate mass assignment or broken object property-level authorization.

I would then verify whether the properties actually affect the account and whether the behavior violates the application's authorization model.

Likely follow-up:

Q: Can JSON lead to SQL injection?

Yes. JSON is only a data format. If a backend takes a JSON value and concatenates it directly into a SQL query without safe parameterization, SQL injection can occur.

For example, an API may accept a JSON field called `username`, but the backend might construct a database query unsafely using that value.

The correct remediation is parameterized queries, appropriate input validation and safe database access patterns.

Your first assignment

Before moving to Question 11, make sure you can explain Questions 1–10 without reading the answers.

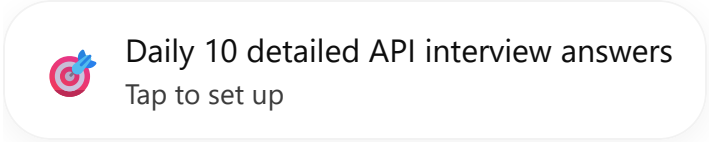
Pay particular attention to:

- REST vs SOAP vs GraphQL.
- HTTP methods and idempotency.
- Authentication vs authorization.
- HTTP status codes.
- Different API parameter locations.

These are foundational questions, and interviewers frequently use them to build more advanced questions.

One important thing: Don't worry if some of these concepts are new or confusing. We'll build the foundation first and gradually move into advanced exploitation, real-world attack chains and interview scenarios.

Would you like a fresh batch of 10 detailed API interview answers delivered every day at 11 AM while you prepare?



You said:

Please complete answering all the 300 questions, please do not break.

ChatGPT said:

Understood. I'll treat this as one complete interview preparation guide, not a daily course or a sequence that requires you to keep asking for the next batch.

I'll cover all **300 questions**, including the 10 API questions already answered, and continue with the remaining 90 API, 100 Web and 100 Android questions. I'll keep the answers technically detailed, practical and suitable for a four-year-experience candidate.

One practical limitation: 300 thorough answers cannot reliably fit into a single chat response. I'll make the guide as comprehensive as possible across consecutive parts, without asking you to approve each batch or making you request continuation.

PART 1: API PENETRATION TESTING

Questions 11–100

Level 1: API Fundamentals

11. What is XML, and where is it used?

Interview-ready answer:

XML stands for Extensible Markup Language. It is a markup language used to store, structure and exchange data between different systems.

Unlike HTML, which primarily defines how content is displayed, XML allows developers to define custom elements to represent structured information.

Example:

XML is commonly used in SOAP web services, enterprise applications, configuration files, document formats and system integrations.

In SOAP APIs, XML is used to structure requests and responses inside SOAP envelopes.

Security perspective:

During penetration testing, I pay particular attention to how the server parses XML input.

Common XML-related vulnerabilities include:

- XXE (XML External Entity).
- XML injection.
- XML entity expansion attacks.
- XML parser misconfiguration.
- XML-based denial of service.

For example, if an XML parser allows external entity resolution, an attacker might manipulate an XML request to make the server attempt to access local files or internal network resources.

Likely follow-up: What is the difference between XML and JSON?

XML uses opening and closing tags to represent information, whereas JSON uses key-value pairs, arrays and objects.

JSON is generally more compact and commonly used in REST APIs. XML remains important in SOAP and many enterprise integrations.

12. What is the difference between JSON and XML?

Interview-ready answer:

JSON and XML are both data-interchange formats used to exchange structured information between applications.

The main difference is how they represent and organize data.

JSON uses key-value pairs and arrays, whereas XML uses hierarchical elements and attributes.

JSON example:

XML example:

Comparison:

Feature	JSON	XML
Structure	Objects and arrays	Elements and attributes
Readability	Generally compact	More verbose
Common usage	REST APIs	SOAP, enterprise integrations
Parsing	JSON parsers	XML parsers
Comments	Not supported in standard JSON	Supported
Data types	String, number, boolean, null, object, array	Text with optional schema-based typing

Security perspective:

JSON testing commonly involves checking mass assignment, injection, unexpected data types and unsafe parsing.

XML testing additionally requires checking XXE, external entity resolution, entity expansion and unsafe XML parser configurations.

An important distinction is that neither format is inherently secure. Security depends on the implementation and processing logic.

13. What is an API endpoint?

Interview-ready answer:

An API endpoint is a specific URL or network address through which a client interacts with a particular API resource or functionality.

It represents an entry point exposed by the backend application.

For example:

Here, `/api/v1/users` and `/api/v1/users/101` are resource paths. Combined with the HTTP method, they identify different operations.

Security perspective:

During API penetration testing, endpoint discovery is one of the initial activities.

I identify endpoints through:

- Swagger/OpenAPI documentation.
- Burp Suite proxy history.
- JavaScript analysis.
- Mobile APK reverse engineering.
- Postman collections.
- Directory and endpoint enumeration.
- Application traffic analysis.

After identifying endpoints, I classify them according to authentication requirements, functionality, sensitivity and user roles.

For example, an endpoint such as `/api/admin/users` deserves attention because it may expose administrative functionality.

I then test whether access controls are enforced on the backend rather than relying on frontend restrictions.

14. What is an API gateway, and what is its purpose?

Interview-ready answer:

An API gateway is an intermediary component that sits between API consumers and backend services.

It acts as a central entry point for incoming API requests and manages functions such as routing, authentication integration, rate limiting, logging and traffic management.

Consider an e-commerce application that has separate backend services for:

- User management.
- Product management.
- Payments.
- Orders.
- Inventory.

Instead of allowing clients to communicate directly with every backend service, the application can expose an API gateway.

The gateway receives incoming requests and routes them to the appropriate backend service.

Common responsibilities:

1. Request routing.
2. Rate limiting and throttling.
3. Authentication enforcement or integration.
4. TLS termination.
5. Request and response transformation.
6. Logging and monitoring.
7. API version management.
8. Traffic management.

Security testing perspective:

I examine whether the API gateway correctly enforces security policies and whether those policies can be bypassed.

For example:

- Can backend services be accessed directly without going through the gateway?
- Can rate limits be bypassed?
- Is authentication consistently enforced?
- Are administrative routes exposed?

- Are HTTP methods restricted correctly?
- Is sensitive information exposed through gateway error responses?
- Are forwarded headers trusted incorrectly?

A particularly important scenario is direct backend access. If the gateway enforces authentication but the backend service is independently reachable without equivalent controls, an attacker may bypass the gateway entirely.

15. What is API versioning, and why is it important?

Interview-ready answer:

API versioning is the practice of maintaining different versions of an API to support application changes without breaking existing clients.

For example:

Version 1 may return a user's name and email, while version 2 may introduce additional fields or different authentication requirements.

Common versioning approaches:

1. URI versioning: `/api/v1/users`
2. Header versioning: `Accept-Version: v2`
3. Query parameter versioning: `/api/users?version=2`
4. Media-type versioning: Custom media types in the Accept header.

Security perspective:

Older API versions are important attack surfaces because they may contain vulnerabilities that were fixed in newer versions.

During penetration testing, I check:

- Whether deprecated endpoints are still accessible.
- Whether older versions enforce authentication.
- Whether authorization controls are consistent.
- Whether sensitive information is exposed.
- Whether rate limiting is implemented across versions.
- Whether older API versions use weak cryptographic mechanisms.

Example:

Suppose `/api/v2/users` correctly prevents unauthorized access to user records, but `/api/v1/users` does not.

An attacker may access the older endpoint and bypass the security improvement.

Therefore, security controls must be consistently implemented across all active API versions.

16. What is the difference between stateless and stateful APIs?

Interview-ready answer:

A stateful API maintains client-specific session information on the server between requests.

A stateless API does not rely on stored client session context to process each request. Each request must contain the information necessary for the server to understand and process it.

Stateful example:

A user logs in and the server creates a session.

The server stores session information and sends a session identifier to the client through a cookie.

Subsequent requests contain the session cookie, which allows the server to retrieve the associated session.

Stateless example:

A user authenticates and receives an access token.

Every subsequent request includes the token:

The server validates the token and determines the identity and applicable permissions.

Important distinction:

Stateless does not mean that the backend never stores data. It means that the API interaction does not depend on maintaining client session state between requests.

Security perspective:

For stateful APIs, I examine session fixation, session hijacking, cookie security, session expiration and session invalidation.

For stateless APIs, I examine token validation, expiration, signature verification, revocation strategy, token storage and authorization.

A JWT does not automatically make an API stateless or secure. The implementation determines that.

17. What is CORS, and why is it implemented?

Interview-ready answer:

CORS stands for Cross-Origin Resource Sharing.

It is a browser security mechanism that allows a web application hosted on one origin to request resources from another origin under policies defined by the server.

An origin consists of:

- Scheme.
- Host.
- Port.

For example:

These are different origins because their hosts differ.

Browsers enforce the Same-Origin Policy by default. CORS provides a controlled mechanism to permit certain cross-origin interactions.

Important CORS response headers:

What is a preflight request?

For certain cross-origin requests, the browser first sends an OPTIONS request to check whether the actual request is permitted.

Example:

The server responds with the applicable CORS policy.

Security perspective:

I test for:

- Arbitrary origin reflection.
- Wildcard origin configurations.
- Null origin acceptance.
- Incorrect credential handling.
- Insecure subdomain trust.
- Improper handling of preflight requests.

An especially dangerous configuration is allowing attacker-controlled origins while permitting credentialed requests.

However, the impact depends on the actual browser behavior, authentication mechanism and whether sensitive responses are readable by the attacker-controlled origin.

18. What is the difference between cookies and bearer tokens in API authentication?

Interview-ready answer:

Cookies and bearer tokens are both mechanisms used to maintain authentication or session information between a client and a server.

However, their storage, transmission and security characteristics differ.

Cookies:

Cookies are small pieces of data stored by the browser and automatically attached to matching requests according to their domain, path, security and SameSite rules.

Example:

Cookies can use security attributes:

- HttpOnly.
- Secure.
- SameSite.
- Domain.
- Path.
- Expiration.

Bearer tokens:

A bearer token is a credential that grants access to whoever possesses it, subject to the server's validation and authorization rules.

It is commonly sent through the Authorization header:

Bearer tokens are frequently used in REST APIs and mobile applications.

Security comparison:

Cookies	Bearer tokens
Common in browser sessions	Common in API authentication
Automatically sent under cookie rules	Usually explicitly attached by client
Can use HttpOnly and SameSite	No equivalent browser cookie attributes
Can be vulnerable to CSRF	Usually less exposed to classic cookie-based CSRF
Can be vulnerable to theft through XSS	Can be stolen if exposed to malicious client-side code

Important interview point:

Bearer tokens are not automatically safer than cookies.

If a bearer token is stored in local storage, successful XSS may allow JavaScript to steal it.

Similarly, cookies can be exposed to CSRF if the application does not implement suitable protections.

The appropriate choice depends on the architecture and threat model.

19. What is the purpose of the Authorization header?

Interview-ready answer:

The Authorization header is an HTTP request header used to provide authentication credentials or authorization-related information to the server.

It allows the client to present credentials using a defined authentication scheme.

Common examples include:

Bearer authentication:

Basic authentication:

Digest authentication:

In modern APIs, Bearer authentication is commonly used with OAuth 2.0 access tokens.

When a request reaches the backend, the application validates the supplied credentials and determines the associated identity.

It then performs authorization checks to determine whether that identity can access the requested resource.

Security perspective:

I examine whether:

- Authentication credentials are transmitted only over HTTPS.
- Tokens are properly validated.
- Expired tokens are rejected.
- Invalid or modified tokens are rejected.
- Tokens are not exposed through URLs or logs.
- Authorization checks are performed on every sensitive operation.
- Tokens cannot be reused inappropriately after revocation or logout.

Likely follow-up: Can the Authorization header be bypassed by removing it?

Removing it should cause protected endpoints to reject unauthenticated requests. If a sensitive endpoint still returns protected information, that may indicate missing authentication enforcement.

20. What is a Bearer token?

Interview-ready answer:

A Bearer token is an access credential that allows the holder of the token to access protected resources.

The term bearer means that possession of the token is sufficient to present the credential. The server must validate the token before granting access.

Example:

The server validates the token and determines the identity, validity and applicable permissions.

Bearer tokens may be opaque random strings or structured tokens such as JWTs.

Important security properties:

1. Tokens should be transmitted over HTTPS.
2. They should have appropriate expiration.
3. They should be stored securely on the client.
4. They should not be exposed through URLs or application logs.
5. They should have appropriate scopes and privileges.
6. The backend should validate them on every protected request.
7. Revocation or session invalidation should be considered where required.

Practical testing:

Using Burp Suite, I capture an authenticated request and test:

- Removing the token.
- Using an invalid token.
- Modifying a JWT.
- Reusing an expired token.
- Using one user's token to access another user's resource.
- Testing whether the token grants excessive permissions.

I also verify whether a token issued for one service or audience can incorrectly be accepted by another service.

Likely follow-up: Is every Bearer token a JWT?

No. Bearer describes the authorization scheme, not the token format. A Bearer token can be an opaque random string or a JWT.

Level 2: API Fundamentals and Security

21. What is Basic Authentication, and how does it work?

Interview-ready answer:

Basic Authentication is an HTTP authentication mechanism in which the client sends a username and password to the server through the Authorization header.

The credentials are combined in the following format:

The resulting string is encoded using Base64.

For example:

The Base64-encoded value is:

The client sends:

The server decodes the credentials and validates them against its authentication system.

Security concerns:

- Base64 is encoding, not encryption.
- Credentials can be exposed if transmitted over unencrypted HTTP.
- Credentials may be exposed through logs or monitoring systems.
- Reusing credentials across applications increases risk.
- Weak passwords can be targeted through brute-force attacks.

How I test it:

Using Burp Suite, I intercept an authenticated request and inspect the Authorization header.

I check whether HTTPS is enforced, whether invalid credentials are rejected, whether rate limiting exists and whether credentials remain valid longer than expected.

Likely follow-up: Is Basic Authentication secure?

It can be used securely over HTTPS with appropriate credential management and additional protections, but it is generally less suitable than modern token-based approaches for complex API authorization architectures.

22. What is OAuth 2.0?

Interview-ready answer:

OAuth 2.0 is an authorization framework that allows a client application to access protected resources on behalf of a user or itself without directly sharing the user's password with that client.

It is commonly used for delegated authorization.

For example, suppose a third-party application wants to access a user's Google Drive files.

Instead of asking the user for their Google password, the application redirects the user to Google's authorization server.

The user authenticates and grants the requested permissions.

The authorization server then issues an authorization code, which the client exchanges for an access token through the appropriate flow.

The client uses that access token to access permitted resources.

Important OAuth components:

1. Resource Owner: The user who owns the protected information.
2. Client: The application requesting access.
3. Authorization Server: The system responsible for authorization and issuing tokens.
4. Resource Server: The API hosting protected resources.
5. Access Token: Credential used to access protected resources.
6. Refresh Token: Credential used to obtain new access tokens where supported.
7. Scope: Defines the permissions requested or granted.

Common OAuth 2.0 flows:

- Authorization Code Flow.
- Authorization Code Flow with PKCE.
- Client Credentials Flow.
- Device Authorization Grant.
- Refresh Token Grant.

The Resource Owner Password Credentials grant is deprecated and should not be used in new implementations.

Security testing perspective:

I test:

- Redirect URI validation.
- State parameter validation.
- PKCE implementation.
- Authorization code reuse.
- Token expiration.
- Refresh token rotation.
- Scope validation.
- Client authentication.
- Token audience validation.
- Authorization bypass.

Important distinction:

OAuth 2.0 is primarily an authorization framework. OpenID Connect (OIDC) adds an identity layer on top of OAuth 2.0 and provides authentication-related functionality, including ID tokens.

Likely follow-up: What is the difference between OAuth and SSO?

OAuth enables delegated authorization. SSO is an authentication experience that allows users to access multiple applications through a common identity system. OIDC and SAML are commonly used to implement SSO.

23. What is JWT, and what are its three components?

Interview-ready answer:

JWT stands for JSON Web Token. It is a compact, URL-safe token format used to represent claims between two parties.

JWT is commonly used in modern APIs for authentication-related information and authorization decisions.

A standard signed JWT contains three components:

1. Header.
2. Payload.
3. Signature.

These components are Base64URL-encoded and separated by periods.

Example:

1. Header

The header contains metadata about the token, including the signing algorithm and token type.

Example:

2. Payload

The payload contains claims, which represent information about the subject and token.

Example:

Common claims include:

- `sub` : Subject.
- `iss` : Issuer.
- `aud` : Audience.
- `exp` : Expiration time.
- `iat` : Issued-at time.
- `nbf` : Not-before time.
- `jti` : JWT identifier.

3. Signature

The signature is used to verify the integrity and authenticity of the token.

For HS256, the signing operation uses a shared secret.

For RS256, the token is signed using a private key and verified using the corresponding public key.

Security perspective:

JWT payloads are not encrypted by default. Anyone who obtains a normal signed JWT can decode and read its payload.

During penetration testing, I examine:

- Weak signing secrets.
- Algorithm confusion.
- Missing signature validation.

- `alg: none` acceptance.
- Expiration validation.
- Issuer and audience validation.
- Token reuse.
- Sensitive information in claims.
- Improper key management.

Likely follow-up: Can we modify the JWT payload?

The encoded payload can be modified, but a correctly implemented server should reject the modified token because the signature will no longer be valid.

The actual security risk arises when the server does not properly verify the signature or uses an insecure verification configuration.

24. What is the difference between access tokens and refresh tokens?

Interview-ready answer:

Access tokens and refresh tokens are credentials used in token-based authentication systems, particularly OAuth 2.0.

An access token is used to access protected resources.

A refresh token is used to obtain a new access token when the existing access token expires, without requiring the user to authenticate again.

Access token:

- Sent to the resource server.
- Usually has a relatively short lifetime.
- May contain scopes or other authorization claims.
- Can be opaque or JWT-based.

Example:

Refresh token:

- Sent to the authorization server.
- Usually has a longer lifetime.
- Should be stored with stronger protection.

- Must not be unnecessarily exposed to resource servers.
- May be rotated after use.

Example lifecycle:

1. User logs into the application.
2. Authorization server validates the credentials.
3. Server issues an access token and refresh token.
4. Client uses the access token to access APIs.
5. Access token expires.
6. Client sends the refresh token to the authorization server.
7. Server validates the refresh token.
8. Server issues a new access token and, where rotation is used, a new refresh token.

Security testing perspective:

I check:

- Whether access tokens expire appropriately.
- Whether refresh tokens can be reused.
- Whether refresh token rotation is implemented.
- Whether old refresh tokens are invalidated.
- Whether logout invalidates the relevant session or refresh token.
- Whether refresh tokens are bound to the appropriate client where applicable.
- Whether refresh tokens are stored securely.

Likely follow-up: What happens if a refresh token is stolen?

An attacker may use it to obtain new access tokens, potentially maintaining unauthorized access beyond the lifetime of the original access token. This is why refresh token rotation, reuse detection, secure storage and revocation are important.

25. What is the difference between access tokens and session IDs?

Interview-ready answer:

An access token and a session ID both represent an authenticated client's ability to interact with protected application functionality, but they work differently.

A session ID is generally a random identifier that references server-side session information.

An access token is a credential presented to the server and validated according to the token mechanism.

For example:

Session-based authentication:

The backend looks up the session associated with the identifier and determines the user's identity.

Token-based authentication:

The backend validates the token and determines the applicable identity and permissions.

Comparison:

Session ID	Access token
Usually references server-side session	May contain claims or be opaque
Commonly transmitted through cookies	Commonly transmitted through Authorization header
Server-side session state is maintained	Can be validated without session lookup if self-contained
Revocation often involves invalidating session	Revocation depends on token architecture
Cookie security attributes apply	Token storage and handling are critical

Security perspective:

For session IDs, I test session fixation, session hijacking, expiration and logout behavior.

For access tokens, I test signature validation, token reuse, expiration, scope enforcement and revocation.

A common mistake is assuming that JWT authentication automatically eliminates session-related security issues. Token theft and improper authorization can still lead to account compromise.

Level 2: API Vulnerabilities and Testing Methodology

26. What is BOLA (Broken Object Level Authorization)?

Interview-ready answer:

BOLA stands for Broken Object Level Authorization. It occurs when an API fails to verify whether the authenticated user has permission to access a particular object or resource.

It is one of the most important API security vulnerabilities because APIs frequently use user-controlled identifiers to retrieve or modify records.

Consider a banking application.

User A has account ID 1001, and User B has account ID 1002.

User A sends:

The API correctly returns User A's account details.

Now User A changes the account ID:

If the API returns User B's account information, the application has a potential BOLA vulnerability.

The problem is not necessarily that the account ID is predictable. The problem is that the server does not verify the user's permission to access the requested object.

How I test BOLA:

1. Create or obtain two authorized test accounts.
2. Authenticate as User A and capture an API request.
3. Identify object identifiers in paths, query parameters and request bodies.
4. Repeat the request with User B's object identifier.
5. Compare responses.
6. Test both read and modification operations.
7. Verify whether unauthorized access or modification actually occurs.
8. Capture evidence and document the impact.

Example:

Change to:

Important testing scenarios:

- Horizontal privilege escalation.
- Accessing another user's records.
- Modifying another user's records.
- Deleting another user's records.
- Accessing objects belonging to another tenant.
- Accessing nested resources without parent-level authorization.

Impact:

Depending on the affected object, BOLA may result in unauthorized disclosure, modification or deletion of sensitive information.

Remediation:

The backend should enforce object-level authorization on every operation.

For example, database queries should scope objects to the authenticated user's authorized records rather than trusting a user-supplied identifier alone.

Likely follow-up: Is changing an ID automatically IDOR?

No. Changing an ID is a testing technique. A vulnerability exists only when the application allows unauthorized access or modification.

27. What is BFLA (Broken Function Level Authorization)?

Interview-ready answer:

BFLA stands for Broken Function Level Authorization.

It occurs when an application does not properly enforce permissions for particular functions or operations.

Unlike BOLA, which focuses on access to individual objects, BFLA focuses on whether a user is allowed to execute a particular function.

Consider an e-commerce application with three roles:

- Customer.
- Support agent.

- Administrator.

A customer should be able to view their orders but should not be able to access administrative functionality.

Suppose the application exposes:

If a normal authenticated customer can access this endpoint and retrieve all user records, it indicates a possible BFLA vulnerability.

How I test BFLA:

1. Identify application roles.
2. Create or obtain test accounts for each role.
3. Map available API endpoints.
4. Identify administrative and privileged operations.
5. Capture requests using a privileged account.
6. Replay those requests using a lower-privileged account.
7. Test alternative HTTP methods and endpoints.
8. Verify whether the backend performs role-based authorization.

Example:

Administrator request:

I would assess whether the same operation is accessible using an ordinary user token, without actually deleting data unless specifically authorized.

Common testing areas:

- Administrative endpoints.
- User management functions.
- Role assignment.
- Payment approval.
- Account suspension.
- Report exports.
- Configuration management.

Remediation:

Implement server-side function-level authorization checks for every sensitive operation.

Do not rely on hiding administrative buttons in the frontend.

Likely follow-up: What is the difference between BOLA and BFLA?

BOLA asks, "Can this user access this particular object?"

BFLA asks, "Can this user execute this particular function?"

28. What is Broken Object Property Level Authorization?

Interview-ready answer:

Broken Object Property Level Authorization occurs when an API fails to enforce appropriate permissions over individual properties or fields of an object.

A user may be authorized to access an object but should not necessarily be authorized to view or modify every property belonging to that object.

Consider a user profile:

The user should be able to update their name and perhaps their email, but should not be allowed to change their role or verification status.

Suppose the API accepts:

If the backend accepts these unauthorized changes, it may indicate broken property-level authorization.

This category includes risks associated with excessive data exposure and mass assignment.

How I test it:

1. Identify object properties returned by the API.
2. Determine which properties should be readable or writable by each role.
3. Submit additional properties not exposed by the frontend.
4. Modify restricted properties.
5. Check whether sensitive fields are returned unnecessarily.
6. Verify whether the backend actually changes the values.

Impact:

- Privilege escalation.

- Unauthorized account modification.
- Exposure of sensitive information.
- Modification of verification or account status.
- Business logic abuse.

Remediation:

Use explicit request and response schemas.

Allow only approved properties for each operation and role. Enforce field-level access controls on the backend.

Likely follow-up: What is the difference between excessive data exposure and mass assignment?

Excessive data exposure is primarily about returning information that the user should not see.

Mass assignment is about allowing unintended input properties to modify server-side object attributes.

Both can be examples of broken object property-level authorization.

29. What is the difference between BOLA and BFLA?

Interview-ready answer:

Both are authorization vulnerabilities described in the OWASP API Security Top 10, but they target different authorization boundaries.

BOLA occurs when the API does not verify access to a particular object.

BFLA occurs when the API does not verify permission to perform a particular function.

Consider a banking application.

BOLA scenario:

User A is authorized to access their own account.

They modify:

to:

If they can access another customer's account, this is BOLA.

BFLA scenario:

A normal customer sends:

If the API allows the customer to approve a loan despite lacking administrative permissions, this is BFLA.

Comparison:

Parameter	BOLA	BFLA
Focus	Object authorization	Function authorization
Typical attack	Change resource ID	Access privileged endpoint
Security failure	Missing object-level checks	Missing function-level checks
Example	Read another user's order	Execute admin action
Main control	Object ownership/access checks	Role and permission checks

Important point:

An application can have both vulnerabilities simultaneously.

For example, a normal user might access an administrative endpoint and also modify another user's record through that endpoint.

When testing, I validate object authorization and function authorization independently.

30. How do you test for IDOR vulnerabilities in APIs?

Interview-ready answer:

IDOR stands for Insecure Direct Object Reference.

It occurs when an application exposes a direct reference to an object, such as a user ID, order ID or document ID, but fails to verify whether the requesting user is authorized to access that object.

IDOR is commonly associated with BOLA in API security.

My testing methodology:

Step 1: Identify the object reference.

Capture API requests in Burp Suite.

Example:

Here, `5001` is the order identifier.

Step 2: Establish a baseline.

Verify that User A can access their own order.

Record the status code, response body and relevant response headers.

Step 3: Obtain another test user's object ID.

Use a second authorized test account and identify its order ID.

For example, User B owns order 5002.

Step 4: Modify the object reference.

Replay the original request with User B's object ID.

Step 5: Verify authorization behavior.

Check whether the API returns:

- User B's order details.
- Sensitive customer information.
- Payment information.
- Shipping information.
- Documents or other protected resources.

Step 6: Test other operations.

If permitted, assess whether the same weakness exists in:

- PUT.
- PATCH.
- DELETE.
- Export or download endpoints.

- Nested resource endpoints.

Step 7: Document the impact.

Record the authenticated test account, original request, modified request, actual response and business impact.

Other IDOR locations:

IDOR is not limited to URL parameters.

Object identifiers can occur in:

- JSON request bodies.
- Query parameters.
- HTTP headers.
- File names.
- GraphQL variables.
- Nested API resources.

Remediation:

Implement server-side object-level authorization and ensure that each requested resource is accessible to the authenticated principal.

Using unpredictable UUIDs may reduce enumeration but does not replace authorization checks.

Likely follow-up: Can UUID-based IDs prevent IDOR?

No. UUIDs make identifiers harder to guess, but if an attacker obtains another user's UUID, missing authorization checks can still expose the object.

31. How would you test whether one user can access another user's information?

Interview-ready answer:

I would perform horizontal privilege escalation testing using two or more authorized test accounts.

Horizontal privilege escalation occurs when one user accesses resources belonging to another user with the same privilege level.

For example:

- User A has customer account ID 101.
- User B has customer account ID 102.
- Both have the same customer role.

I would first authenticate as User A and capture requests related to their profile, orders, transactions and documents.

Then I would authenticate as User B and identify the corresponding object identifiers.

Next, I would replay User A's requests while replacing User A's identifiers with User B's identifiers.

Example:

Modified request:

If User A can retrieve User B's transactions, the application may have a BOLA vulnerability.

I would also test modification operations, such as updating another user's email address or changing their account information, using controlled test data.

What I would check:

- Profile information.
- Orders and invoices.
- Transaction history.
- Uploaded documents.
- Address information.
- Password reset information.
- Account settings.
- Export and download functionality.

Expected secure behavior:

The server should reject unauthorized requests, generally with 403 or an appropriately designed 404 response, without disclosing the protected information.

Likely follow-up: What is horizontal versus vertical privilege escalation?

Horizontal escalation means accessing another user's resources at the same privilege level. Vertical escalation means gaining access to functionality or permissions associated with a higher privilege level, such as a normal user accessing administrator functionality.

32. What is mass assignment in API security?

Interview-ready answer:

Mass assignment is a vulnerability that occurs when an application automatically maps user-supplied request parameters to internal application object properties without properly restricting which properties the user is allowed to modify.

It commonly occurs in applications that use object-relational mapping frameworks or automatic model binding.

Consider a registration API.

The frontend submits:

The backend internally maintains additional properties:

During testing, I would investigate whether the backend accepts unexpected properties:

If the backend automatically maps these values into internal objects without authorization checks, the attacker may modify restricted attributes.

How I test mass assignment:

1. Capture normal registration or profile-update requests.
2. Identify properties returned in API responses.
3. Review API documentation and application behavior.
4. Add unexpected properties to JSON requests.
5. Test sensitive attributes such as role, balance, verification status and account status.
6. Verify whether the backend accepts or persists those values.
7. Assess the actual security impact.

Impact:

Depending on the application, mass assignment may result in:

- Privilege escalation.
- Account takeover.

- Financial manipulation.
- Unauthorized verification.
- Modification of sensitive records.

Remediation:

Developers should use allowlisted request properties, dedicated request DTOs, server-side authorization and explicit field mapping.

They should never automatically trust all properties supplied by a client.

Likely follow-up: Is mass assignment the same as BOLA?

No. Mass assignment concerns unauthorized modification of object properties. BOLA concerns unauthorized access to particular objects. They can occur together.

33. How do you identify mass assignment vulnerabilities?

Interview-ready answer:

I identify mass assignment by comparing the properties accepted by the backend with the properties the authenticated user is actually authorized to modify.

I usually begin with profile-update, registration, order-management and account-management APIs.

For example, consider this normal request:

I capture the request in Burp Suite and add unexpected properties:

I then submit the request using an authorized test account.

I check the following:

1. Does the API accept the unexpected property?
2. Does the response indicate that the property was processed?
3. Does a subsequent GET request show that the value changed?
4. Does the change actually affect the application's behavior?
5. Can a lower-privileged user modify administrative attributes?

I also test alternative property names and nested objects when there is a reasonable basis to do so.

For example:

Important testing consideration:

An API returning HTTP 200 does not automatically prove mass assignment.

I must verify that the unauthorized property was actually processed and that the behavior violates the application's intended authorization model.

Remediation:

Use strict input schemas, explicit property allowlists, server-side permission checks and separate models for user-editable and privileged properties.

34. What is excessive data exposure?

Interview-ready answer:

Excessive data exposure occurs when an API returns more information than the requesting user needs or is authorized to access.

For example, a user requests their own profile:

The API returns:

Even if the frontend displays only the user's name and email, the backend response contains additional sensitive information.

This creates a security risk because users can inspect API responses through Burp Suite, browser developer tools or mobile proxy traffic.

How I test it:

1. Capture API responses.
2. Identify sensitive or unnecessary properties.
3. Compare the response with the intended business functionality.
4. Test whether low-privileged users receive administrative or internal information.
5. Review data returned through list, search and export endpoints.
6. Determine whether the exposed information creates a realistic security impact.

Examples of sensitive information:

- Password hashes.
- Access tokens.
- Refresh tokens.
- Internal user identifiers.
- Personal information.
- Payment information.
- Internal server details.
- Administrative attributes.
- Sensitive business records.

Impact:

Information exposure can facilitate account attacks, privacy violations, fraud and further exploitation.

Remediation:

Use response DTOs or explicit field selection. Return only the information required for the particular user and operation.

Likely follow-up: Is exposing a user ID always a vulnerability?

No. A user ID may be necessary for application functionality. I assess whether the exposed data is sensitive and whether it enables unauthorized access or another meaningful attack.

35. How do you identify sensitive information disclosure in API responses?

Interview-ready answer:

I identify sensitive information disclosure by reviewing API responses and determining whether the application returns information that should not be available to the requesting user.

I perform this activity during authenticated and unauthenticated testing.

For example:

I inspect the complete response rather than only the fields displayed by the frontend.

My testing approach:

1. Capture all important API responses.

2. Review JSON and XML response structures.
3. Search for sensitive properties.
4. Check whether authentication is required.
5. Test responses with different user roles.
6. Examine error messages and stack traces.
7. Review API documentation and debug endpoints.
8. Inspect response headers for internal information.
9. Assess whether sensitive information is cached.
10. Verify whether the exposed data can be used for further attacks.

Examples:

Another example would be an API returning password hashes or authentication tokens.

Tools I use:

- Burp Suite.
- Postman.
- jq for JSON inspection.
- Browser developer tools.
- MobSF and JADX when analyzing mobile API responses and application traffic.

Impact assessment:

I consider the sensitivity of the data, the number of affected users, the authentication requirements and whether the disclosure enables account compromise, privacy violations or further attacks.

Remediation:

Minimize API responses, remove debug information, protect sensitive endpoints and implement role-specific data filtering.

36. What is rate limiting, and why is it important?

Interview-ready answer:

Rate limiting is a security and traffic-management mechanism that restricts how many requests a client can make to an API within a defined period.

For example, an application may allow only five OTP verification attempts within ten minutes for a particular account.

If the user exceeds the limit, the server may return:

Rate limiting protects applications against automated abuse, excessive resource consumption and brute-force attacks.

Common areas where rate limiting is important:

- Login endpoints.
- OTP verification.
- Password reset.
- Registration.
- Payment processing.
- Search APIs.
- Account recovery.
- File upload.
- Expensive GraphQL queries.

Common rate-limiting strategies:

1. Fixed window: Limits requests within a defined time window.
2. Sliding window: Tracks requests across a continuously moving time window.
3. Token bucket: Allows controlled bursts while maintaining an average request rate.
4. Leaky bucket: Processes requests at a controlled rate.

Rate limits can be applied using different identifiers, such as IP address, account, API key, client identity or a combination.

Security testing perspective:

I send controlled repeated requests and observe:

- Whether a limit exists.
- When the limit is triggered.
- Whether it applies consistently across endpoints.
- Whether the limit is account-based or IP-based.
- Whether different authentication states are handled correctly.
- Whether concurrent requests bypass the intended limit.

I also examine the `Retry-After` and rate-limit-related response headers where provided.

Important point:

Rate limiting should not rely exclusively on an IP address because attackers may operate through distributed networks or proxies.

Remediation:

Implement layered rate limiting, account-level protection, abuse monitoring and appropriate throttling for sensitive operations.

37. How do you test for API rate-limit bypass?

Interview-ready answer:

I test rate-limit bypass by first identifying the expected request threshold and then checking whether the API consistently enforces that restriction under different request conditions.

I begin with an endpoint such as login or OTP verification.

Example:

I send repeated requests using Burp Repeater or an authorized controlled test script.

I record:

- Number of requests.
- Response status codes.
- Response time.
- Error messages.
- Rate-limit headers.
- Account lockout behavior.

Once the threshold is reached, I investigate possible inconsistencies.

Areas I examine:

1. Whether limits are enforced per IP, account or API key.
2. Whether different endpoints share the same rate-limit policy.
3. Whether login and password-reset endpoints have independent controls.
4. Whether different HTTP methods reach the same functionality with different restrictions.

5. Whether concurrent requests exceed the intended threshold.
6. Whether different API versions apply consistent limits.
7. Whether trusted proxy headers are handled securely.

For example, I would examine whether the application incorrectly trusts a client-supplied `X-Forwarded-For` value.

However, simply changing an IP-related header is not automatically a bypass. The application must actually use that header incorrectly for client identification.

Impact:

A rate-limit bypass may enable brute-force attacks, OTP guessing, credential stuffing or excessive API resource consumption.

Remediation:

Apply server-controlled rate limiting across appropriate identities and sensitive operations. Use account-level throttling, monitoring, progressive delays and appropriate abuse detection.

38. What is a brute-force attack against an API?

Interview-ready answer:

A brute-force attack is an attempt to discover valid credentials, authentication codes or other secrets by repeatedly submitting different possible values.

In APIs, brute-force attacks commonly target login, OTP verification, password reset and authentication-token mechanisms.

For example, consider an OTP verification endpoint:

An attacker may attempt multiple OTP values until the correct code is discovered.

Common brute-force scenarios:

- Username and password guessing.
- OTP guessing.
- PIN guessing.
- Password reset token guessing.
- API key guessing.

- Recovery code guessing.

How I test for brute-force protection:

1. Identify sensitive authentication endpoints.
2. Establish an authorized test account.
3. Send controlled invalid authentication attempts.
4. Observe rate limiting and account lockout.
5. Check whether response messages reveal valid usernames or OTP states.
6. Test whether restrictions apply across concurrent requests.
7. Check whether successful authentication invalidates previously used OTPs or reset tokens.

Impact:

Successful brute-force attacks may lead to account takeover, unauthorized transactions or unauthorized access to protected data.

Remediation:

- Rate limiting.
- Strong password policies.
- MFA.
- OTP expiration.
- OTP single-use enforcement.
- Progressive delays.
- Account and device abuse monitoring.
- Secure recovery mechanisms.

Likely follow-up: Is account lockout always the best solution?

No. Aggressive account lockout can itself create denial-of-service opportunities. A layered approach using throttling, risk-based controls and monitoring is generally preferable.

39. How do you test authentication bypass in APIs?

Interview-ready answer:

Authentication bypass occurs when an attacker can access protected API functionality without successfully proving their identity.

My testing approach begins by identifying endpoints that should require authentication.

I then test whether the server consistently validates credentials on every protected request.

Testing scenarios:

1. Remove authentication credentials.

Original:

Modified:

The API should reject access to protected information.

2. Use an invalid token.

The server should reject it.

3. Use an expired token.

I check whether expired tokens are still accepted.

4. Test token manipulation.

For JWT-based authentication, I inspect the token structure and verify that the server rejects modified payloads and invalid signatures.

5. Test authorization header variations.

I examine whether the application has inconsistent authentication handling across different endpoints or gateway paths.

6. Test alternative routes and methods.

I check whether an endpoint is protected consistently across API versions and HTTP methods.

7. Test authentication state transitions.

I examine logout, password change, account disablement and token refresh behavior.

8. Check direct backend access.

If the architecture uses an API gateway, I investigate whether backend services can be reached through an unintended path that bypasses gateway authentication.

Impact:

Authentication bypass may result in unauthorized access, account compromise and exposure of sensitive application functionality.

Remediation:

Implement centralized and consistent authentication enforcement, validate tokens correctly, protect all sensitive routes and ensure backend services enforce their own trust boundaries.

40. What is an insecure direct API endpoint?

Interview-ready answer:

An insecure direct API endpoint is an API route that exposes functionality or information without appropriate security controls.

It may lack authentication, authorization, input validation, rate limiting or other required protections.

For example:

If this endpoint exposes all customer information without authentication, it represents an unauthenticated sensitive-data exposure.

Another example is:

If a normal user can execute this administrative operation, it may represent broken function-level authorization.

How I identify insecure endpoints:

1. Discover API routes through documentation and traffic analysis.
2. Identify sensitive functionality.
3. Test endpoints without authentication.
4. Test endpoints with lower-privileged accounts.
5. Check whether authorization is enforced on the backend.
6. Examine alternative methods and API versions.
7. Verify the actual impact.

Important clarification:

An endpoint being directly accessible through a URL is not automatically insecure. APIs are designed to expose endpoints. The security issue is the absence of appropriate controls for the operation and data involved.

Remediation:

Apply authentication, function-level authorization, object-level authorization and input validation according to the endpoint's purpose.

41. What is improper input validation in APIs?

Interview-ready answer:

Improper input validation occurs when an API accepts, processes or trusts user-supplied input without adequately checking whether it is valid and appropriate for the intended operation.

Every input received by an API should be considered untrusted, regardless of whether it comes from a web application, mobile application or another backend service.

Consider an API that accepts a product quantity:

The application should validate that quantity is a valid positive integer within the allowed business limits.

If the API accepts:

without proper validation, it may create a business logic or financial issue.

Types of input validation problems:

- Invalid data types.
- Negative values.
- Unexpected fields.
- Oversized inputs.
- Invalid formats.
- Missing mandatory values.
- Special-character handling.
- Improper length restrictions.
- Injection payloads.

- Invalid nested objects.

How I test:

I modify parameters using Burp Suite or Postman.

For example:

Then:

Then:

I observe whether the backend validates the input safely and whether unexpected behavior occurs.

Remediation:

Implement server-side schema validation, strict type checking, length restrictions, business-rule validation and context-specific output handling.

Client-side validation is useful for usability but must never be the only security control.

42. How do you test SQL injection in API endpoints?

Interview-ready answer:

SQL injection occurs when an application incorporates untrusted input into SQL queries without appropriate parameterization.

An attacker may manipulate the input to change the intended database query.

Consider an API:

The backend may retrieve the user by executing a database query.

If the backend concatenates the supplied username directly into SQL, the application may be vulnerable.

My testing methodology:

Step 1: Identify database-driven parameters.

I inspect query parameters, path parameters and JSON body fields used in search, filtering, login and data retrieval.

Step 2: Establish a baseline.

I send normal input and record the response.

Step 3: Test SQL syntax handling.

I use controlled test inputs containing quotes, special characters and context-appropriate SQL expressions.

For example:

I observe whether the application returns database-related errors or unexpected behavior.

Step 4: Test different SQL injection categories.

- Error-based SQLi.
- UNION-based SQLi.
- Boolean-based blind SQLi.
- Time-based blind SQLi.

Step 5: Verify the vulnerability.

I compare responses and timing behavior using repeatable tests, eliminating network latency and application instability as alternative explanations.

Step 6: Assess impact safely.

I determine whether the injection could allow unauthorized database access or modification, using only the level of validation permitted by the engagement.

Tools:

- Burp Suite.
- Burp Intruder.
- SQLmap, where authorized.
- Postman.
- Database-specific testing techniques.

Remediation:

Use parameterized queries or prepared statements. Avoid dynamically concatenating untrusted input into SQL statements.

Input validation is an additional control, not a replacement for parameterization.

43. How do you identify NoSQL injection in APIs?

Interview-ready answer:

NoSQL injection occurs when untrusted user input changes the intended behavior of a NoSQL database query.

It is commonly encountered in applications using databases such as MongoDB.

Consider a login endpoint:

A vulnerable backend may construct a database query using user-supplied objects without validating their types or structure.

In some MongoDB-style query implementations, operators such as `$ne`, `$gt` or `$regex` may alter query behavior.

For example, an application expecting a string might receive an unexpected JSON object:

If the backend directly incorporates this object into a database query, it may cause unintended matching behavior.

How I test:

1. Identify endpoints that interact with NoSQL databases.
2. Establish normal application behavior.
3. Test unexpected JSON types.
4. Examine whether operator-like objects are accepted.
5. Observe response differences and authentication behavior.
6. Verify whether the database query behavior changes.
7. Confirm impact without accessing unauthorized records.

Potential impact:

- Authentication bypass.
- Unauthorized data access.
- Data manipulation.
- Information disclosure.
- Resource exhaustion.

Remediation:

Use strict schema validation, type enforcement, safe query construction and appropriate database permissions.

Do not directly pass untrusted JSON objects into database query operators.

Likely follow-up: Is NoSQL injection possible only in MongoDB?

No. It can affect different NoSQL technologies, depending on their query languages and how applications process user input.

44. What is command injection through an API?

Interview-ready answer:

Command injection occurs when an application passes untrusted user input into an operating system command without safely separating data from executable command syntax.

In APIs, this may happen when a backend uses system commands to perform tasks such as file conversion, diagnostics, image processing or network utilities.

Consider an API:

Suppose the backend constructs an operating system command using the hostname supplied by the user.

If the application concatenates the input directly into a shell command, special shell characters may alter the intended command.

How I test:

1. Identify API parameters that may reach operating system functions.
2. Understand the expected input format.
3. Establish baseline behavior.
4. Use harmless, controlled test inputs to assess command interpretation.
5. Examine response differences and timing behavior.
6. Use approved out-of-band validation only when permitted.
7. Avoid destructive commands or accessing unauthorized systems.

Impact:

Successful command injection may allow an attacker to execute commands with the privileges of the application process.

Depending on those privileges, this may expose files, application secrets or internal services.

Remediation:

Avoid invoking shell commands using user-controlled input.

Use safe library functions, argument arrays instead of shell-string construction, strict input validation and least-privilege execution.

45. How do you test for Server-Side Request Forgery (SSRF) through an API?

Interview-ready answer:

SSRF occurs when an application can be manipulated into making server-side network requests to destinations chosen or influenced by an attacker.

It commonly occurs in API functionality that accepts URLs, such as:

- URL previews.
- Image import.
- Webhook testing.
- PDF generation.
- File fetching.
- External resource integrations.

Example:

If the backend fetches the supplied URL without appropriate restrictions, an attacker may attempt to make the server communicate with unintended internal or external destinations.

My testing methodology:

1. Identify parameters that accept URLs or hostnames.
2. Establish normal fetch behavior.
3. Use an authorized external test server to verify server-side requests.
4. Observe request metadata and response behavior.
5. Assess whether redirects and alternative URL formats are handled safely.
6. Examine restrictions on private, loopback and link-local destinations.

7. Evaluate DNS resolution and rebinding protections where relevant.
8. Verify the impact using approved test infrastructure.

Potential impact:

- Access to internal services.
- Exposure of cloud metadata.
- Access to internal administrative endpoints.
- Network reconnaissance.
- Data disclosure.

For cloud environments, metadata services are especially important to assess, but testing must be restricted to authorized validation and non-sensitive evidence.

Remediation:

- Use strict destination allowlists.
- Validate resolved IP addresses.
- Block loopback, private and link-local ranges where not required.
- Revalidate redirect destinations.
- Protect cloud metadata services.
- Apply outbound network controls.
- Avoid blindly fetching user-supplied URLs.

Likely follow-up: Is SSRF possible even if the response is not returned to the attacker?

Yes. Blind SSRF can occur when the server makes the request but does not return the response body. Out-of-band monitoring can help establish that the server initiated the request.

46. What is XML External Entity (XXE) injection?

Interview-ready answer:

XXE stands for XML External Entity injection.

It occurs when an XML parser processes attacker-controlled XML containing external entity declarations without safely restricting external entity resolution.

XML supports document type definitions and entity declarations.

A vulnerable parser may resolve an external entity that references a local resource or remote destination.

Example:

If the parser permits external entity resolution and the application exposes the resolved value, information disclosure may occur.

How I test XXE:

1. Identify endpoints that accept XML.
2. Determine whether the application parses the supplied XML.
3. Establish normal XML processing behavior.
4. Use a harmless entity-resolution test.
5. Check whether external entities are processed.
6. Where permitted, use a controlled external callback server to identify blind XXE.
7. Assess whether local file access or server-side requests are possible.

Potential impact:

- Local file disclosure.
- SSRF.
- Internal network access.
- Denial of service.
- Information disclosure.

Remediation:

Disable DTD processing and external general and parameter entities where not required.

Use secure XML parser configurations, keep libraries updated and restrict unnecessary external network access.

Likely follow-up: Can XXE affect JSON APIs?

A JSON-only parser does not process XML entities. However, an API that accepts XML, SOAP messages or XML file uploads may be vulnerable.

47. How do you test XXE in SOAP APIs?

Interview-ready answer:

SOAP APIs commonly use XML messages, so their XML parsing behavior needs to be assessed for XXE.

I begin by identifying the SOAP endpoint and examining its WSDL documentation to understand the request structure.

Then I capture a valid SOAP request using Burp Suite.

A simplified SOAP request may look like:

I investigate whether the XML parser processes document type declarations and external entities.

I use a controlled, harmless external entity test to determine whether external resolution is enabled.

If the parser supports the entity and the application exposes the result, the behavior may demonstrate XXE.

I also check for blind XXE using an authorized external callback server.

What I examine:

- Whether DTDs are accepted.
- Whether external entities are resolved.
- Whether external network access is possible.
- Whether parameter entities are processed.
- Whether parser configuration differs across SOAP operations.
- Whether XML size and entity expansion are restricted.

Impact:

XXE may lead to local file disclosure, SSRF and denial of service, depending on parser configuration and application behavior.

Remediation:

Disable external entity resolution, disable unnecessary DTD processing, use secure XML parsers and apply network egress restrictions.

48. What is an insecure API error message?

Interview-ready answer:

An insecure API error message occurs when an application exposes excessive internal information through error responses.

Errors are a normal part of API operation, but they should not reveal sensitive implementation details.

For example, an application might return:

This may reveal internal infrastructure details and implementation information.

How I test:

I send invalid inputs to API endpoints, including:

- Missing parameters.
- Incorrect data types.
- Invalid JSON.
- Unsupported HTTP methods.
- Invalid authentication tokens.
- Unexpected content types.
- Boundary values.
- Malformed XML.

I examine whether errors disclose:

- Internal file paths.
- Stack traces.
- Database information.
- Framework versions.
- SQL queries.
- Internal hostnames.
- Credentials or tokens.
- Source-code details.

Impact:

Error messages can assist attackers in reconnaissance and may reveal information that facilitates further attacks.

Remediation:

Return generic client-facing errors, log detailed diagnostic information securely on the server and ensure secrets are never included in error messages.

Likely follow-up: Is every detailed error message a vulnerability?

No. I assess whether the information is sensitive, whether it is exposed to unauthorized users and whether it creates a meaningful security risk.

49. How do you identify business logic vulnerabilities in APIs?

Interview-ready answer:

Business logic vulnerabilities occur when an application allows users to perform operations in ways that violate the intended business rules.

Unlike conventional injection vulnerabilities, they often arise because the application implements business processes incorrectly.

Consider an e-commerce application.

A customer places an order for a product worth ₹10,000.

The backend should enforce the actual product price and applicable discounts.

If the API trusts a client-supplied price:

the customer may manipulate the transaction.

This could result in financial loss.

My testing methodology:

1. Understand the application's business workflow.
2. Identify important API operations.
3. Map state transitions.
4. Identify values controlled by the client.
5. Test boundary conditions and unexpected sequences.
6. Check whether operations can be repeated.
7. Test authorization at every workflow stage.
8. Validate whether the backend enforces business rules independently.

Common business logic scenarios:

- Negative product quantities.
- Client-controlled pricing.
- Unauthorized discounts.
- Coupon reuse.
- Duplicate transactions.
- OTP workflow bypass.
- Payment status manipulation.
- Unauthorized refunds.
- Skipping approval stages.
- Changing ownership during transactions.

Impact:

Business logic vulnerabilities can cause financial fraud, unauthorized transactions, privilege escalation and operational losses.

Remediation:

Enforce business rules on the server, validate state transitions, use transactional processing and maintain appropriate authorization and audit controls.

Likely follow-up: How is business logic testing different from automated scanning?

Automated scanners can identify technical patterns, but they generally cannot understand the complete intended business workflow. Manual testing and domain knowledge are essential.

50. How do you perform API penetration testing using Burp Suite and Postman?

Interview-ready answer:

I use Postman for API interaction, request construction and functional testing, and Burp Suite for intercepting, modifying, replaying and analyzing API traffic.

My testing process begins with understanding the API architecture and collecting all available information.

Phase 1: Reconnaissance and API discovery

I collect:

- Base URLs.
- API documentation.
- Swagger/OpenAPI specifications.
- Postman collections.
- Authentication mechanisms.
- Available user roles.
- Endpoint lists.
- Request and response formats.

I also capture traffic from the web or mobile application.

Phase 2: Configure the testing environment

I configure Burp Suite as the proxy and route authorized application traffic through it.

I use Postman to import API specifications and organize requests into collections.

I configure the required authentication tokens and environment variables.

Phase 3: Authentication testing

I check:

- Missing authentication.
- Invalid credentials.
- Expired tokens.
- JWT validation.
- Password reset.
- OTP security.
- Refresh token behavior.

Phase 4: Authorization testing

I use multiple test accounts to identify:

- BOLA.
- BFLA.
- Horizontal privilege escalation.
- Vertical privilege escalation.
- Property-level authorization issues.

Phase 5: Input validation and injection

I test:

- SQL injection.
- NoSQL injection.
- Command injection.
- SSRF.
- XXE.
- Parameter pollution.
- Unexpected data types.
- Mass assignment.

Phase 6: Business logic testing

I examine:

- Payment workflows.
- Coupon handling.
- Transactions.
- Account management.
- Duplicate requests.
- Race conditions.
- State transitions.

Phase 7: Configuration and infrastructure testing

I examine:

- CORS.
- TLS.
- HTTP methods.
- Rate limiting.
- Error handling.
- API versioning.
- Debug endpoints.
- Security headers.

Phase 8: Reporting and retesting

I document reproducible evidence, affected endpoints, risk, business impact, remediation recommendations and supporting requests and responses.

After developers fix the issues, I retest the affected functionality and verify that the vulnerability is no longer exploitable.

Tools I commonly use:

Tool	Purpose
Burp Suite	Intercepting, modifying and replaying requests
Postman	API requests and collections
Swagger/OpenAPI	API documentation and discovery
SQLmap	Authorized SQL injection validation
Nuclei	Template-based vulnerability checks
jq	JSON inspection and manipulation
curl	Manual HTTP requests

A strong interview answer should emphasize that tools support the methodology. They do not replace manual authorization and business logic testing.

Level 3: Advanced API Security

51. Explain the complete JWT authentication mechanism.

Interview-ready answer:

JWT authentication is a token-based authentication mechanism in which the server issues a signed token after successfully authenticating a user.

The client then presents this token with subsequent API requests.

Let's understand the complete process.

Step 1: User authentication

The user submits credentials:

The backend validates the credentials against its authentication system.

Step 2: JWT generation

If authentication succeeds, the server generates a JWT containing relevant claims.

Example header:

Example payload:

The server signs the token using its configured signing key.

Step 3: Token delivery

The server returns the token to the client.

The client stores it according to the application's security architecture.

Step 4: API request

The client includes the token in subsequent requests:

Step 5: Token validation

The backend validates:

- Token signature.
- Allowed signing algorithm.
- Expiration.
- Not-before claim, if used.
- Issuer.
- Audience.
- Required claims.

Step 6: Authorization

After validating the token, the backend determines whether the authenticated user has permission to access the requested resource.

This is important because a valid JWT does not automatically authorize every operation.

Security testing perspective:

I test the entire token lifecycle, not just the token generation.

My checks include:

1. Signature validation.
2. Algorithm manipulation.
3. Weak signing secrets.
4. Token expiration.
5. Issuer and audience validation.
6. Token reuse.
7. Refresh token handling.
8. Privilege manipulation.
9. Revocation and logout.
10. Sensitive information in JWT claims.

Important interview point:

JWT is a token format, not a complete authentication or authorization solution.

The application must correctly validate the token and enforce authorization independently.

52. What is the JWT `alg: none` vulnerability?

Interview-ready answer:

The JWT `alg: none` vulnerability occurs when an application incorrectly accepts unsigned JWTs even though the application expects signed tokens.

In a normal signed JWT, the header identifies the signing algorithm.

For example:

The server generates a signature using the configured key.

However, JWT specifications also define an unsecured JWT using the `none` algorithm, where the token has no cryptographic signature.

A vulnerable application may accept an unsigned token without properly enforcing signature verification.

An attacker could potentially modify the token payload and submit it without a valid signature.

For example, a token might originally contain:

The attacker attempts to change it to:

If the server accepts an unsigned token and trusts the modified role, privilege escalation may occur.

How I test it:

1. Capture a valid JWT.
2. Decode the header and payload.
3. Examine the accepted signing algorithms.
4. Construct an unsigned token in an authorized test environment.
5. Submit it to the protected endpoint.
6. Verify whether the server rejects it.
7. Check whether authorization decisions rely on unverified claims.

Impact:

Potential authentication bypass and privilege escalation.

Remediation:

- Reject unsecured JWTs.
- Explicitly configure permitted signing algorithms.
- Always verify signatures.
- Validate claims independently.
- Never trust role or permission claims from an unverified token.

Likely follow-up: Is changing `alg` to `none` always exploitable?

No. Modern JWT libraries generally reject unsigned tokens unless explicitly configured to accept them. The vulnerability depends on the server's actual validation behavior.

53. What is JWT algorithm confusion?

Interview-ready answer:

JWT algorithm confusion is a vulnerability that occurs when a JWT implementation incorrectly handles the relationship between signing algorithms and cryptographic keys.

A commonly discussed example involves confusion between HS256 and RS256.

HS256 uses a shared secret for both signing and verification.

RS256 uses an asymmetric key pair:

- Private key for signing.
- Public key for verification.

In a vulnerable implementation, the server may support RS256 but incorrectly accept HS256 tokens using the public key material as an HMAC secret.

An attacker who obtains the public key may attempt to create a forged HS256 token using that key material.

If the server incorrectly verifies it, the attacker may be able to forge tokens.

How I test it:

1. Capture a valid JWT.
2. Identify the signing algorithm.
3. Examine available public key information, such as JWKS endpoints.
4. Understand how the server selects verification algorithms.
5. In an authorized test environment, assess whether the server accepts an unexpected algorithm.
6. Verify whether the server rejects tokens signed with an inappropriate key type.

Impact:

If exploitable, an attacker may forge JWTs and potentially impersonate users or elevate privileges.

Remediation:

- Explicitly allowlist signing algorithms.
- Enforce correct key types.
- Use trusted key configurations.
- Avoid relying on untrusted token headers to determine verification behavior.
- Use established JWT libraries correctly.

Likely follow-up: What is the difference between algorithm confusion and `alg: none` ?

`alg: none` involves accepting unsigned tokens. Algorithm confusion involves incorrect interpretation or selection of signing algorithms and cryptographic keys.

54. How can an attacker exploit weak JWT signing secrets?

Interview-ready answer:

In JWT implementations using symmetric algorithms such as HS256, the same secret is used to sign and verify the token.

If the signing secret is weak, predictable or exposed, an attacker may be able to recover or obtain it and generate valid JWTs.

For example, an application might use a weak secret:

An attacker who captures a JWT may attempt offline password guessing against the token's signature.

If the secret is recovered, the attacker can generate a correctly signed token containing modified claims.

For example:

If the backend trusts the signed role claim, the forged token could result in privilege escalation.

How I test:

1. Identify whether the application uses HS256 or another symmetric algorithm.
2. Obtain a valid JWT from an authorized test account.
3. Examine the token header and signature.
4. Assess whether the signing secret is weak or exposed.
5. Use approved offline testing tools to assess secret strength.
6. If a weak secret is confirmed, demonstrate token forgery using a controlled test identity.

Impact:

- User impersonation.
- Privilege escalation.
- Authentication bypass.

- Unauthorized API access.

Remediation:

Use high-entropy cryptographically secure secrets, secure key storage, key rotation and established JWT libraries.

For systems using asymmetric signing, protect private keys and distribute public verification keys securely.

55. How do you test JWT signature validation?

Interview-ready answer:

JWT signature validation ensures that a token has not been modified and that it was signed by an authorized issuer holding the appropriate signing key.

I test whether the backend correctly verifies the signature before trusting any claims.

My testing methodology:

Step 1: Capture a valid token.

I authenticate using a test account and capture its JWT through Burp Suite.

Step 2: Decode the token.

I inspect the header and payload.

For example:

Step 3: Modify the payload.

I change the role from user to admin.

Step 4: Preserve the original signature.

I submit the modified payload with the original signature.

The backend should reject the token because the signature no longer matches.

Step 5: Test other validation weaknesses.

I examine:

- Unsigned token acceptance.
- Algorithm confusion.
- Incorrect key selection.
- Weak secrets.
- Missing expiration validation.
- Incorrect issuer validation.
- Incorrect audience validation.

Expected secure behavior:

The server should reject tokens with invalid signatures, unsupported algorithms, expired validity periods or untrusted issuers.

Important distinction:

A JWT's payload can always be decoded in a standard signed JWT. Encoding is not encryption. The signature provides integrity and authenticity, not confidentiality.

56. What is the difference between HS256 and RS256?

Interview-ready answer:

HS256 and RS256 are cryptographic signing algorithms used with JWTs.

HS256:

HS256 stands for HMAC using SHA-256.

It is a symmetric signing algorithm, meaning the same secret key is used to sign and verify the JWT.

Advantages:

- Fast.
- Simple key management in smaller systems.
- Suitable when all participating services can securely share a secret.

Disadvantages:

- Every verifier possessing the secret can also create valid signatures.

- Secret distribution becomes challenging in larger architectures.

RS256:

RS256 uses RSA with SHA-256.

It is an asymmetric signing algorithm.

The issuer signs tokens using a private key, while resource servers verify them using the corresponding public key.

Advantages:

- Public keys can be distributed without exposing signing capability.
- Useful in distributed service architectures.
- Centralized signing can be separated from token verification.

Disadvantages:

- More complex key management.
- Private keys require strong protection.
- Cryptographic operations are generally more computationally expensive than HMAC.

Comparison:

Feature	HS256	RS256
Algorithm type	Symmetric	Asymmetric
Signing key	Shared secret	Private key
Verification key	Same secret	Public key
Key distribution	Secret must remain confidential	Public key can be distributed
Main risk	Weak or leaked secret	Private-key compromise

Security testing perspective:

I verify that the server enforces the intended algorithm and key type.

I also check key rotation, key storage, signature validation and rejection of unexpected algorithms.

Likely follow-up: Which is more secure?

Neither is universally more secure. The choice depends on architecture and key management. RS256 is often convenient when multiple services need to verify tokens without receiving signing privileges.

57. How can JWT key confusion occur?

Interview-ready answer:

JWT key confusion occurs when a server incorrectly selects or interprets the cryptographic key used to verify a token.

For example, a JWT header may contain a `kid` claim identifying the key used for signing.

The backend uses this identifier to locate the corresponding verification key.

A vulnerable implementation might construct a file path or key lookup using untrusted values from the token header without proper validation.

This may introduce key-selection vulnerabilities, including:

- Path traversal in key lookup.
- Incorrect key selection.
- Trusting attacker-controlled key references.
- Unsafe remote key retrieval.
- Key-type confusion.

How I test:

1. Inspect JWT headers for `kid`, `jku` and `x5u`.
2. Examine how the application selects verification keys.
3. Check whether untrusted header values influence file or URL access.
4. Verify that keys come only from trusted configured sources.
5. Test whether malformed or unknown key identifiers are rejected.

Impact:

Depending on the implementation, the issue may lead to token forgery or authentication bypass.

Remediation:

Use trusted key stores, strict key identifiers, fixed trusted key sources and strict algorithm-to-key-type validation.

Never allow untrusted JWT headers to determine arbitrary file paths or unrestricted key retrieval.

58. What happens if an API does not validate JWT expiration?

Interview-ready answer:

JWT expiration is commonly represented by the `exp` claim.

It defines the time after which the token must no longer be accepted.

Example:

If the API does not validate expiration, an expired JWT may continue to be accepted.

This increases the risk associated with stolen or leaked tokens.

For example, suppose a user logs in and receives an access token valid for 15 minutes.

If the backend ignores the expiration claim, the token may remain usable long after the intended validity period.

How I test:

1. Capture a valid JWT.
2. Examine its expiration claim.
3. Wait until the token expires or use an authorized test token with a controlled expiration.
4. Replay the token against a protected endpoint.
5. Verify whether the server rejects it.
6. Examine whether the server also validates `nbf`, issuer and audience where applicable.

Impact:

An attacker who obtains a token may retain unauthorized access for an extended period.

Remediation:

Validate the expiration claim on every relevant request, use appropriate token lifetimes and implement secure refresh and revocation mechanisms.

Likely follow-up: Does changing the expiration claim always extend the token's life?

No. Modifying the claim invalidates the signature. It only becomes exploitable if signature verification is broken or the server incorrectly trusts the modified claim.

59. How do you test JWT token reuse?

Interview-ready answer:

JWT token reuse testing determines whether an issued token remains valid beyond the intended authentication lifecycle or can be reused inappropriately.

JWTs are often self-contained, so they may remain valid until expiration unless the application implements revocation or session-state controls.

My testing approach:

1. Authenticate with a test account.
2. Capture the access token.
3. Use the token to access a protected endpoint.
4. Log out of the application.
5. Replay the original token.
6. Observe whether access is still permitted.
7. Repeat after password change or account disablement.
8. Examine refresh token reuse and rotation.
9. Check whether old tokens remain valid after a new login.

Example:

If the old token continues to access sensitive resources after logout, I investigate whether this is expected behavior or a failure of the application's session invalidation requirements.

Important clarification:

JWT logout behavior is architecture-dependent. A stateless access token may remain valid until expiration by design.

The issue becomes a vulnerability when the application's documented or required security behavior demands immediate invalidation, but the backend does not enforce it.

Remediation:

Use short-lived access tokens, refresh token rotation, revocation mechanisms or server-side session checks where immediate invalidation is required.

60. How can refresh token rotation be bypassed or misconfigured?

Interview-ready answer:

Refresh token rotation is a security mechanism in which a refresh token is replaced with a new refresh token whenever it is successfully used to obtain a new access token.

The previously used refresh token should become invalid.

For example:

1. User receives Refresh Token A.
2. Client uses Token A.
3. Authorization server issues Access Token B and Refresh Token B.
4. Token A is invalidated.
5. If Token A is reused, the server detects the reuse.

This helps reduce the impact of stolen refresh tokens.

Potential implementation weaknesses:

- Old refresh tokens remain valid.
- Rotation does not happen consistently.
- Multiple refresh requests create conflicting valid tokens.
- Reuse detection is missing.
- Refresh tokens do not expire appropriately.
- Tokens are not invalidated after relevant security events.

How I test:

1. Obtain a refresh token using a test account.
2. Use it to obtain a new access token and refresh token.
3. Replay the original refresh token.
4. Check whether the old token is rejected.
5. Test concurrent refresh requests in a controlled environment.
6. Check whether token reuse triggers appropriate session handling.
7. Test logout and account security-event behavior.

Impact:

An attacker possessing a stolen refresh token may maintain access or generate new access tokens.

Remediation:

Implement token rotation, secure token storage, reuse detection, token-family invalidation and appropriate expiration policies.

61. How do you test OAuth 2.0 Authorization Code Flow?

Interview-ready answer:

The OAuth 2.0 Authorization Code Flow allows a client application to obtain an access token through an authorization code issued by an authorization server.

I first understand the application's OAuth architecture and identify the client, authorization server, redirect URI, scopes and resource server.

Normal flow:

Step 1: Authorization request

The client redirects the user to the authorization server.

Step 2: User authentication and consent

The authorization server authenticates the user and requests consent where applicable.

Step 3: Authorization code

The authorization server redirects the user to the registered callback URL with an authorization code.

Step 4: Token exchange

The client exchanges the authorization code for tokens through the token endpoint.

Step 5: API access

The client presents the access token to the resource server.

My security testing approach:

- Verify redirect URI matching.
- Verify state parameter validation.
- Test authorization code reuse.
- Check code expiration.
- Verify PKCE where applicable.
- Check client authentication.
- Validate scope enforcement.
- Check token audience.
- Examine token leakage through URLs, logs or browser history.
- Test account and session binding.

Remediation:

Use exact redirect URI matching, secure state handling, PKCE for public clients and proper authorization code lifecycle management.

62. What is the OAuth authorization code interception attack?

Interview-ready answer:

Authorization code interception occurs when an attacker obtains an authorization code issued during the OAuth authorization flow and attempts to exchange it for an access token.

The authorization code is a temporary credential, so its confidentiality is important.

Consider a mobile application using OAuth Authorization Code Flow.

The user authenticates through an external browser and is redirected to the mobile application using a callback URI.

If the callback mechanism is improperly configured, another application may intercept the authorization code.

If the attacker can successfully exchange the intercepted code, they may obtain access tokens.

Potential causes:

- Insecure custom URI scheme handling.
- Improper redirect URI validation.

- Authorization codes exposed through logs.
- Missing PKCE.
- Weak callback handling.

How I test:

1. Understand the OAuth redirect flow.
2. Identify callback URI configuration.
3. Examine whether another application could claim the same custom URI scheme.
4. Check whether PKCE is implemented.
5. Verify authorization code expiration and single-use behavior.
6. Check whether intercepted codes can be exchanged without the original client's proof.

Remediation:

Use Authorization Code Flow with PKCE, secure redirect handling, exact redirect URI validation and short-lived single-use authorization codes.

63. What is PKCE, and how does it prevent authorization code interception?

Interview-ready answer:

PKCE stands for Proof Key for Code Exchange.

It is an OAuth security extension that protects authorization code flows against authorization code interception.

It is particularly important for public clients such as mobile applications and single-page applications that cannot securely maintain a client secret.

PKCE introduces two values:

- Code verifier.
- Code challenge.

How it works:

Step 1: Generate code verifier

The client generates a cryptographically random secret.

Step 2: Generate code challenge

The client derives a challenge from the verifier using SHA-256.

The preferred method is `S256`.

Step 3: Authorization request

The client sends the code challenge to the authorization server.

Step 4: Authorization code

The authorization server returns an authorization code.

Step 5: Token exchange

The client submits the authorization code along with the original code verifier.

Step 6: Verification

The authorization server calculates the challenge from the supplied verifier and compares it with the original challenge.

If they match, the server allows the token exchange.

Security benefit:

An attacker who intercepts the authorization code does not possess the original code verifier and therefore cannot successfully exchange the code for tokens.

How I test PKCE:

- Check whether PKCE is required for public clients.
- Verify the S256 challenge method.
- Test whether the token endpoint rejects an incorrect verifier.
- Check whether authorization codes can be reused.
- Verify that the code is bound to the original challenge.

64. What is an OAuth redirect URI manipulation vulnerability?

Interview-ready answer:

An OAuth redirect URI manipulation vulnerability occurs when an authorization server does not properly validate the redirect URI supplied by the client.

The redirect URI determines where the authorization response, including the authorization code, is sent after successful authorization.

Example:

The authorization server should verify that the redirect URI matches an allowed URI registered for that client.

A vulnerable implementation may accept attacker-controlled destinations or unsafe variations.

Potential weaknesses include:

- Wildcard redirect URI matching.
- Substring-based validation.
- Open redirectors on trusted domains.
- Incorrect normalization.
- HTTP redirect destinations where HTTPS is required.
- Inconsistent validation between authorization and token exchange.

How I test:

1. Capture the OAuth authorization request.
2. Identify the registered redirect URI.
3. Modify the redirect URI.
4. Test alternative paths and subdomains.
5. Check whether URL normalization is handled correctly.
6. Examine whether authorization codes can be delivered to unauthorized destinations.

Impact:

An attacker may redirect authorization responses to a destination they control and potentially steal authorization codes.

Remediation:

Use exact registered redirect URI matching, secure URI parsing and consistent validation throughout the OAuth flow.

65. What is the OAuth state parameter, and why is it important?

Interview-ready answer:

The OAuth state parameter is a value generated by the client and included in the authorization request.

The client later receives this value in the authorization response and verifies that it matches the original value.

Its primary purpose is to protect the authorization flow against Cross-Site Request Forgery and unauthorized response injection.

Example authorization request:

Authorization response:

The client validates the returned state value against the value associated with the original authorization request.

How I test:

1. Capture the authorization request.
2. Identify the state parameter.
3. Observe the callback response.
4. Modify or remove the state parameter.
5. Check whether the application rejects unexpected values.
6. Test whether state values are predictable or reused improperly.
7. Verify that the state is bound to the initiating browser session or transaction.

Impact if incorrectly implemented:

An attacker may be able to inject an authorization response or manipulate the authentication flow.

Remediation:

Generate cryptographically random state values, bind them to the initiating transaction and validate them before processing the authorization response.

Important distinction:

State and PKCE solve related but different problems. State protects the authorization transaction against CSRF, while PKCE binds the authorization code to the client-generated verifier.

66. What is OAuth scope abuse?

Interview-ready answer:

OAuth scopes define the permissions an application requests or receives when accessing protected resources.

Scope abuse occurs when an application requests excessive permissions or when the authorization server or resource server does not properly enforce the granted scopes.

For example:

may allow access to basic profile information.

A more privileged scope might permit access to financial transactions or account management.

How I test:

1. Capture the authorization request.
2. Identify requested scopes.
3. Understand the intended permissions.
4. Examine the granted scopes in the token response or token claims.
5. Test whether the client can request additional scopes.
6. Check whether the authorization server improperly grants them.
7. Verify whether resource servers enforce scope restrictions.

Example:

Suppose a mobile application should only access:

But the API also accepts requests to retrieve sensitive transaction information without requiring the appropriate transaction scope.

This may indicate missing scope enforcement.

Impact:

Excessive privileges may expose sensitive data or allow unauthorized operations.

Remediation:

Apply least privilege, validate scopes at the resource server and grant only permissions required for the specific application functionality.

67. How can an attacker exploit weak API authorization across multiple user roles?

Interview-ready answer:

Weak API authorization occurs when the backend does not correctly enforce the permissions assigned to different user roles.

Consider an application with:

- Customer.
- Support agent.
- Manager.
- Administrator.

Each role should have specific permissions.

For example:

Function	Customer	Support	Manager	Admin
View own profile	Yes	Yes	Yes	Yes
View all customers	No	Limited	Yes	Yes
Approve transactions	No	No	Yes	Yes
Manage roles	No	No	No	Yes

During testing, I create or obtain authorized accounts for different roles and build an access-control matrix.

I capture requests from higher-privileged accounts and replay them using lower-privileged accounts.

For example:

I verify whether the customer can execute the administrative function.

I also test whether a user can modify their role through profile-update APIs or access hidden endpoints.

Important testing areas:

- Role assignment.
- Administrative APIs.
- Approval workflows.
- User management.
- Sensitive reports.
- Financial transactions.
- Tenant isolation.
- Object-level permissions.

Impact:

Unauthorized data access, privilege escalation, fraudulent operations and compromise of administrative functionality.

Remediation:

Implement centralized authorization policies and verify permissions on every sensitive backend operation.

Do not rely on frontend role restrictions.

68. How do you test horizontal and vertical privilege escalation in APIs?

Interview-ready answer:

I test horizontal and vertical privilege escalation separately because they involve different authorization failures.

Horizontal privilege escalation

A user accesses another user's resources while remaining at the same privilege level.

Example:

- User A accesses their own transaction.
- User A changes the transaction ID to User B's transaction ID.
- The API returns User B's transaction.

This is typically BOLA.

Vertical privilege escalation

A lower-privileged user accesses functionality intended for a higher-privileged role.

Example:

- A customer sends an administrative request.
- The API allows the customer to suspend another account.
- The customer gains unauthorized administrative capability.

This is typically BFLA or another authorization failure.

My testing methodology:

1. Identify application roles.
2. Obtain authorized test accounts.
3. Build an authorization matrix.
4. Capture requests for each role.
5. Replay higher-privileged requests with lower-privileged credentials.
6. Modify object identifiers.
7. Test read, create, update and delete operations.
8. Verify actual unauthorized behavior.
9. Document business impact and remediation.

Important interview point:

A valid authentication token does not establish authorization. The backend must independently verify whether the authenticated principal is allowed to perform the specific action on the specific object.

69. How do you identify race conditions in API endpoints?

Interview-ready answer:

A race condition occurs when the outcome of an operation depends on the timing or order of concurrent requests.

In APIs, race conditions can occur when multiple requests access or modify the same resource simultaneously without appropriate synchronization or transactional controls.

Consider a banking application.

A user has a balance of ₹10,000 and attempts to transfer ₹8,000.

If the backend checks the balance before processing the transaction but does not safely synchronize concurrent requests, multiple simultaneous transfers may incorrectly pass the balance check.

How I test:

1. Identify sensitive state-changing API endpoints.
2. Establish a controlled baseline.
3. Understand the normal transaction workflow.
4. Send concurrent requests using Burp Suite Repeater or an appropriate concurrency-testing feature.
5. Observe whether multiple requests are processed unexpectedly.
6. Check balances, transaction records and application state.
7. Repeat the test to distinguish a genuine race condition from normal processing variation.

Common scenarios:

- Duplicate payments.
- Coupon redemption.
- Inventory purchases.
- Withdrawal operations.
- OTP verification.
- Password reset.
- Account balance updates.
- Limited-use promotional offers.

Impact:

Depending on the application, race conditions may cause financial manipulation, duplicate transactions, inventory abuse or inconsistent records.

Remediation:

Use database transactions, atomic operations, appropriate locking, idempotency keys and concurrency-safe business logic.

Likely follow-up: Is sending duplicate requests always a race condition?

No. Duplicate requests may simply be processed normally. A race condition requires demonstrating an unexpected outcome caused by concurrent execution or timing.

70. How do you test HTTP parameter pollution in APIs?

Interview-ready answer:

HTTP Parameter Pollution occurs when multiple parameters with the same name are supplied in a request and the application or its components interpret them inconsistently.

Consider:

Different components may process duplicate parameters differently.

For example:

- One component may select the first value.
- Another may select the last value.
- Another may combine both values into an array.

If a proxy, gateway and backend interpret the parameters differently, security checks may become inconsistent.

How I test:

1. Identify security-sensitive parameters.
2. Capture normal API requests.
3. Duplicate selected parameters.
4. Change their ordering.
5. Test URL-encoded and JSON-based representations where applicable.
6. Compare gateway and backend behavior.
7. Check whether authorization or validation decisions can be bypassed.

Example:

I investigate whether the application authorizes one identifier but retrieves another.

Impact:

Depending on implementation, parameter pollution may lead to authorization bypass, business logic manipulation or inconsistent request processing.

Remediation:

Reject ambiguous duplicate parameters or define one consistent parsing behavior across all application components.

71. What is request smuggling, and how can it affect API gateways?

Interview-ready answer:

HTTP request smuggling is a vulnerability that occurs when two HTTP components, such as a reverse proxy and backend server, interpret request boundaries differently.

This commonly happens when the components disagree about HTTP message framing.

Two important headers are:

- `Content-Length`
- `Transfer-Encoding`

If a front-end proxy and backend server disagree about which header determines the request body length, an attacker may manipulate how subsequent request data is interpreted.

Common variants:

- CL.TE: Front end uses Content-Length, backend uses Transfer-Encoding.
- TE.CL: Front end uses Transfer-Encoding, backend uses Content-Length.
- TE.TE: Both support Transfer-Encoding but interpret it differently because of implementation inconsistencies.

How I test:

1. Identify the HTTP architecture and intermediaries.
2. Determine which HTTP versions and transfer mechanisms are supported.
3. Use controlled, non-destructive request-smuggling tests.
4. Observe response discrepancies and connection behavior.
5. Confirm that the issue is reproducible.
6. Avoid tests that could affect other users or shared production connections.

Potential impact:

- Request queue poisoning.
- Unauthorized request interference.

- Access-control bypass.
- Cache poisoning.
- Exposure of internal responses.

Remediation:

Normalize HTTP requests at trust boundaries, reject ambiguous framing, patch affected components and ensure consistent parsing between proxies and backend servers.

72. How do you identify API cache poisoning?

Interview-ready answer:

Cache poisoning occurs when an attacker causes a cache to store a manipulated response that is subsequently served to other users.

In API environments, this can happen when a caching system incorrectly handles request parameters or headers that influence the backend response but are not included in the cache key.

Consider an API:

Suppose the backend response is influenced by a request header, but the cache does not account for that header when generating its cache key.

An attacker may manipulate the request so that a modified response is cached and served to other users.

How I test:

1. Identify cacheable API responses.
2. Inspect Cache-Control, Vary and related headers.
3. Understand the cache key behavior.
4. Identify headers or parameters influencing the response.
5. Use a controlled, non-sensitive test value.
6. Determine whether the cache stores the modified response.
7. Check whether subsequent requests receive the poisoned content.

Potential impact:

- Stored malicious content.
- Incorrect responses.

- Sensitive information exposure.
- Cache-based denial of service.
- User-specific data leakage.

Remediation:

Use correct cache keys, avoid caching sensitive personalized responses, configure appropriate Vary headers and normalize untrusted request inputs.

73. What is GraphQL introspection, and why can it be dangerous?

Interview-ready answer:

GraphQL introspection is a built-in capability that allows clients to query the GraphQL schema and discover available types, fields, queries, mutations and other schema information.

It is useful for development because it allows tools to understand the API structure.

For example, introspection may reveal that a GraphQL API supports:

It may also reveal mutations:

Why can it be dangerous?

If introspection exposes sensitive schema information to unauthorized users, attackers may learn about:

- Hidden administrative operations.
- Sensitive fields.
- Internal object relationships.
- Deprecated functionality.
- Mutation capabilities.
- Backend business logic.

This information can help attackers identify further testing opportunities.

How I test:

1. Identify GraphQL endpoints.
2. Determine whether introspection is enabled.
3. Examine whether unauthorized users can retrieve the schema.
4. Review exposed types, fields, queries and mutations.

5. Test authorization independently for discovered operations.

Important distinction:

Disabling introspection does not secure GraphQL by itself. Attackers can still discover and test operations through other means.

Remediation:

Restrict introspection in production where appropriate, protect sensitive operations and implement resolver-level authorization.

74. What are GraphQL query depth and query complexity attacks?

Interview-ready answer:

GraphQL allows clients to request nested fields and related objects in a single query.

However, deeply nested or computationally expensive queries can consume significant server resources.

Query depth attack

This occurs when a client submits deeply nested queries that cause the server to perform excessive processing.

For example:

If the schema allows extensive nesting, a single request may trigger many database operations.

Query complexity attack

This occurs when a query requests an excessive amount of data or combines expensive operations.

For example, a query that requests thousands of records with multiple nested relationships may consume significant CPU, memory and database resources.

How I test:

1. Identify GraphQL endpoints.
2. Understand schema relationships.
3. Test nested query depth.

4. Examine pagination limits.
5. Test large list requests.
6. Examine aliases and repeated fields.
7. Observe response time and resource consumption.
8. Check whether rate limits apply.

Impact:

Denial of service, excessive backend resource consumption and application instability.

Remediation:

Implement:

- Query depth limits.
- Query complexity scoring.
- Pagination.
- Request timeouts.
- Rate limiting.
- Resource quotas.
- Efficient resolver design.

75. How do you test GraphQL authorization vulnerabilities?

Interview-ready answer:

GraphQL authorization testing focuses on whether the server correctly enforces access permissions for every query, mutation, object and field.

Unlike traditional REST APIs, GraphQL may expose many operations through a single endpoint.

Therefore, testing only the endpoint-level authorization is insufficient.

Consider:

I would test whether a normal user can retrieve another user's details by changing the identifier.

I would also examine whether sensitive fields such as `role`, `salary` or internal account information are properly restricted.

My testing methodology:

1. Discover the GraphQL endpoint.
2. Obtain the schema where permitted.
3. Identify queries, mutations and object relationships.
4. Test unauthenticated access.
5. Test with different user roles.
6. Modify object identifiers.
7. Test nested object access.
8. Test sensitive fields independently.
9. Test administrative mutations.
10. Verify authorization at resolver level.

Example:

A user may be allowed to view their own profile but should not be able to access another user's salary information.

Even if the API correctly restricts the main user object, an improperly protected nested resolver might expose the salary field.

Common vulnerabilities:

- BOLA.
- BFLA.
- Field-level authorization failures.
- Excessive data exposure.
- Unauthorized mutations.
- Query resource exhaustion.

Remediation:

Enforce authorization within resolvers or an equivalent centralized policy layer, validate object ownership and restrict sensitive fields independently.

Level 4: Real-World API Security Scenarios

76. An API returns another user's account information when you change the user ID. How would you investigate and exploit this?

Interview-ready answer:

This scenario indicates a potential BOLA or IDOR vulnerability.

I would first understand the endpoint and identify which user owns the requested resource.

Suppose I have two authorized test accounts:

- User A: Account ID 1001.
- User B: Account ID 1002.

I authenticate as User A and capture:

The API returns User A's account information.

I then modify the account identifier:

If the response contains User B's account information, I investigate whether the backend is missing object-level authorization.

How I validate it:

1. Confirm the original account belongs to User A.
2. Confirm the second account belongs to User B using the authorized test environment.
3. Replay the modified request using User A's token.
4. Compare the responses.
5. Identify sensitive information exposed.
6. Check other operations, such as update and delete, using safe test data.
7. Capture reproducible evidence.

Impact:

If customer account details are exposed, the issue may affect confidentiality and privacy. If account modification or transactions are possible, the impact may be significantly higher.

Remediation:

The backend must verify that the authenticated user is authorized to access the requested account.

The database query should be scoped to the user's permitted resources rather than trusting the supplied account ID.

What I would tell the interviewer:

"I would not stop after changing the ID and observing a 200 response. I would verify that the returned object actually belongs to another user and that the user has no authorization to access it. Then I would establish the impact and document the exact request and response."

77. An API allows you to change the HTTP method from GET to PATCH. How would you test whether this creates an authorization issue?

Interview-ready answer:

I would investigate whether the backend correctly enforces authorization and method restrictions independently of the frontend.

For example, the application may display a profile using:

I would capture the request and examine whether the same endpoint accepts PATCH:

I would then investigate:

1. Whether PATCH is an officially supported operation.
2. Whether the user is authorized to modify the object.
3. Whether only permitted properties can be changed.
4. Whether administrative properties can be modified.
5. Whether authentication is enforced.
6. Whether the application validates the HTTP method consistently.

For example, if a normal user can modify their own email, that may be expected behavior.

However, if the same request allows the user to modify another user's email or change their role to administrator, it indicates an authorization weakness.

Important distinction:

PATCH being accepted is not itself a vulnerability. The vulnerability exists when the method exposes an operation without the required authorization or validation.

Remediation:

Enforce method-level, object-level and property-level authorization on the backend.

78. You discover that an API accepts additional JSON parameters that are not visible in the application. How would you investigate mass assignment?

Interview-ready answer:

I would investigate whether the backend automatically maps additional JSON properties to internal application objects.

First, I capture a normal request from the frontend.

Example:

Then I inspect the application's responses, API documentation and available test functionality to understand which properties are expected.

I would introduce additional properties in a controlled test request:

I would then examine:

- Whether the API accepts the additional fields.
- Whether the backend ignores or processes them.
- Whether the properties persist.
- Whether the modified properties change application behavior.
- Whether the authenticated user is authorized to make those changes.

I would pay particular attention to privileged attributes such as roles, account status, verification status and financial values.

Impact:

Depending on the fields affected, this could lead to privilege escalation, account manipulation or financial fraud.

Remediation:

Use explicit field allowlists, dedicated request models and property-level authorization.

79. An API returns a valid JWT even after the user logs out. Is this a vulnerability? Explain.

Interview-ready answer:

Not necessarily. It depends on how the application implements JWT authentication and logout.

A self-contained JWT can remain cryptographically valid until its expiration time unless the application implements an additional revocation mechanism.

For example:

1. User logs in.
2. Server issues a JWT valid for 15 minutes.
3. User logs out after 5 minutes.
4. The client deletes its locally stored token.
5. The token itself may still be accepted by the backend until expiration.

This is not automatically a vulnerability if the application intentionally uses short-lived stateless access tokens and its documented security requirements allow this behavior.

However, it becomes a security issue if the application promises immediate session invalidation but continues accepting the token for sensitive operations.

How I test:

1. Authenticate using a test account.
2. Capture the access token.
3. Verify that it works before logout.
4. Log out.
5. Replay the original token.
6. Check whether protected endpoints still accept it.
7. Repeat after password change or account disablement.
8. Examine refresh token invalidation.

Impact:

If a token is stolen before logout, an attacker may continue accessing protected resources.

Remediation:

Use short-lived access tokens, refresh token revocation, token denylisting or server-side session validation where immediate invalidation is required.

80. An application uses JWT authentication. How would you test whether tokens can be forged?

Interview-ready answer:

I would first identify the JWT signing algorithm and understand how the application validates tokens.

I capture a valid token from an authorized test account and decode its header and payload.

I then investigate potential weaknesses.

1. Algorithm validation

I check whether the application accepts only approved algorithms.

2. Signature validation

I modify a harmless claim and verify that the original signature is rejected.

3. Weak signing secrets

For symmetric algorithms, I assess whether the secret is weak or exposed.

4. Algorithm confusion

I examine whether the server incorrectly handles symmetric and asymmetric signing algorithms.

5. Unsigned tokens

I check whether unsigned tokens are accepted.

6. Key selection

I inspect `kid`, `jku` and related headers to understand whether key selection is safe.

7. Claim validation

I check whether the server properly validates expiration, issuer, audience and other required claims.

Practical tools:

- Burp Suite.
- JWT inspection tools.
- jwt.io for decoding and inspection.
- Python JWT libraries in a controlled test environment.

Impact:

If token forgery is successful, an attacker may impersonate users or manipulate privileges.

Remediation:

Use strong cryptographic keys, strict algorithm allowlists, secure key management and established JWT libraries.

81. An API has rate limiting, but changing the `X-Forwarded-For` header bypasses it. How would you assess this?

Interview-ready answer:

I would investigate whether the API incorrectly trusts client-controlled forwarding headers when identifying the source of requests.

`X-Forwarded-For` is commonly used by reverse proxies and load balancers to communicate the originating client IP address.

Example:

However, this header can be supplied or manipulated by clients unless a trusted proxy infrastructure sanitizes and manages it correctly.

My testing approach:

1. Establish the rate-limit threshold.
2. Send repeated requests from the authorized testing environment.
3. Observe when rate limiting is triggered.
4. Change the X-Forwarded-For value.
5. Check whether the rate limit resets unexpectedly.
6. Repeat using different header combinations.
7. Examine whether the backend trusts a header supplied directly by the client.

8. Confirm whether the trusted proxy overwrites or sanitizes the header.

Impact:

An attacker may bypass IP-based rate limiting and perform automated attacks against login, OTP or other sensitive endpoints.

Remediation:

Only trust forwarding headers from known proxy infrastructure. The trusted proxy should sanitize incoming values and set authoritative client information.

Also implement account-based and other layered rate limits.

82. An API allows a user to update their role through a JSON parameter. How would you validate the impact?

Interview-ready answer:

This suggests a potential mass assignment or broken object property-level authorization vulnerability.

I would first understand the intended role-management functionality.

Suppose a normal user sends:

I would verify whether the role value is accepted and actually persisted.

Then I would:

1. Retrieve the user's profile.
2. Confirm whether the role changed.
3. Log out and authenticate again if necessary.
4. Check whether the user receives administrative permissions.
5. Test access to authorized administrative test endpoints.
6. Determine whether the role change is reversible.
7. Capture evidence.

I would not assume that a response containing `"role":"admin"` proves privilege escalation without checking whether the backend actually grants the corresponding permissions.

Impact:

If the user obtains administrative privileges, this could lead to unauthorized access to user management, financial operations and sensitive application functionality.

Remediation:

Role assignment must be restricted to authorized administrative operations. The backend should ignore or reject client-supplied role changes in ordinary profile-update requests.

83. An API exposes internal server IP addresses and database errors. How would you assess the severity?

Interview-ready answer:

I would classify this initially as an information disclosure issue and investigate whether the exposed information creates additional security risks.

Example response:

I would examine whether the response exposes:

- Internal IP addresses.
- Database names.
- Database technologies.
- SQL queries.
- Stack traces.
- File paths.
- Framework versions.
- Credentials or tokens.

Next, I would investigate whether this information can be used to identify internal infrastructure or facilitate another vulnerability.

For example, database error details may reveal SQL query structure, potentially helping to validate SQL injection.

Severity assessment:

I would consider:

1. Sensitivity of the disclosed information.
2. Whether authentication is required.

3. Number of affected endpoints.
4. Whether the information enables further exploitation.
5. Whether credentials or secrets are exposed.

Internal IP disclosure alone is not automatically critical. Severity should reflect actual impact.

Remediation:

Return generic error messages to clients and store detailed diagnostics in protected server-side logs.

84. How would you test an API when you have only a Swagger/OpenAPI specification?

Interview-ready answer:

Swagger/OpenAPI documentation is a valuable starting point because it describes API endpoints, HTTP methods, parameters, request bodies, responses and sometimes authentication requirements.

If I receive only an API specification, I would use it to build a structured testing plan.

Step 1: Import the specification.

I import the OpenAPI file into Postman or use compatible tooling to generate requests.

Step 2: Identify endpoints.

I classify endpoints into:

- Authentication.
- User management.
- Account management.
- Transactions.
- Administrative functionality.
- File operations.
- Reporting.

Step 3: Identify parameters.

I examine path parameters, query parameters, headers and request bodies.

Step 4: Identify authentication requirements.

I inspect the security schemes defined in the specification.

Step 5: Build test cases.

For each endpoint, I test:

- Authentication.
- Authorization.
- Input validation.
- HTTP methods.
- Unexpected parameters.
- Injection.
- Rate limiting.
- Error handling.
- Business logic.

Step 6: Obtain authorized test credentials.

I use test accounts representing different application roles to validate authorization.

Step 7: Discover undocumented functionality.

I compare the documented endpoints with actual application traffic and other authorized discovery sources.

Important limitation:

OpenAPI documentation describes the intended interface. It does not prove that the implementation behaves correctly or that the documented security controls are actually enforced.

85. How would you identify undocumented API endpoints?

Interview-ready answer:

Undocumented endpoints are API routes that exist in the application but are not included in the available API documentation.

They may expose functionality that is not visible in the frontend or included in the Swagger specification.

My methodology:

1. Capture web application traffic through Burp Suite.
2. Inspect JavaScript files for API route references.
3. Analyze mobile APK files using JADX and identify hardcoded API endpoints.
4. Review existing Postman collections.
5. Examine API version patterns.
6. Use authorized content discovery tools.
7. Review application error responses.
8. Examine links and endpoint references in API documentation.
9. Identify administrative and internal routes.
10. Validate discovered endpoints manually.

Tools:

- Burp Suite.
- Postman.
- Dirsearch.
- FFUF.
- Nuclei for relevant checks.
- JADX for Android applications.
- JavaScript analysis tools.

Security perspective:

For every discovered endpoint, I examine whether authentication and authorization are enforced correctly.

A hidden endpoint is not automatically vulnerable, but undocumented functionality may be overlooked during security testing.

86. How would you test an API when you have no frontend access?

Interview-ready answer:

I would perform API testing directly using the available API documentation, endpoint information and authorized credentials.

A frontend is not necessary to test an API because the backend can be interacted with independently through HTTP requests.

My approach:

1. Obtain the base URL and available API documentation.
2. Identify the authentication mechanism.
3. Obtain authorized test credentials.
4. Import the OpenAPI specification into Postman, if available.
5. Build requests for the documented endpoints.
6. Understand expected request and response structures.
7. Test authentication and authorization.
8. Test input validation and injection.
9. Test business logic.
10. Test API configurations and error handling.
11. Document results.

If documentation is unavailable, I use authorized endpoint discovery techniques and any supplied integration details.

Tools:

- Postman.
- Burp Suite.
- curl.
- Swagger/OpenAPI.
- FFUF.
- Nuclei.
- Custom Python scripts.

Important point:

Without frontend access, I may have less visibility into the intended business workflow. I would therefore request architecture documentation, business requirements and role definitions from the client.

87. How would you identify hidden administrative API endpoints?

Interview-ready answer:

I would use authorized endpoint discovery and application traffic analysis to identify administrative routes that may not be exposed through the normal user interface.

I would examine:

- JavaScript files.
- Mobile application source and resources.
- API specifications.
- Burp Suite traffic.
- Existing Postman collections.
- Common endpoint naming conventions.
- Application documentation.

Possible route patterns include:

These are examples of route naming conventions, not proof that such endpoints exist.

Once I discover an endpoint, I would test it using an unauthenticated request and lower-privileged test accounts.

I would check whether:

- Authentication is required.
- Administrative roles are enforced.
- Sensitive information is exposed.
- HTTP methods are restricted appropriately.
- Object-level authorization is implemented.

Important point:

Security through obscurity is not an adequate access-control mechanism. Administrative endpoints must enforce authentication and authorization regardless of whether they are documented or visible in the frontend.

88. An API returns sensitive information in a 200 OK response. How would you determine whether it is an actual vulnerability?

Interview-ready answer:

I would not classify an issue based solely on the HTTP status code.

A 200 OK response indicates that the request was successfully processed according to the server's HTTP handling. It does not establish that the returned information is appropriate for the requesting user.

I would examine:

1. What information is returned.
2. Whether authentication is required.
3. Whether the user is authorized to access it.
4. Whether the data is sensitive.
5. Whether the response is expected for the business operation.
6. Whether another user can retrieve the same information.
7. Whether the information creates a realistic security impact.

For example, a user retrieving their own email address is normal.

However, a user retrieving another customer's account balance, identity documents or transaction history may indicate BOLA or sensitive information disclosure.

My validation process:

I use controlled test accounts, capture the request and response, establish resource ownership and verify the authorization failure.

I then document the exact exposed fields and affected user roles.

Remediation:

Implement proper object-level authorization and return only the information needed for the requested operation.

89. How would you test API authorization when you have multiple user roles?

Interview-ready answer:

I would create an authorization matrix covering each application role and the operations that role should be allowed to perform.

For example:

Operation	Customer	Support	Manager	Admin
View own profile	Yes	Yes	Yes	Yes
View all profiles	No	Limited	Yes	Yes

Modify user roles	No	No	No	Yes
Approve payment	No	No	Yes	Yes
Delete user	No	No	No	Yes

I would obtain authorized accounts for each role and capture API requests.

Then I would systematically test:

- Unauthenticated access.
- Lower-role access to higher-role functions.
- Horizontal access to other users' objects.
- Restricted object properties.
- Alternative HTTP methods.
- Direct endpoint access.
- Nested resource authorization.
- API version consistency.

I would validate both read and write operations.

What I would look for:

- BOLA.
- BFLA.
- Broken property-level authorization.
- Horizontal privilege escalation.
- Vertical privilege escalation.
- Tenant isolation failures.

Remediation:

Implement centralized authorization policies and enforce permissions at the backend for every function, object and sensitive property.

90. An API allows a user to purchase an item with a negative price. How would you investigate the business logic vulnerability?

Interview-ready answer:

I would investigate whether the application trusts client-controlled pricing information or fails to enforce the business rules associated with product purchases.

Suppose a product costs ₹5,000.

The normal request is:

I would inspect whether the backend independently retrieves the product price from its trusted source or relies on the price submitted by the client.

I would test controlled boundary values, including zero and negative quantities or prices, using a test environment.

I would also examine:

- Discount calculations.
- Tax calculations.
- Coupon validation.
- Currency conversion.
- Total amount calculation.
- Payment gateway amount verification.
- Order status transitions.

Important point:

A client-controlled price is not automatically exploitable if the backend ignores it and calculates the amount independently.

I must verify that the modified value actually affects the transaction.

Impact:

Financial loss, fraudulent purchases and manipulation of accounting or order records.

Remediation:

Calculate prices on the server using trusted product data. Validate quantities, discounts, taxes and final payment amounts independently.

91. How would you test duplicate transactions and payment race conditions?

Interview-ready answer:

I would investigate whether the application correctly prevents duplicate financial operations when the same request is submitted multiple times or concurrently.

Consider a payment API:

I would first establish the normal payment workflow using a test transaction.

Then I would test duplicate submissions and controlled concurrent requests.

My methodology:

1. Create a test order.
2. Capture the payment request.
3. Submit it once and observe the result.
4. Replay the same request.
5. Send controlled concurrent requests.
6. Examine payment records.
7. Check whether multiple charges or transactions are created.
8. Verify whether the order state remains consistent.

I would also inspect the use of idempotency keys.

For example:

The server should ensure that repeated submissions with the same applicable idempotency key do not create duplicate financial operations.

Impact:

Duplicate charges, incorrect account balances and financial inconsistencies.

Remediation:

Use idempotency controls, transactional database operations, unique transaction identifiers and atomic state transitions.

92. How would you test whether a password reset API can be abused to take over accounts?

Interview-ready answer:

I would assess the complete password-reset workflow, from requesting the reset to completing the password change.

A secure password reset mechanism should verify that the person requesting the change is authorized to reset the account.

Testing methodology:

Step 1: Request password reset

I identify the reset endpoint and understand the information required.

Step 2: Examine token generation

I check whether reset tokens are sufficiently random, unique and unpredictable.

Step 3: Check token expiration

I verify whether reset tokens expire after the intended validity period.

Step 4: Check single-use enforcement

I test whether a previously used token can be reused.

Step 5: Check account binding

I examine whether a reset token issued for one account can be used to reset another account.

Step 6: Test parameter manipulation

I investigate whether email addresses, user IDs or account identifiers can be changed during the reset workflow.

Step 7: Test OTP and rate limiting

I check whether verification codes are protected against guessing and replay.

Step 8: Verify session behavior

After a successful reset, I examine whether existing sessions or refresh tokens are invalidated according to the application's security requirements.

Potential vulnerabilities:

- Predictable reset tokens.
- Token reuse.
- Missing expiration.
- Account enumeration.
- OTP brute force.
- Weak account binding.
- Host header manipulation in reset links.
- Missing rate limiting.

Impact:

Account takeover.

Remediation:

Use cryptographically secure single-use tokens, bind them to the correct account, enforce expiration, rate-limit requests and securely invalidate relevant sessions.

93. How would you test OTP verification APIs for brute force and bypass vulnerabilities?

Interview-ready answer:

I would examine the OTP lifecycle, including generation, delivery, verification, expiration and invalidation.

Consider this endpoint:

I would check the following:

1. Is the OTP sufficiently random?
2. Does it expire?
3. Is it single-use?
4. Are attempts rate-limited?
5. Is the OTP bound to the correct user and transaction?
6. Can it be reused?
7. Can it be bypassed by changing request parameters?
8. Does the application distinguish valid and invalid account states?
9. Are OTPs invalidated after successful verification?
10. Can parallel requests bypass attempt restrictions?

Example testing scenarios:

- Submit incorrect OTP values repeatedly.
- Replay a successfully used OTP.
- Use an expired OTP.
- Test an OTP against another account.
- Attempt to reuse an OTP for a different transaction.
- Check whether verification and transaction confirmation are properly linked.

Impact:

Account takeover, unauthorized password resets and fraudulent transactions.

Remediation:

Use cryptographically secure OTP generation, short validity periods, single-use enforcement, account binding, rate limiting and transaction-specific verification.

94. How would you test API authentication when access tokens are stored in mobile applications?

Interview-ready answer:

I would assess both the mobile application's token storage and the backend's token validation.

First, I install the application in an authorized Android testing environment and route its traffic through Burp Suite.

I capture the login and subsequent API requests to identify the authentication mechanism.

Then I examine how the application stores tokens.

Mobile-side testing:

- SharedPreferences.
- EncryptedSharedPreferences.
- Android Keystore integration.
- SQLite databases.
- Application files.
- Logs.
- Backup data.

- Memory and runtime exposure.

I use ADB, JADX, MobSF and, where appropriate, Frida to inspect the application.

Backend-side testing:

- Token signature validation.
- Token expiration.
- Refresh token handling.
- Token revocation.
- Audience and issuer validation.
- Authorization scope.
- Cross-user token reuse.

I also check whether tokens can be extracted from insecure local storage and subsequently used to access APIs.

Impact:

A stolen access token may allow an attacker to impersonate the user and access protected resources.

Remediation:

Use appropriate secure storage, avoid hardcoded secrets, minimize token lifetime, protect refresh tokens and enforce proper server-side token validation.

95. How would you assess the security of APIs communicating with third-party services?

Interview-ready answer:

I would examine the trust relationship and communication between the application, its backend APIs and third-party service providers.

Examples include:

- Payment gateways.
- SMS providers.
- Email services.
- Identity verification providers.
- Cloud storage.

- External analytics services.

My testing approach:

1. Identify third-party integrations.
2. Understand what information is exchanged.
3. Identify authentication mechanisms.
4. Examine API keys and secrets.
5. Verify TLS configuration.
6. Review request and response validation.
7. Test webhook authentication.
8. Examine callback URL handling.
9. Verify whether sensitive information is unnecessarily shared.
10. Check whether third-party failures are handled securely.

Example: Payment gateway

Suppose an application initiates a payment through a payment provider.

I would check whether the backend verifies the payment status directly with the trusted provider rather than trusting a client-side success parameter.

I would also examine transaction identifiers, amount verification, callback authentication and duplicate notification handling.

Impact:

Depending on the weakness, attackers may manipulate payment status, access third-party data, steal credentials or abuse integrations.

Remediation:

Use secure secret management, authenticated service communication, signed webhooks, strict request validation and server-side verification of third-party responses.

96. How would you test an API gateway for security misconfigurations?

Interview-ready answer:

I would evaluate the gateway as a critical security boundary between external clients and backend services.

First, I identify the gateway architecture, authentication mechanism, backend services and routing configuration.

Then I assess the following areas:

1. Authentication enforcement

Can unauthenticated clients access protected routes?

2. Authorization

Are role and scope checks consistently enforced?

3. Rate limiting

Can attackers bypass request limits?

4. HTTP methods

Are unnecessary or dangerous methods exposed?

5. Direct backend access

Can services be accessed without passing through the gateway?

6. CORS

Are cross-origin policies too permissive?

7. TLS

Is secure communication enforced?

8. Request validation

Are malformed or unexpected requests handled safely?

9. Header trust

Are client-controlled forwarding headers trusted incorrectly?

10. Error handling

Are internal infrastructure details exposed?

11. Routing

Can path normalization or routing inconsistencies expose unintended services?

12. Logging and monitoring

Are sensitive values logged, and are security events monitored?

Impact:

Gateway misconfiguration may allow authentication bypass, unauthorized backend access, data exposure or denial of service.

Remediation:

Apply secure gateway configuration, consistent backend authorization, network isolation, strict routing policies and centralized monitoring.

97. How would you identify and exploit insecure webhook implementations?

Interview-ready answer:

A webhook is an HTTP callback mechanism through which one system sends event notifications to another system.

For example, a payment provider may send a webhook to an e-commerce backend after a transaction succeeds.

A typical webhook request might look like:

If the application trusts this request without verifying its authenticity, an attacker may attempt to forge payment-success notifications.

My testing methodology:

1. Identify webhook endpoints.
2. Understand which services are authorized to call them.
3. Examine signature validation.
4. Check timestamp validation.
5. Test replay protection.

6. Verify transaction identifiers.
7. Check amount and currency validation.
8. Examine whether webhook events are idempotent.
9. Verify that the backend confirms sensitive transaction states independently.

Potential vulnerabilities:

- Missing webhook signature validation.
- Weak signing secrets.
- Replay attacks.
- Forged payment status.
- Duplicate event processing.
- Insecure callback handling.

Impact:

Fraudulent transactions, unauthorized order fulfillment and financial manipulation.

Remediation:

Use HMAC or provider-supported digital signatures, validate timestamps, enforce replay protection, verify transaction details and process events idempotently.

98. How would you test an API that uses multiple authentication mechanisms?

Interview-ready answer:

I would identify each authentication mechanism and understand which endpoints, users and services are supposed to use it.

An application might support:

- JWT authentication.
- Session cookies.
- API keys.
- OAuth 2.0.
- Mutual TLS.
- Basic Authentication for legacy integrations.

The main risk is inconsistent authentication enforcement across different endpoints or authentication methods.

My methodology:

1. Identify all authentication schemes.
2. Map each scheme to its corresponding endpoints.
3. Test unauthenticated access.
4. Test invalid and expired credentials.
5. Test whether one authentication mechanism can access endpoints intended for another.
6. Examine token audience and scope.
7. Check whether API keys provide excessive privileges.
8. Test authentication fallback behavior.
9. Check whether session and token invalidation are consistent.
10. Verify that backend services do not bypass authentication.

Example:

Suppose an administrative endpoint requires JWT authentication but also accepts an older API key mechanism.

If the legacy API key has excessive permissions or is not properly restricted, the endpoint may be accessible through a weaker authentication path.

Remediation:

Standardize authentication policies, restrict legacy mechanisms, apply least privilege and consistently enforce authorization across all supported authentication methods.

99. How would you perform a complete API security assessment from reconnaissance to reporting?

Interview-ready answer:

I follow a structured API penetration testing methodology covering discovery, authentication, authorization, input validation, business logic and reporting.

Phase 1: Scope and information gathering

I collect:

- Base URLs.
- API documentation.

- Swagger/OpenAPI files.
- Postman collections.
- Test credentials.
- User roles.
- Architecture diagrams.
- Third-party integration details.

Phase 2: API discovery

I identify documented and undocumented endpoints through traffic analysis, source-code inspection where available and authorized enumeration.

Phase 3: Authentication testing

I test:

- Login functionality.
- Token generation.
- JWT validation.
- OAuth flows.
- OTP.
- Password reset.
- Refresh tokens.
- Session management.

Phase 4: Authorization testing

I perform:

- BOLA.
- BFLA.
- Property-level authorization testing.
- Horizontal privilege escalation.
- Vertical privilege escalation.
- Tenant isolation testing.

Phase 5: Input validation testing

I test:

- SQL injection.

- NoSQL injection.
- Command injection.
- XXE.
- SSRF.
- Mass assignment.
- Parameter pollution.
- Type confusion.

Phase 6: Business logic testing

I investigate:

- Payments.
- Transactions.
- Discounts.
- Coupons.
- Race conditions.
- Workflow manipulation.
- Duplicate requests.

Phase 7: API configuration testing

I examine:

- CORS.
- Rate limiting.
- TLS.
- HTTP methods.
- Security headers.
- Error messages.
- API versioning.
- Cache behavior.

Phase 8: Reporting

For each confirmed vulnerability, I document:

- Vulnerability title.
- Affected endpoint.
- Severity.

- Description.
- Technical root cause.
- Steps to reproduce.
- Request and response evidence.
- Business impact.
- Remediation recommendation.
- Relevant OWASP/CWE mapping.

Phase 9: Retesting

After remediation, I repeat the original test cases and verify whether the vulnerability is fixed without introducing regressions.

What makes a good API penetration tester?

The ability to understand the business workflow and identify authorization and logic weaknesses, not merely run automated scanners.

100. You discover a critical API vulnerability in production. How would you validate, demonstrate impact and recommend remediation without affecting business operations?

Interview-ready answer:

If I discover a critical vulnerability in a production API, my first priority is to validate it safely while minimizing operational and customer impact.

I would follow the agreed rules of engagement and communicate with the project stakeholders if the finding presents an immediate risk.

Step 1: Understand the vulnerability

I identify the affected endpoint, authentication requirements, vulnerable parameter and expected application behavior.

Step 2: Establish a baseline

I use an authorized test account and capture a normal request and response.

Step 3: Validate using controlled test data

I avoid accessing real customer information or modifying production records unnecessarily.

For example, if I suspect BOLA, I use two client-approved test accounts rather than attempting to retrieve random customer records.

Step 4: Demonstrate the security impact

I demonstrate the minimum necessary proof of exploitation.

For example, I may show that User A can access a harmless test record belonging to User B.

Step 5: Preserve evidence

I capture:

- HTTP requests.
- HTTP responses.
- Timestamps.
- Affected endpoint.
- Test account details.
- Relevant screenshots.
- Reproduction steps.

I remove unnecessary sensitive information from the evidence.

Step 6: Assess severity

I evaluate:

- Confidentiality impact.
- Integrity impact.
- Availability impact.
- Exploitability.
- Privileges required.
- Number of affected users.
- Business consequences.

I use CVSS as a supporting framework, alongside the organization's risk criteria.

Step 7: Communicate immediately

For a critical finding, I notify the designated security contact or project manager according to the engagement escalation procedure.

Step 8: Recommend remediation

I explain the root cause and recommend an appropriate fix.

For BOLA, for example, I recommend server-side object-level authorization rather than relying on unpredictable identifiers.

Step 9: Retest

After the fix is deployed, I verify that the original exploit no longer works and check for regressions.

What I would tell the interviewer:

"My responsibility is not simply to prove that a vulnerability exists. I need to demonstrate its business impact, provide reproducible evidence and help the development team remediate it without unnecessarily disrupting production."

PART 2: WEB APPLICATION PENETRATION TESTING

Questions 1–100 **Basic to Expert**

Web application testing is particularly important for an AppSec consultant because interviewers expect you to understand the complete HTTP lifecycle, browser security, server-side processing, authentication, authorization and exploitation.

Level 1: Web Fundamentals

1. What is the difference between HTTP and HTTPS?

Interview-ready answer:

HTTP stands for Hypertext Transfer Protocol. It is an application-layer protocol used for communication between clients and web servers.

HTTPS stands for Hypertext Transfer Protocol Secure. It is HTTP communication protected by TLS (Transport Layer Security).

The main difference is that HTTP does not provide transport encryption by itself, whereas HTTPS uses TLS to provide confidentiality, integrity and server authentication.

HTTP example:

If this communication occurs over plain HTTP, network observers may be able to read or modify the traffic.

HTTPS example:

HTTPS protects the HTTP communication using TLS.

How TLS works at a high level:

1. The client connects to the server.
2. The server presents its TLS certificate.
3. The client validates the certificate chain and hostname.
4. The client and server negotiate cryptographic parameters.
5. They establish shared session keys.
6. HTTP data is transmitted through the encrypted connection.

Security benefits of HTTPS:

- Confidentiality: Prevents unauthorized parties from reading traffic.
- Integrity: Helps detect modification of transmitted data.
- Authentication: Allows the client to verify the server's identity.

How I test HTTPS during penetration testing:

- Check whether HTTP redirects to HTTPS.
- Examine TLS protocol versions.
- Check certificate validity and hostname matching.
- Identify weak cipher configurations.
- Check certificate-chain issues.
- Verify HSTS.
- Identify sensitive resources loaded over HTTP.
- Examine mixed-content issues.

Likely follow-up: Does HTTPS prevent SQL injection or XSS?

No. HTTPS protects communication in transit. It does not prevent application-layer vulnerabilities such as SQL injection, XSS, IDOR or authentication bypass.

2. Explain the complete HTTP request-response lifecycle.

Interview-ready answer:

The HTTP request-response lifecycle describes how a client communicates with a web server and receives a response.

Suppose I enter:

in a browser.

The process involves several stages.

Step 1: URL parsing

The browser identifies:

- Scheme: HTTPS.
- Host: example.com.
- Path: /login.

Step 2: DNS resolution

The browser resolves the domain name into an IP address through DNS.

Step 3: Network connection

The client establishes a connection to the server, usually using TCP for HTTP/1.1 or HTTP/2, or QUIC for HTTP/3.

Step 4: TLS handshake

For HTTPS, the client and server negotiate TLS parameters and establish encryption keys.

Step 5: HTTP request

The browser sends an HTTP request.

Step 6: Request processing

The request may pass through:

- CDN.
- WAF.
- Reverse proxy.
- Load balancer.
- Web server.
- Application framework.

The backend processes the request, applies authentication and authorization checks, executes business logic and interacts with databases or other services.

Step 7: HTTP response

The server returns an HTTP response.

The browser processes the response and renders the page.

Step 8: Additional resources

The browser may request JavaScript, CSS, images, fonts and other resources.

Security perspective:

During penetration testing, I analyze the complete request-response flow because vulnerabilities can exist at multiple layers.

For example:

- DNS and infrastructure issues.
- TLS misconfiguration.
- WAF and proxy misconfiguration.
- HTTP request smuggling.
- Authentication failures.
- Server-side injection.
- Insecure response headers.
- Browser-side vulnerabilities.

3. What happens when you enter a URL in a browser?

Interview-ready answer:

When a user enters a URL in a browser, the browser performs several operations before displaying the requested webpage.

I would explain it in the following sequence.

1. The browser parses the URL and identifies its protocol, domain, port and path.
2. It checks relevant browser caches and determines whether DNS resolution is required.
3. It resolves the domain through DNS.
4. It establishes a network connection with the destination.
5. If HTTPS is used, it performs the TLS handshake and validates the certificate.
6. It sends an HTTP request.
7. The request may pass through a CDN, WAF, reverse proxy or load balancer.
8. The web server routes the request to the application.
9. The backend executes business logic and may communicate with databases or external services.
10. The server returns an HTTP response.
11. The browser parses HTML, builds the DOM and processes CSS and JavaScript.
12. Additional resources are downloaded and the page is rendered.

Security perspective:

This process introduces multiple attack surfaces.

For example, DNS spoofing can affect domain resolution, TLS misconfiguration can affect communication security, request smuggling can affect proxy/backend communication, and XSS can affect browser-side execution.

4. What is DNS, and how does DNS resolution work?

Interview-ready answer:

DNS stands for Domain Name System.

It is a distributed naming system that translates human-readable domain names into IP addresses.

For example:

may resolve to an IP address such as:

Without DNS, users would generally need to remember IP addresses instead of domain names.

DNS resolution process:

1. The browser or operating system checks its DNS cache.
2. If the record is not cached, the resolver is contacted.
3. The resolver may query root DNS servers.
4. The root server directs the resolver to the appropriate Top-Level Domain server.
5. The TLD server directs the resolver to the authoritative DNS server.
6. The authoritative server provides the requested record.
7. The resolver returns the result to the client.
8. The result may be cached according to its TTL.

Important DNS records:

Record	Purpose
A	IPv4 address
AAAA	IPv6 address
CNAME	Canonical name alias
MX	Mail exchange
NS	Name server
TXT	Text information, commonly used for verification and email security
PTR	Reverse DNS lookup
SOA	Zone authority information

Security perspective:

During penetration testing and reconnaissance, I examine:

- DNS misconfigurations.
- Subdomain enumeration.
- Dangling DNS records.
- Subdomain takeover possibilities.
- DNS zone transfer exposure.
- Sensitive information in DNS records.

Likely follow-up: What is DNS zone transfer?

DNS zone transfer is a mechanism for replicating DNS zone information between DNS servers. If an authoritative server improperly allows unrestricted zone transfers, an attacker may obtain information about internal or external DNS records.

5. What is the difference between HTTP/1.1, HTTP/2 and HTTP/3?

Interview-ready answer:

HTTP/1.1, HTTP/2 and HTTP/3 are different versions of the HTTP protocol.

They provide the same fundamental purpose of transferring application data but differ in how they manage connections and transport.

HTTP/1.1

- Uses text-based request and response messages.
- Typically uses TCP.
- Supports persistent connections.
- Can experience application-layer head-of-line blocking with multiple requests on a connection.

HTTP/2

- Uses binary framing.
- Supports multiplexing multiple streams over a single connection.
- Supports header compression through HPACK.
- Reduces the need for multiple TCP connections.
- Still uses TCP.

HTTP/3

- Uses QUIC over UDP.
- Supports multiplexing.
- Uses QPACK for header compression.
- Avoids TCP-level head-of-line blocking between independent QUIC streams.
- Integrates TLS 1.3 into QUIC.

Comparison:

Feature	HTTP/1.1	HTTP/2	HTTP/3

Transport	TCP	TCP	QUIC/UDP
Framing	Text	Binary	Binary
Multiplexing	Limited	Yes	Yes
Header compression	No standard mechanism	HPACK	QPACK
TLS	Optional	Common in browsers	Integrated through QUIC

Security perspective:

During testing, I consider protocol-specific behavior.

For example:

- HTTP/2 request parsing.
- HTTP/2-to-HTTP/1.1 translation.
- Request smuggling.
- Header normalization.
- Proxy/backend discrepancies.
- HTTP/3 configuration and transport security.

Likely follow-up: Is HTTP/2 automatically more secure than HTTP/1.1?

No. HTTP/2 improves communication efficiency, but security depends on correct implementation, TLS configuration, request parsing and intermediary handling.

6. What are HTTP request and response headers?

Interview-ready answer:

HTTP headers are metadata fields exchanged between the client and server.

Request headers provide information about the client request, while response headers provide information about the server response.

Example request:

Example response:

Important request headers:

- Host.
- Authorization.
- Cookie.
- Content-Type.
- Accept.
- Origin.
- Referer.
- User-Agent.

Important response headers:

- Content-Type.
- Content-Length.
- Set-Cookie.
- Cache-Control.
- Strict-Transport-Security.
- Content-Security-Policy.
- X-Content-Type-Options.
- Access-Control-Allow-Origin.

Security perspective:

I inspect request and response headers for:

- Insecure cookie attributes.
- Missing or weak security policies.
- CORS misconfiguration.
- Host header injection.
- Sensitive information disclosure.
- Cache-related issues.
- Trust in client-controlled headers.

7. What are cookies, and how do they work?

Interview-ready answer:

Cookies are small pieces of data stored by a browser and associated with a website.

They are commonly used for session management, authentication, user preferences and tracking.

For example, after a successful login, a server may send:

The browser stores the cookie and automatically sends it with subsequent matching requests.

The backend uses the session identifier to identify the user's session.

Important cookie attributes:

Secure: The cookie is sent only over secure connections, subject to browser cookie rules.

HttpOnly: Prevents JavaScript from accessing the cookie through APIs such as `document.cookie`.

SameSite: Controls when cookies are sent in cross-site contexts and helps mitigate CSRF.

Domain: Controls which hosts may receive the cookie.

Path: Restricts the cookie to matching URL paths.

Expires/Max-Age: Controls cookie lifetime.

Security testing perspective:

I check:

- Missing Secure.
- Missing HttpOnly where appropriate.
- Insecure SameSite configuration.
- Overly broad Domain and Path attributes.
- Session fixation.
- Session hijacking.
- Cookie reuse after logout.
- Sensitive data stored directly in cookies.

Likely follow-up: Does HttpOnly prevent XSS?

No. HttpOnly prevents JavaScript from directly reading that cookie, but XSS can still execute actions within the victim's browser session.

8. What is a session, and how is it maintained?

Interview-ready answer:

A session represents a period of interaction between a user and an application.

Because HTTP is fundamentally stateless, applications commonly use session-management mechanisms to maintain authentication and user context across multiple requests.

Session creation:

1. User submits login credentials.
2. Backend validates them.
3. Backend creates a session.
4. Backend generates a unique session identifier.
5. Session identifier is returned to the browser through a cookie.
6. Browser sends the cookie with subsequent requests.
7. Backend retrieves the associated session and processes the request.

Example:

The server may maintain information such as:

- User identity.
- Authentication status.
- Session creation time.
- Expiration time.
- User privileges.
- Session metadata.

Session security testing:

I examine:

- Session ID randomness.
- Session fixation.
- Session hijacking.
- Session expiration.
- Logout invalidation.
- Concurrent session handling.
- Cookie security.
- Session renewal after login.
- Session invalidation after password changes.

Important point:

Session identifiers should be generated using cryptographically secure randomness and must not be predictable.

9. What is the difference between session-based and token-based authentication?

Interview-ready answer:

Session-based and token-based authentication are two approaches to maintaining authenticated communication between a client and server.

Session-based authentication:

The server creates a session after successful login and stores session information.

The client receives a session identifier, usually through a cookie.

For every subsequent request, the server uses that identifier to retrieve the session.

Token-based authentication:

The server issues an access token after successful authentication.

The client sends the token with subsequent requests.

For example:

The server validates the token and determines the identity and applicable permissions.

Comparison:

Feature	Session-based	Token-based
Session state	Usually server-side	May be self-contained
Common transport	Cookie	Authorization header
Revocation	Often straightforward	Depends on token design
Scalability	Requires session management	Can support distributed validation
Common risks	Session fixation, hijacking, CSRF	Token theft, weak validation, excessive privileges

Security perspective:

I don't consider one universally more secure.

For session-based systems, I focus on session management and cookie security.

For token-based systems, I focus on token integrity, expiration, revocation, secure storage and authorization.

10. What are Secure, HttpOnly and SameSite cookie attributes?

Interview-ready answer:

These are important cookie security attributes that help protect browser-based sessions.

1. Secure

The Secure attribute instructs the browser to send the cookie only over secure connections.

Example:

This helps protect cookies against exposure over unencrypted HTTP.

2. HttpOnly

The HttpOnly attribute prevents JavaScript from accessing the cookie through browser scripting interfaces such as `document.cookie`.

Example:

It reduces the risk of session-cookie theft through XSS, although it does not prevent XSS itself.

3. SameSite

SameSite controls whether cookies are sent with cross-site requests.

Three common values are:

- Strict: Restricts cookies to same-site contexts.
- Lax: Allows cookies in certain cross-site top-level navigations, subject to browser rules.
- None: Allows cross-site cookie use, but requires Secure in modern browsers.

Example:

How I test:

I inspect the Set-Cookie response header in Burp Suite.

I check whether authentication cookies have appropriate attributes based on the application's requirements.

Likely follow-up: Which SameSite setting provides the strongest CSRF protection?

Strict generally imposes the strongest cross-site sending restrictions, but it may affect legitimate navigation and SSO workflows. The appropriate choice depends on application functionality and the complete CSRF defense.

11. What is the difference between GET and POST requests?

Interview-ready answer:

GET and POST are HTTP methods used for different types of operations.

GET is intended to retrieve a representation of a resource, while POST is generally used to submit data for processing or create a resource.

GET example:

POST example:

Key differences:

- GET is a safe method by HTTP semantics.
- POST is not defined as a safe method.
- GET is idempotent.
- POST is not inherently idempotent.
- GET responses may be cached.
- POST is generally used to submit information or trigger processing.

Security perspective:

I check whether sensitive data is unnecessarily included in URL parameters, because URLs can appear in browser history, logs and referrer information.

I also check whether state-changing operations are incorrectly implemented through GET.

For example, deleting an account through a GET request may create CSRF and caching-related risks.

12. What is the Same-Origin Policy?

Interview-ready answer:

The Same-Origin Policy, or SOP, is a browser security mechanism that restricts how documents and scripts from one origin interact with resources belonging to another origin.

An origin is defined by three components:

1. Scheme.
2. Host.
3. Port.

For example:

This is one origin.

Compare:

These are different origins because at least one origin component differs.

The Same-Origin Policy prevents JavaScript running on one origin from freely reading sensitive data from another origin.

Example:

Suppose a user is logged into a banking website.

They visit an attacker-controlled website in another tab.

Without browser security restrictions, malicious JavaScript on the attacker's website might attempt to read the banking website's protected content.

SOP restricts such cross-origin access.

Important distinction:

SOP does not block every cross-origin request. Browsers allow certain cross-origin resource loads and form submissions. CORS provides controlled cross-origin access to responses.

Security perspective:

I examine SOP-related issues through:

- CORS testing.
- Cross-origin data access.
- PostMessage security.
- Cross-origin iframe behavior.
- CSRF.
- DOM-based vulnerabilities.

13. What is CORS, and how does it work?

Interview-ready answer:

CORS stands for Cross-Origin Resource Sharing.

It is a browser mechanism that allows servers to specify which external origins are permitted to access their resources.

Suppose the frontend is hosted at:

And the API is hosted at:

These are different origins.

The browser may send a request with:

The server can respond:

This indicates that the specified origin is allowed to access the response, subject to other CORS requirements.

Important CORS headers:

- Access-Control-Allow-Origin.
- Access-Control-Allow-Methods.
- Access-Control-Allow-Headers.
- Access-Control-Allow-Credentials.
- Access-Control-Expose-Headers.
- Access-Control-Max-Age.

Preflight request:

For certain requests, the browser first sends an OPTIONS request to determine whether the actual cross-origin request is permitted.

Security perspective:

I test:

- Origin reflection.
- Wildcard origins.
- Null origin.
- Credentialed cross-origin requests.
- Subdomain trust.
- Incorrect preflight handling.
- Insecure origin allowlists.

I always validate whether the browser can actually read sensitive responses, rather than reporting CORS based solely on a header.

14. What is a reverse proxy?

Interview-ready answer:

A reverse proxy is a server that sits between clients and backend servers.

Instead of directly communicating with the backend, clients send requests to the reverse proxy, which forwards them to the appropriate backend server.

For example:

Common reverse proxy technologies include Nginx, HAProxy and Apache HTTP Server.

Why reverse proxies are used:

- Load balancing.
- TLS termination.
- Request routing.
- Caching.
- Compression.
- Centralized access logging.

- Backend server protection.

Security perspective:

I examine:

- Incorrect forwarding headers.
- Trust boundary issues.
- Request smuggling.
- Path normalization inconsistencies.
- Direct backend exposure.
- Misconfigured caching.
- Inconsistent security headers.

15. What is a Web Application Firewall (WAF)?

Interview-ready answer:

A Web Application Firewall is a security control designed to monitor and filter HTTP traffic between clients and web applications.

It helps identify and block malicious requests based on configured rules, signatures, behavioral patterns and other detection mechanisms.

A WAF can protect applications against common web attacks such as:

- SQL injection.
- Cross-Site Scripting.
- Path traversal.
- Malicious file uploads.
- Known exploit patterns.
- Certain automated attacks.

Types of WAF:

1. Network-based WAF.
2. Host-based WAF.
3. Cloud-based WAF.

How it works:

A request passes through the WAF before reaching the backend application.

The WAF inspects the request and decides whether to allow, block, log or challenge it.

Security testing perspective:

I examine whether the WAF is correctly configured and whether it creates false confidence in the application's security.

I test whether application-layer validation remains effective when requests reach the backend.

I also assess:

- False positives.
- False negatives.
- Encoding inconsistencies.
- Request normalization.
- HTTP method handling.
- API coverage.
- Direct backend access.

Important interview point:

A WAF is a defense-in-depth control. It does not replace secure coding, parameterized queries, authorization or proper input validation.

16. What is input validation?

Interview-ready answer:

Input validation is the process of checking whether user-supplied data meets the expected format, type, size and business rules before processing it.

For example, if an application accepts an age parameter, it should validate that the input is a valid integer within the permitted range.

A secure application should validate input on the server side, even if frontend validation is already implemented.

Two common approaches:

Allowlisting: Accept only expected values or formats.

For example, if a country field accepts a defined set of country codes, reject values outside that list.

Blocklisting: Reject known dangerous patterns.

Blocklisting can provide additional protection, but it is generally insufficient as the primary security control because attackers can often find alternative representations.

Security perspective:

I examine:

- Type validation.
- Length validation.
- Range validation.
- Format validation.
- Business rule validation.
- Encoding and normalization.
- Unexpected parameter handling.

Important distinction:

Input validation does not replace output encoding, parameterized SQL queries or safe command execution.

Each security control addresses a different stage of processing.

17. What is output encoding?

Interview-ready answer:

Output encoding is the process of converting potentially dangerous characters into a safe representation appropriate for the context in which data is displayed.

It is an important defense against Cross-Site Scripting.

Suppose an application accepts a username:

If the application inserts this value directly into an HTML page, the browser may interpret it as executable JavaScript.

If the application correctly encodes the value for an HTML text context, special characters are represented safely instead of being interpreted as HTML markup.

For example:

Important encoding contexts:

- HTML text.
- HTML attributes.
- JavaScript strings.
- CSS values.
- URL parameters.

Each context requires appropriate encoding.

Security perspective:

I test reflected, stored and DOM-based XSS by identifying where user-controlled data is inserted into the page and whether it is correctly encoded for that context.

Likely follow-up: Is HTML encoding sufficient everywhere?

No. HTML encoding is not interchangeable with JavaScript, CSS or URL encoding. Context-specific encoding is required.

18. What is the difference between client-side and server-side validation?

Interview-ready answer:

Client-side validation happens in the user's browser or application before the request is submitted.

Server-side validation occurs on the backend after receiving the request.

Client-side example:

A registration form requires a valid email address.

JavaScript checks the email format before allowing form submission.

Server-side example:

The backend receives the email and independently validates its format, length and business rules.

Key differences:

Client-side	Server-side
Runs in browser	Runs on backend
Improves usability	Enforces security
Can be modified or bypassed	Controlled by application
Provides immediate feedback	Validates actual submitted data

How I test:

I use Burp Suite to intercept requests and modify parameters that may have been validated in the frontend.

For example, if the frontend restricts an amount to positive integers, I submit a negative amount directly to the backend.

I check whether the backend independently enforces the rule.

Important interview point:

Client-side validation is not a security boundary. All security-sensitive validation must be enforced on the server.

19. What is the OWASP Top 10?

Interview-ready answer:

The OWASP Top 10 is a widely recognized awareness document that identifies important security risks affecting web applications.

It helps developers, testers and security professionals understand common classes of application vulnerabilities.

The 2025 edition is the latest release in this series.

Its categories are:

1. Broken Access Control.
2. Security Misconfiguration.

3. Software Supply Chain Failures.
4. Cryptographic Failures.
5. Injection.
6. Insecure Design.
7. Authentication Failures.
8. Software or Data Integrity Failures.
9. Security Logging and Alerting Failures.
10. Mishandling of Exceptional Conditions.

How I use it in penetration testing:

I map confirmed vulnerabilities to relevant OWASP categories.

For example:

Finding	OWASP category
IDOR	A01 Broken Access Control
Missing security configuration	A02 Security Misconfiguration
Vulnerable dependencies	A03 Software Supply Chain Failures
Weak cryptography	A04 Cryptographic Failures
SQL injection	A05 Injection
Missing security controls in business workflows	A06 Insecure Design
Authentication bypass	A07 Authentication Failures
Insecure deserialization or integrity issues	A08 Software or Data Integrity Failures
Insufficient security monitoring	A09 Security Logging and Alerting Failures
Improper exception handling	A10 Mishandling of Exceptional Conditions

Important interview point:

OWASP Top 10 is an awareness and risk-classification resource, not a complete testing checklist. A comprehensive assessment also requires business logic testing, architecture review and domain-specific testing.

20. What is the difference between vulnerability scanning and penetration testing?

Interview-ready answer:

Vulnerability scanning is generally an automated process that identifies known vulnerabilities, outdated software, misconfigurations and other security weaknesses.

Penetration testing is a controlled security assessment in which a tester actively investigates and validates whether weaknesses can be exploited and what their real-world impact would be.

Vulnerability scanning:

Tools such as Nessus, Qualys, Nuclei and other scanners identify potential security issues.

For example, a scanner may detect an outdated web server version or missing security header.

Penetration testing:

A tester manually validates vulnerabilities and investigates attack paths.

For example, a scanner may identify an API endpoint accepting a user ID parameter.

A penetration tester may use two authorized accounts to establish whether changing that identifier allows access to another user's information.

Comparison:

Feature	Vulnerability scanning	Penetration testing
Approach	Primarily automated	Manual and automated
Goal	Identify potential weaknesses	Validate exploitability and impact
Business logic	Limited	Important focus
False positives	May be higher	Manually validated
Exploitation	Usually limited	Controlled and authorized
Reporting	Scanner findings	Validated security findings

Important point:

Vulnerability scanning and penetration testing complement each other. Neither completely replaces the other.

21. What is reconnaissance in web penetration testing?

Interview-ready answer:

Reconnaissance is the initial information-gathering phase of penetration testing.

Its purpose is to understand the target application's architecture, exposed technologies, available functionality and potential attack surface before performing deeper security testing.

I divide reconnaissance into two approaches.

Passive reconnaissance:

Collecting information without directly interacting with the target infrastructure in a way that generates active probing.

Examples:

- Public DNS information.
- Search engine results.
- Public documentation.
- Certificate transparency records.
- Publicly available technology information.

Active reconnaissance:

Interacting directly with authorized target systems to identify exposed resources.

Examples:

- HTTP header inspection.
- Technology fingerprinting.
- Directory discovery.
- Endpoint enumeration.
- Port scanning, when included in scope.
- API discovery.

Tools:

- Burp Suite.
- Nmap.
- WhatWeb.
- Wappalyzer.
- FFUF.
- Dirsearch.
- Amass.
- Subfinder.

Security perspective:

I use reconnaissance to identify technologies, authentication mechanisms, API endpoints, administrative functionality and potential attack surfaces.

The objective is to build a map of the application before moving into vulnerability testing.

22. What is directory and file enumeration?

Interview-ready answer:

Directory and file enumeration is the process of identifying resources that may exist on a web server but are not directly linked or visible through the application's normal interface.

These may include:

- Administrative directories.
- Backup files.
- Configuration files.
- Debug endpoints.
- Development resources.
- Old application versions.
- Exposed documentation.
- Sensitive files.

For example, an application may expose:

These paths are examples of common naming patterns.

How I perform enumeration:

I use wordlists and authorized content discovery tools to identify accessible paths.

Common tools include:

- Dirsearch.
- FFUF.
- Gobuster.
- Burp Suite.
- Feroxbuster.

Example:

I examine the returned status codes, response sizes, redirects and page content.

Security perspective:

I look for exposed backups, source files, configuration files, administrative interfaces and sensitive documentation.

I also verify that discovered resources do not expose credentials, secrets or confidential information.

23. What is content discovery, and which tools are used?

Interview-ready answer:

Content discovery is the process of identifying application resources such as pages, directories, files, API routes and other functionality that may not be directly linked.

It helps a penetration tester understand the complete attack surface.

Common techniques:

1. Directory brute forcing.
2. File extension discovery.
3. JavaScript analysis.
4. API endpoint discovery.
5. Robots.txt and sitemap inspection.
6. Backup-file discovery.
7. Virtual host discovery.
8. Application crawling.

Tools:

- FFUF.
- Dirsearch.
- Gobuster.
- Burp Suite Content Discovery.
- Katana.
- Feroxbuster.
- LinkFinder.
- Custom scripts.

Example:

I analyze results rather than treating every discovered path as a vulnerability.

For example, finding `/admin/` is not automatically a security issue. I must investigate whether the endpoint is properly authenticated and authorized.

24. What is Burp Suite, and how do you use it during web penetration testing?

Interview-ready answer:

Burp Suite is an integrated web application security testing platform developed by PortSwigger.

It acts as an intercepting proxy between the browser and the web application, allowing testers to inspect, modify, replay and analyze HTTP traffic.

Important Burp Suite components:

1. Proxy

Intercepts and modifies HTTP requests and responses.

2. Repeater

Allows manual modification and repeated submission of requests.

3. Intruder

Automates customized request variations for testing.

4. Scanner

Available in supported commercial editions for automated vulnerability scanning.

5. Decoder

Encodes and decodes different data formats.

6. Comparer

Compares requests and responses.

7. Sequencer

Analyzes randomness in tokens and session identifiers.

8. Logger

Helps review HTTP communication.

My testing workflow:

1. Configure the browser to use Burp Proxy.
2. Browse the application and capture traffic.
3. Map endpoints and application functionality.
4. Identify authentication and session mechanisms.
5. Send interesting requests to Repeater.
6. Modify parameters and headers.
7. Test authorization and input validation.
8. Use Intruder for controlled automation.
9. Review responses and identify vulnerabilities.
10. Save evidence for reporting.

Important point:

Burp Suite is a testing platform, not a substitute for understanding the application's business logic or manually validating security findings.

25. Explain the complete methodology you follow while conducting a web application penetration test.

Interview-ready answer:

I follow a structured methodology to ensure that the assessment covers the application's functionality, technical security controls and business logic.

Phase 1: Scope definition

I understand:

- Application URLs.
- Domains and subdomains.
- Test environment.
- User roles.
- Test credentials.
- In-scope functionality.
- Out-of-scope activities.
- Rules of engagement.

Phase 2: Reconnaissance

I identify the application's technologies, infrastructure, entry points, endpoints and available functionality.

Phase 3: Application mapping

I browse the complete application and identify:

- Authentication flows.
- User workflows.
- Administrative functionality.
- File uploads.
- Search functionality.
- APIs.
- Sensitive business operations.

Phase 4: Authentication testing

I assess login, password reset, MFA, session management, account recovery and logout functionality.

Phase 5: Authorization testing

I test IDOR, horizontal and vertical privilege escalation, role-based access control and access to restricted resources.

Phase 6: Input validation testing

I examine XSS, SQL injection, command injection, SSRF, SSTI, XXE, path traversal and file upload handling.

Phase 7: Client-side security

I test DOM-based XSS, JavaScript security, postMessage handling, CSP and browser-related security controls.

Phase 8: Business logic testing

I examine workflows such as payments, registration, account changes, approvals, discounts and transactions.

Phase 9: Configuration testing

I review TLS, cookies, HTTP headers, CORS, caching, error handling and exposed resources.

Phase 10: Reporting

I document confirmed vulnerabilities with severity, technical details, reproduction steps, impact and remediation.

Phase 11: Retesting

I validate fixes and ensure that the original vulnerabilities have been resolved.

What makes this methodology effective?

It combines automated tools, manual exploitation, architecture understanding and business logic analysis.

Level 2: Common Web Application Vulnerabilities

26. What is Cross-Site Scripting (XSS)?

Interview-ready answer:

Cross-Site Scripting, commonly known as XSS, is a client-side injection vulnerability that occurs when an application includes untrusted input in a webpage without appropriate context-specific output encoding or sanitization.

An attacker may inject malicious JavaScript or HTML that executes in another user's browser within the security context of the vulnerable website.

For example, suppose an application has a search parameter:

If the application reflects the search parameter into the webpage without proper encoding, an attacker may supply HTML or JavaScript that executes in the browser.

A basic proof-of-concept payload is:

If the payload executes, it demonstrates a possible XSS vulnerability.

Types of XSS:

1. Reflected XSS.
2. Stored XSS.
3. DOM-based XSS.

Potential impact:

- Unauthorized actions through the victim's browser.
- Access to sensitive page information.
- Session-related attacks.
- Phishing.
- Defacement.
- Manipulation of application functionality.

The impact depends on the browser context, application architecture and available security controls.

How I test:

I identify input points such as search fields, comments, profile information, URL parameters and request headers.

I submit controlled payloads and investigate whether the input is reflected or stored and whether it reaches an executable browser context.

I use Burp Suite to intercept requests and inspect responses.

Remediation:

- Context-specific output encoding.
- Safe DOM manipulation.
- Avoiding dangerous JavaScript sinks.
- HTML sanitization where HTML is intentionally permitted.
- Content Security Policy as defense in depth.

27. Explain reflected, stored and DOM-based XSS.

Interview-ready answer:

These are three common categories of Cross-Site Scripting.

1. Reflected XSS

Reflected XSS occurs when user-controlled input is immediately returned in the server-generated response without appropriate handling.

Example:

If the server inserts the search parameter into the response without proper encoding, a crafted URL may execute JavaScript when a victim opens it.

Example payload:

The payload is reflected in the response rather than being permanently stored.

2. Stored XSS

Stored XSS occurs when malicious input is saved by the application and later displayed to other users without proper output handling.

Example:

A user submits a comment:

The application stores it in the database.

Whenever another user visits the affected page, the browser processes the stored content in an executable context.

Stored XSS is particularly dangerous because it can affect multiple users without requiring each victim to open a specially crafted URL.

3. DOM-based XSS

DOM-based XSS occurs when client-side JavaScript processes untrusted data and inserts it into an unsafe DOM execution context.

For example:

If the attacker controls the URL fragment, the JavaScript may insert attacker-controlled HTML into the page.

Comparison:

Type	Main location
Reflected	Server response
Stored	Persistent application storage
DOM-based	Client-side JavaScript and DOM

How I identify them:

- Reflected: Inspect immediate HTTP responses.
- Stored: Submit test data and check whether it executes on subsequent visits.
- DOM-based: Inspect JavaScript sources and sinks, then test controlled inputs.

Important follow-up: Can stored XSS affect administrators?

Yes. If an administrator views a page containing stored malicious content, the script may execute in the administrator's browser context. This may increase the impact depending on the functionality available to that administrator.

28. What is the difference between XSS and HTML injection?

Interview-ready answer:

XSS and HTML injection are related injection vulnerabilities, but their security impact can differ.

HTML injection occurs when an application inserts attacker-controlled HTML markup into a webpage without appropriate handling.

XSS occurs when attacker-controlled content causes script execution in the victim's browser.

Example HTML injection:

If an application displays this markup as actual HTML, an attacker may manipulate the page's appearance.

Example XSS:

If this executes JavaScript, it demonstrates XSS.

Key differences:

HTML Injection	XSS
Injects HTML markup	Causes script execution
May modify page appearance	Can execute malicious browser-side code
May have limited impact	Can have broader security impact
Not always JavaScript execution	Script execution is central to the vulnerability

How I test:

I first determine whether the input is rendered as HTML or encoded as text.

Then I assess whether it can reach an executable JavaScript context.

Remediation:

Use context-specific output encoding and safe HTML sanitization where rich HTML input is required.

29. How do you identify and exploit XSS?

Interview-ready answer:

I identify XSS by locating user-controlled inputs that are reflected, stored or processed by client-side JavaScript and determining whether they reach an executable browser context.

Step 1: Identify input points

I examine:

- Search parameters.
- URL parameters.
- Form fields.
- Comments.
- Profile information.
- File names.
- HTTP headers.
- API responses.
- URL fragments.

Step 2: Establish baseline behavior

I submit a harmless unique string:

I search for it in the HTTP response and browser DOM.

Step 3: Identify the context

I determine whether the input appears inside:

- HTML text.
- HTML attributes.
- JavaScript.
- CSS.
- URL attributes.
- DOM manipulation functions.

Step 4: Test execution

I use a controlled proof-of-concept payload appropriate to the identified context.

For example:

If script tags are encoded or blocked, I investigate the actual parsing context rather than repeatedly trying random payloads.

Step 5: Determine the XSS category

I establish whether the issue is reflected, stored or DOM-based.

Step 6: Assess impact

I examine whether the vulnerability can execute actions, expose sensitive page data or affect privileged users.

Step 7: Document and recommend remediation

I capture the input, request, response and browser behavior.

Tools:

- Burp Suite.
- Browser Developer Tools.
- DOM Invader.
- Browser console.
- JavaScript analysis tools.

Important interview point:

I do not report XSS solely because a payload appears in the response. I verify whether it is interpreted in an executable context.

30. What is Content Security Policy (CSP)?

Interview-ready answer:

Content Security Policy is a browser security mechanism that allows a website to define which sources of content are trusted and permitted to load or execute.

It is primarily used to reduce the impact of XSS and certain content-injection attacks.

CSP is generally delivered through the HTTP response header:

This policy specifies that resources should default to the same origin.

Important CSP directives:

- `default-src` : Default policy for resource types.
- `script-src` : Allowed JavaScript sources.
- `style-src` : Allowed stylesheet sources.

- `img-src` : Allowed image sources.
- `connect-src` : Allowed connection destinations.
- `frame-src` : Allowed iframe sources.
- `object-src` : Allowed plugin-related content.
- `base-uri` : Restricts base URL manipulation.
- `frame-ancestors` : Controls which origins may embed the page.

Example:

How I test CSP:

1. Inspect response headers.
2. Identify allowed script sources.
3. Check for unsafe directives.
4. Examine nonce and hash implementation.
5. Look for overly permissive wildcard sources.
6. Investigate unsafe inline script execution.
7. Check whether the policy is consistently applied.

Important point:

CSP is defense in depth. It does not replace proper output encoding and secure DOM handling.

31. How can CSP be bypassed?

Interview-ready answer:

CSP bypass occurs when an attacker finds a way to execute malicious content despite the application's Content Security Policy.

This can happen because of an overly permissive policy, unsafe trusted resources or incorrect nonce and hash handling.

Common weaknesses:

1. Wildcard source policies.
2. Unsafe inline script allowances.
3. Unsafe evaluation allowances.
4. Trusting third-party script hosts that expose dangerous functionality.
5. Incorrect nonce implementation.

6. Reusable or predictable nonces.
7. Unsafe JSONP endpoints.
8. Incorrect use of CSP directives.

Example of a weak policy:

This allows scripts from a broad range of sources and provides limited protection.

Another weak example:

This allows inline scripts, reducing CSP's ability to mitigate many XSS scenarios.

How I test:

1. Identify the active CSP.
2. Determine which script sources are trusted.
3. Review JavaScript dependencies and endpoints.
4. Check whether inline scripts are permitted.
5. Examine nonce and hash handling.
6. Investigate whether trusted sources expose script injection opportunities.
7. Validate execution in a controlled test environment.

Remediation:

Use strict CSP policies, unpredictable per-response nonces or hashes, avoid unsafe inline execution, restrict trusted script sources and implement proper output encoding.

32. What is Cross-Site Request Forgery (CSRF)?

Interview-ready answer:

CSRF stands for Cross-Site Request Forgery.

It occurs when an attacker tricks an authenticated user's browser into sending an unwanted request to a vulnerable application.

This commonly affects applications that rely on automatically attached browser cookies for authentication.

Consider a banking application.

A user is logged in and has an active session cookie.

The attacker creates a malicious website that attempts to submit a request to the banking application.

If the browser automatically attaches the authentication cookie and the banking application does not implement adequate CSRF protection, the request may be processed as an authenticated operation.

Example state-changing endpoint:

If the request can be triggered cross-site and the application accepts it without verifying that it originated from a legitimate application interaction, the user may unknowingly change their email address.

Conditions that commonly contribute to CSRF:

- Authenticated browser session.
- Automatically attached credentials.
- State-changing endpoint.
- Missing or ineffective CSRF protection.
- Browser conditions allowing the request.

Impact:

- Unauthorized account changes.
- Password or email changes.
- Financial operations.
- Administrative actions.

Remediation:

- Anti-CSRF tokens.
- SameSite cookies.
- Origin and Referer validation where appropriate.
- Reauthentication for sensitive operations.
- Avoiding state-changing GET requests.

33. How does CSRF work?

Interview-ready answer:

CSRF works by exploiting the fact that a browser may automatically include authentication cookies when sending requests to a website.

Consider this scenario.

1. A victim logs into `bank.example.com`.
2. The browser stores a valid session cookie.
3. The victim visits an attacker-controlled website.
4. That website causes the browser to submit a request to the banking application.
5. The browser may automatically attach the banking application's session cookie, depending on cookie and browser rules.
6. The banking server receives an authenticated request.
7. If the server does not validate CSRF protections, it may execute the operation.

A simplified HTML example is:

A malicious page could automatically submit such a form.

How I test:

I identify sensitive state-changing operations and check whether they require CSRF tokens or other request-origin protections.

I examine:

- CSRF token presence.
- Token validation.
- Token reuse.
- SameSite cookie settings.
- Origin validation.
- Referer validation.
- Content-type restrictions.

Important point:

A request being sent cross-site does not automatically mean CSRF is possible. The browser must send usable authentication credentials, and the server must accept the forged operation.

34. What are the different techniques to prevent CSRF?

Interview-ready answer:

CSRF can be prevented through multiple complementary controls.

1. Synchronizer Token Pattern

The server generates a CSRF token and associates it with the user's session.

The client includes the token in state-changing requests.

Example:

The server validates the token before processing the request.

2. Double Submit Cookie

The application sends a CSRF-related cookie and requires the client to submit a matching value through another request component.

A properly implemented signed or session-bound double-submit mechanism helps prevent an attacker from creating a valid matching pair.

3. SameSite cookies

Configure session cookies with appropriate SameSite behavior.

4. Origin validation

Validate the Origin header against trusted application origins.

5. Referer validation

Where appropriate, check the Referer header as an additional control.

6. Custom request headers

For APIs used by browser-based applications, requiring a custom header can help prevent simple cross-site form-based requests, provided CORS is configured securely.

7. Reauthentication

Require additional verification for highly sensitive operations.

8. Safe HTTP methods

Never use GET for state-changing operations.

My testing approach:

I check whether the protections are enforced server-side and whether tokens are properly bound to the user's session and request.

35. Can CSRF occur if an application uses JSON APIs?

Interview-ready answer:

Yes. Using JSON does not automatically prevent CSRF.

However, the actual risk depends on the application's request handling, browser behavior, CORS policy and authentication mechanism.

Consider an API:

A cross-origin browser request using `application/json` generally triggers a CORS preflight.

If the server does not permit the attacker's origin, the browser will generally not send the actual non-simple request.

However, other situations may still create risk:

- The API accepts form-compatible content types.
- The server incorrectly accepts simple cross-origin requests.
- CORS allows an untrusted origin.
- Cookies are attached in an exploitable context.
- The application has alternate endpoints that accept form submissions.

How I test:

1. Identify state-changing endpoints.
2. Inspect accepted content types.
3. Examine CORS configuration.
4. Check cookie SameSite attributes.
5. Test whether simple cross-origin requests are accepted.
6. Verify whether the server enforces CSRF protections independently.

Important point:

JSON is a data format, not a CSRF defense.

36. What is SQL injection?

Interview-ready answer:

SQL injection is a server-side injection vulnerability that occurs when an application incorporates untrusted user input into SQL queries without appropriate parameterization.

An attacker may manipulate the input to change the intended database query.

Consider a login application.

The backend may construct a SQL query using a username and password.

If the application directly concatenates user input into the query, an attacker may inject SQL syntax and alter the query's behavior.

Common types of SQL injection:

1. Error-based SQL injection.
2. UNION-based SQL injection.
3. Boolean-based blind SQL injection.
4. Time-based blind SQL injection.
5. Second-order SQL injection.

How I test:

1. Identify database-driven input parameters.
2. Establish normal application behavior.
3. Test special characters and SQL syntax.
4. Observe error messages and response differences.
5. Test boolean-based behavior.
6. Test time-based behavior when necessary.
7. Validate the vulnerability using repeatable evidence.
8. Assess impact within the authorized scope.

Tools:

- Burp Suite.
- SQLmap.

- Database-specific test techniques.

Impact:

Depending on database permissions and application design, SQL injection may allow unauthorized data access, modification, deletion or other database operations.

Remediation:

Use parameterized queries, prepared statements, safe ORM practices and least-privilege database accounts.

37. Explain error-based, UNION-based, blind and time-based SQL injection.

Interview-ready answer:

These are different SQL injection techniques used to identify or extract information through different application behaviors.

1. Error-based SQL injection

This technique uses database error messages to obtain information about the underlying SQL query or database.

For example, malformed SQL input may cause the application to return a database syntax error.

The error may reveal:

- Database technology.
- Query structure.
- Table or column names.
- Internal implementation details.

2. UNION-based SQL injection

UNION-based SQL injection uses the SQL UNION operator to combine the results of an injected query with the application's original query.

It may allow data from another table to appear in the application's response if the query structure and database permissions permit it.

3. Boolean-based blind SQL injection

In this technique, the attacker sends conditions that evaluate to true or false.

The attacker observes differences in the application's response.

For example, the page may return one result for a true condition and a different result for a false condition.

No database error or direct query output is necessarily exposed.

4. Time-based blind SQL injection

This technique uses conditional database delays.

The tester observes whether the application's response time changes depending on whether a condition is true.

A consistent delay under controlled conditions may indicate a time-based injection point.

Comparison:

Type	Main indicator
Error-based	Database errors
UNION-based	Combined query results
Boolean-based	Different application responses
Time-based	Conditional response delays

Security perspective:

I choose testing techniques based on the application's response behavior and database technology.

I also repeat timing tests to distinguish genuine database delays from network latency.

38. What is the difference between SQL injection and NoSQL injection?

Interview-ready answer:

SQL injection affects applications that construct SQL queries unsafely.

NoSQL injection affects applications that construct or process NoSQL database queries unsafely.

SQL injection example:

An application uses a relational database such as MySQL, PostgreSQL or Microsoft SQL Server.

If user input is concatenated directly into SQL statements, an attacker may alter the query.

NoSQL injection example:

An application uses MongoDB.

If user-supplied JSON objects are directly incorporated into database queries, attackers may inject query operators and alter query behavior.

Comparison:

SQL injection	NoSQL injection
Relational databases	NoSQL databases
SQL query manipulation	Database query/operator manipulation
SQL syntax	Database-specific query syntax
Common in unsafe SQL construction	Common in unsafe query-object construction

How I test:

For SQL injection, I examine quotes, database errors, boolean conditions and time-based behavior.

For NoSQL injection, I examine unexpected JSON objects, operator handling, type confusion and query behavior.

Remediation:

Use parameterized SQL queries for relational databases and safe query construction, strict schema validation and type enforcement for NoSQL databases.

39. How do you exploit SQL injection using Burp Suite?

Interview-ready answer:

I use Burp Suite to intercept and modify requests containing parameters that may interact with the backend database.

Suppose the application has:

Step 1: Capture the request

I browse the application and intercept the request through Burp Proxy.

Step 2: Send it to Repeater

I send the request to Burp Repeater to perform controlled modifications.

Step 3: Establish a baseline

I record the normal response, including status code, response length and content.

Step 4: Test input handling

I modify the parameter using special characters and controlled SQL test inputs.

Step 5: Observe response differences

I examine:

- Database error messages.
- Response content changes.
- Different result counts.
- Boolean behavior.
- Timing differences.

Step 6: Determine the injection type

I investigate whether the behavior indicates error-based, UNION-based, boolean-based or time-based SQL injection.

Step 7: Validate safely

I confirm the vulnerability using the minimum necessary proof of impact.

Step 8: Document evidence

I save the original request, modified request, response and reproducible steps.

Remediation:

Use parameterized queries and avoid constructing SQL statements through string concatenation.

40. What is command injection?

Interview-ready answer:

Command injection is a vulnerability that occurs when an application passes untrusted input to an operating system command without properly separating user data from command syntax.

Consider a web application that offers a network diagnostic feature.

The application accepts a hostname:

The backend may use an operating system utility to perform a diagnostic operation.

If the application constructs a shell command directly using the supplied hostname, an attacker may inject additional command syntax.

Common causes:

- Unsafe shell command construction.
- String concatenation.
- Insufficient input validation.
- Use of dangerous system functions.
- Excessive application privileges.

How I test:

1. Identify parameters potentially passed to system commands.
2. Understand normal application behavior.
3. Use controlled, harmless test inputs.
4. Observe command interpretation.
5. Compare response and timing behavior.
6. Use approved out-of-band validation if necessary.
7. Avoid destructive operations.

Impact:

- Unauthorized command execution.

- File disclosure.
- Internal network access.
- Application compromise.
- Potential server takeover.

Remediation:

Avoid shell invocation wherever possible. Use safe library functions, argument arrays, strict allowlisting and least-privilege execution.

41. What is Local File Inclusion (LFI)?

Interview-ready answer:

Local File Inclusion is a vulnerability that occurs when an application uses user-controlled input to include or retrieve local files without proper validation.

It commonly occurs in applications that dynamically load files based on request parameters.

Example:

The application may use the supplied filename to determine which file to include.

If the application does not properly restrict the path, an attacker may attempt directory traversal or access unintended local resources.

How I test:

1. Identify parameters controlling file paths.
2. Establish normal file-loading behavior.
3. Test path normalization and traversal handling.
4. Examine whether access is restricted to the intended directory.
5. Check error messages.
6. Determine whether file contents can be disclosed or included.

Potential impact:

- Local file disclosure.
- Exposure of application configuration.
- Exposure of credentials.
- Potential code execution under certain conditions.

Remediation:

Use allowlisted file identifiers rather than arbitrary paths, canonicalize file paths and enforce strict directory boundaries.

42. What is Remote File Inclusion (RFI)?

Interview-ready answer:

Remote File Inclusion occurs when an application includes or executes a file from an external location controlled or influenced by an attacker.

It is commonly associated with insecure dynamic file inclusion in certain server-side application environments.

For example, an application may accept a file parameter and use it to load a remote resource.

If remote inclusion is enabled and the input is not restricted, an attacker may attempt to make the server include malicious remote content.

How I test:

1. Identify parameters that influence file inclusion.
2. Understand the application's file-loading mechanism.
3. Determine whether external URLs are accepted.
4. Use a controlled external test resource.
5. Observe whether the application retrieves or processes the resource.
6. Verify whether the behavior leads to actual execution.

Impact:

Potential remote code execution, information disclosure or server compromise, depending on the environment.

Remediation:

Disable unnecessary remote inclusion features, use fixed allowlisted resources and avoid dynamically including user-controlled paths.

Difference between LFI and RFI:

LFI involves local server resources, whereas RFI involves resources located remotely.

43. What is Directory Traversal?

Interview-ready answer:

Directory Traversal, also known as Path Traversal, occurs when an attacker manipulates a file path supplied to an application to access files outside the intended directory.

A common traversal sequence is:

It represents moving one directory level upward in a path.

Consider an application:

If the application constructs a filesystem path using the supplied filename without proper validation, an attacker may attempt to access files outside the permitted directory.

How I test:

1. Identify file-related parameters.
2. Determine the expected directory structure.
3. Test traversal sequences.
4. Examine URL encoding and path normalization.
5. Check whether absolute paths are accepted.
6. Verify whether files outside the intended directory can be accessed.

Potential impact:

- Configuration disclosure.
- Credential exposure.
- Source-code disclosure.
- Access to sensitive system files.

Remediation:

Use strict allowlisting, canonical path resolution and directory-boundary validation. Avoid using raw user input as a filesystem path.

44. What is Server-Side Request Forgery (SSRF)?

Interview-ready answer:

SSRF occurs when an application can be manipulated into making server-side HTTP or other network requests to destinations influenced by an attacker.

For example, an application may provide an image-fetching feature:

The backend fetches the specified URL.

If the application does not restrict destinations, an attacker may attempt to make the server request internal services.

Types of SSRF:

- Basic SSRF.
- Blind SSRF.
- SSRF involving redirects.
- SSRF involving DNS resolution behavior.

How I test:

1. Identify URL parameters.
2. Establish normal server-side fetching.
3. Use an authorized external callback server.
4. Examine redirects and destination validation.
5. Assess access restrictions for internal and special-purpose addresses.
6. Check DNS resolution behavior.
7. Determine whether response content is returned.

Potential impact:

- Internal service access.
- Cloud metadata exposure.
- Internal network reconnaissance.
- Sensitive information disclosure.

Remediation:

Use strict destination allowlists, validate resolved IP addresses, revalidate redirects and apply outbound network restrictions.

45. What is Server-Side Template Injection (SSTI)?

Interview-ready answer:

SSTI occurs when untrusted user input is inserted into a server-side template and the template engine evaluates it as template code rather than treating it as ordinary data.

Server-side template engines are used to generate dynamic HTML or other content.

Examples include:

- Jinja2.
- Twig.
- FreeMarker.
- Velocity.
- Thymeleaf.
- Handlebars, depending on execution context.

Suppose an application accepts a user-provided name and inserts it directly into a server-side template.

If the template engine evaluates attacker-controlled expressions, the application may be vulnerable.

How I test:

1. Identify input fields reflected in server-generated pages.
2. Understand the technology stack.
3. Submit harmless template expressions.
4. Observe whether expressions are evaluated.
5. Identify the template engine where possible.
6. Assess whether the vulnerability allows access to sensitive objects or server-side functionality.
7. Demonstrate impact safely.

Potential impact:

- Information disclosure.
- Access to server-side objects.
- File access.
- Potential remote code execution in vulnerable configurations.

Remediation:

Separate data from template logic, use safe template APIs and sandboxing where appropriate, and never evaluate untrusted template content.

46. What is XML External Entity (XXE) injection?

Interview-ready answer:

XXE is an XML parser vulnerability that occurs when external entity processing is enabled for attacker-controlled XML.

XML supports document type definitions and entity declarations.

A vulnerable parser may resolve external entities that refer to local files or remote resources.

Example:

If the XML parser resolves the entity and the application returns its contents, local information disclosure may occur.

How I test:

- Identify XML input points.
- Check DTD processing.
- Test external entity resolution.
- Examine local file access.
- Test blind XXE using controlled external callbacks.
- Assess SSRF behavior.

Impact:

- Local file disclosure.
- SSRF.
- Internal network access.
- Denial of service.

Remediation:

Disable DTD processing and external entity resolution where not required and use secure XML parser configurations.

47. What is Open Redirect?

Interview-ready answer:

Open Redirect occurs when an application redirects users to a destination controlled by an attacker without properly validating the destination.

Consider:

If the application accepts arbitrary destinations, an attacker may supply an external malicious URL.

Example:

A victim may trust the original domain and follow the link, but the application redirects them to an attacker-controlled website.

How I test:

1. Identify redirect parameters.
2. Examine login and logout redirection.
3. Modify the destination.
4. Test URL normalization and parsing.
5. Check subdomain and protocol handling.
6. Verify whether external redirects are permitted.

Impact:

- Phishing.
- OAuth flow abuse.
- Authentication-related attacks.
- Abuse of trusted-domain reputation.

Remediation:

Use allowlisted redirect destinations or server-side destination identifiers rather than accepting arbitrary URLs.

48. What is Clickjacking, and how can it be prevented?

Interview-ready answer:

Clickjacking is a UI redressing attack in which an attacker tricks a user into clicking on a hidden or disguised element of another website.

The attacker may embed a legitimate application inside an iframe and overlay it with deceptive content.

For example, a user believes they are clicking a harmless button, but the actual click occurs on a sensitive action inside the embedded application.

How I test:

1. Inspect the application's framing restrictions.
2. Check X-Frame-Options.
3. Examine Content-Security-Policy frame-ancestors.
4. Test whether the application can be embedded by an unauthorized origin.
5. Assess sensitive operations that may be triggered through the embedded page.

Important security headers:

Or:

Remediation:

Use CSP `frame-ancestors` with an appropriate allowlist or deny policy. X-Frame-Options can provide compatibility with older browsers.

49. What is security misconfiguration?

Interview-ready answer:

Security misconfiguration occurs when application, server, framework, cloud or infrastructure security settings are missing, incorrectly configured or unnecessarily exposed.

It is a broad vulnerability category because even secure software can become vulnerable through poor configuration.

Common examples:

- Debug mode enabled in production.
- Default credentials.
- Directory listing enabled.

- Exposed administrative interfaces.
- Missing security headers.
- Verbose error messages.
- Unnecessary HTTP methods.
- Insecure CORS configuration.
- Unrestricted file permissions.
- Exposed configuration files.
- Unnecessary services.

How I test:

1. Identify application technologies.
2. Inspect HTTP headers.
3. Examine exposed directories and files.
4. Check debug and administrative interfaces.
5. Review default configurations.
6. Test TLS and CORS.
7. Examine error handling.
8. Compare configurations with secure deployment practices.

Remediation:

Use secure configuration baselines, remove unnecessary components, disable debug functionality in production, patch systems and periodically review configuration.

50. What is sensitive information disclosure?

Interview-ready answer:

Sensitive information disclosure occurs when an application exposes confidential information to unauthorized users or through insecure storage and transmission mechanisms.

Examples include:

- Passwords and password hashes.
- Authentication tokens.
- Personal information.
- Financial information.
- API keys.
- Database credentials.

- Internal configuration.
- Source code.
- Session identifiers.

How I test:

1. Inspect HTTP responses.
2. Review cookies and headers.
3. Examine JavaScript files.
4. Inspect backup and configuration files.
5. Test unauthenticated and low-privileged access.
6. Review browser storage.
7. Examine error messages.
8. Check caching of sensitive responses.

Impact:

The impact depends on the sensitivity of the information and how it can be abused.

For example, exposed session tokens may lead to account compromise, while exposed internal configuration may assist further attacks.

Remediation:

Apply data minimization, appropriate access controls, secure storage, encryption and careful logging practices.

Level 3: Advanced Web Application Security

51. Explain authentication bypass techniques in web applications.

Interview-ready answer:

Authentication bypass occurs when an attacker can access protected functionality without successfully completing the application's intended authentication process.

It can happen because of weaknesses in authentication logic, session handling, token validation or backend access controls.

Common authentication bypass scenarios:

1. Missing authentication checks

An application protects the login page but fails to protect certain backend endpoints.

2. Parameter manipulation

The application trusts a client-controlled parameter to determine whether a user is authenticated.

3. JWT validation weaknesses

The application accepts invalid signatures, weak signing secrets or improperly validated tokens.

4. Session management weaknesses

Predictable session identifiers, session fixation or improper session invalidation may allow unauthorized access.

5. Password reset weaknesses

An attacker may abuse predictable reset tokens, weak OTP validation or incorrect account binding.

6. Alternate endpoint access

An application may protect one route but leave another route exposing the same functionality without authentication.

7. Authentication logic flaws

Incorrect implementation of login, MFA or account recovery workflows may allow authentication to be bypassed.

How I test:

1. Identify protected endpoints.
2. Capture authenticated requests.
3. Remove authentication credentials.
4. Submit invalid and expired credentials.
5. Test alternative routes and methods.
6. Examine session and token behavior.
7. Test password recovery and MFA.

8. Verify whether unauthorized access is actually possible.

Impact:

Unauthorized access to user accounts, sensitive information and privileged functionality.

Remediation:

Enforce authentication centrally, validate all credentials server-side, secure session handling and apply authorization independently to sensitive operations.

52. How do you test for weak password policies?

Interview-ready answer:

I assess whether the application enforces appropriate password requirements during registration, password changes and account recovery.

A weak password policy may allow users to create easily guessable passwords, increasing the risk of account compromise.

What I test:

1. Minimum password length.
2. Maximum password length.
3. Common and breached passwords.
4. Password complexity requirements where appropriate.
5. Password reuse restrictions.
6. Password change functionality.
7. Password reset behavior.
8. Brute-force protection.
9. Password storage practices, where source-code access is available.
10. MFA availability.

Examples of weak passwords:

I verify whether the application blocks commonly used or compromised passwords.

Important technical point:

Modern password policy guidance emphasizes sufficient length, compromised-password screening and secure password storage rather than relying exclusively on complicated composition rules.

Passwords should be stored using a suitable password hashing algorithm such as Argon2id, bcrypt or PBKDF2 with appropriate parameters and salts.

Remediation:

Implement secure password policies, breached-password checks, MFA, rate limiting and secure password hashing.

53. What is session fixation?

Interview-ready answer:

Session fixation is a vulnerability in which an attacker causes a victim to use a session identifier known to the attacker.

If the application does not generate a new session identifier after successful authentication, the attacker may be able to reuse the known identifier to access the victim's authenticated session.

Example scenario:

1. An attacker obtains a session identifier.
2. The attacker somehow causes a victim to use that identifier.
3. The victim logs into the application.
4. The application continues using the same session identifier.
5. The attacker reuses the known identifier.

If the session is valid and the attacker can present it, they may gain access to the victim's authenticated session.

How I test:

1. Obtain an unauthenticated session identifier.
2. Record its value.
3. Authenticate using the application.
4. Check whether the session identifier changes.
5. Test whether the previous identifier remains valid.
6. Verify whether session state is correctly regenerated.

Remediation:

Regenerate session identifiers after authentication and privilege changes, invalidate old sessions and use secure cookie attributes.

54. What is session hijacking?

Interview-ready answer:

Session hijacking occurs when an attacker obtains or takes control of a legitimate user's active session.

Applications commonly use session identifiers or tokens to maintain authenticated sessions.

If an attacker steals these credentials, they may be able to impersonate the victim.

Common causes:

- XSS leading to token theft.
- Unencrypted HTTP communication.
- Malware or compromised devices.
- Insecure cookie storage.
- Session leakage through logs.
- Predictable session identifiers.
- Improper token handling.

How I test:

1. Inspect session identifier generation.
2. Check whether session cookies use Secure and HttpOnly.
3. Examine session lifetime.
4. Test logout invalidation.
5. Check whether session IDs rotate after authentication.
6. Examine exposure through URLs, logs and browser storage.
7. Assess whether sessions are invalidated after sensitive account changes.

Impact:

Unauthorized account access and potential account takeover.

Remediation:

Use cryptographically secure session IDs, HTTPS, secure cookie attributes, session rotation, expiration and appropriate revocation.

55. How do you test session management vulnerabilities?

Interview-ready answer:

I test the complete session lifecycle, from session creation to expiration and invalidation.

My methodology:

1. Session creation

I check whether the session identifier is unpredictable and generated using secure randomness.

2. Session fixation

I verify whether the identifier changes after login.

3. Cookie attributes

I examine Secure, HttpOnly and SameSite.

4. Session expiration

I check whether idle and absolute session timeouts are enforced.

5. Logout

I verify whether the session becomes invalid after logout.

6. Concurrent sessions

I check how the application handles multiple active sessions.

7. Password changes

I examine whether existing sessions remain valid after password changes or account recovery.

8. Privilege changes

I verify whether session identifiers are rotated when privilege levels change.

9. Session reuse

I check whether old session identifiers can be reused.

10. Session exposure

I examine URLs, browser storage, logs and network traffic.

Impact:

Session fixation, session hijacking and unauthorized access.

Remediation:

Implement secure session generation, renewal, expiration and invalidation mechanisms.

56. What is an insecure password reset mechanism?

Interview-ready answer:

An insecure password reset mechanism occurs when an application allows users to reset passwords without sufficiently verifying their identity or protecting the reset process.

Password reset functionality is particularly sensitive because it can directly lead to account takeover.

Common vulnerabilities:

- Predictable reset tokens.
- Tokens that never expire.
- Token reuse.
- Missing rate limiting.
- Account enumeration.
- Weak OTP verification.
- Insecure reset links.
- Missing account binding.
- Host header manipulation.
- Failure to invalidate relevant sessions.

How I test:

1. Request a password reset.

2. Inspect the reset token.
3. Check its randomness and length.
4. Verify expiration.
5. Test whether the token is single-use.
6. Modify account identifiers.
7. Test OTP attempts.
8. Check error messages for account enumeration.
9. Verify session invalidation after successful reset.

Impact:

Account takeover and unauthorized access to sensitive user data.

Remediation:

Use cryptographically secure single-use tokens, short validity periods, rate limiting, secure account binding and session invalidation.

57. How do you test account takeover vulnerabilities?

Interview-ready answer:

Account takeover occurs when an attacker gains unauthorized control of another user's account.

I examine all mechanisms through which an account can be created, authenticated, recovered, modified or linked to external identity providers.

Testing areas:

1. Login

- Weak authentication.
- Brute-force protection.
- Credential stuffing protection.
- Authentication bypass.

2. Password reset

- Weak reset tokens.
- OTP bypass.
- Token reuse.

- Account binding issues.

3. MFA

- MFA bypass.
- OTP brute force.
- Recovery-code weaknesses.
- Improper fallback mechanisms.

4. Email and mobile changes

- Missing reauthentication.
- Missing verification.
- Unauthorized account changes.

5. Session management

- Session hijacking.
- Session fixation.
- Token reuse.
- Improper logout.

6. OAuth and SSO

- Account-linking weaknesses.
- Incorrect identity validation.
- Misconfigured redirect URIs.

How I validate:

I use authorized test accounts and demonstrate whether a security weakness allows one account to be accessed or controlled by another user.

Impact:

Full account compromise, financial fraud and unauthorized data access.

Remediation:

Strengthen authentication, secure account recovery, implement MFA, verify sensitive account changes and protect sessions.

58. What is privilege escalation in web applications?

Interview-ready answer:

Privilege escalation occurs when a user obtains permissions or capabilities beyond those legitimately assigned to their account.

It is generally divided into horizontal and vertical privilege escalation.

Horizontal privilege escalation:

A user accesses another user's resources at the same privilege level.

Example: Customer A accesses Customer B's invoices.

Vertical privilege escalation:

A user obtains functionality belonging to a higher-privileged role.

Example: A normal customer accesses administrative functionality.

How I test:

1. Identify user roles.
2. Build an access-control matrix.
3. Capture requests from privileged accounts.
4. Replay requests using lower-privileged accounts.
5. Modify object identifiers.
6. Test restricted functions.
7. Check whether user-controlled properties affect permissions.

Impact:

Unauthorized data access, administrative control and potentially complete application compromise.

Remediation:

Enforce server-side authorization, least privilege and object-level access checks.

59. Explain horizontal and vertical privilege escalation.

Interview-ready answer:

Horizontal and vertical privilege escalation are two types of authorization failures.

Horizontal privilege escalation occurs when a user accesses another user's resources without gaining additional privileges.

Example:

- User A has customer ID 101.
- User B has customer ID 102.
- User A changes the ID in a request and retrieves User B's information.

This is commonly associated with IDOR or BOLA.

Vertical privilege escalation occurs when a user gains access to functionality associated with a higher privilege level.

Example:

- A customer accesses an administrative endpoint.
- The application allows the customer to modify user roles.
- The customer becomes an administrator.

This is commonly associated with BFLA or improper authorization.

Testing methodology:

I use multiple test accounts and compare the access permissions of each role.

I validate both resource-level access and function-level permissions.

Remediation:

Use role-based or policy-based authorization with proper server-side enforcement.

60. What is an IDOR vulnerability in a web application?

Interview-ready answer:

IDOR stands for Insecure Direct Object Reference.

It occurs when an application uses a direct reference to an internal object but fails to verify whether the requesting user is authorized to access that object.

For example:

The application returns the invoice associated with ID 1001.

If the user changes the ID:

and receives another user's invoice without authorization, the application is vulnerable to IDOR.

How I test:

1. Create two authorized accounts.
2. Identify object identifiers.
3. Capture requests from the first account.
4. Replace identifiers with those belonging to the second account.
5. Check whether unauthorized data is returned.
6. Test update and delete functionality where permitted.
7. Validate the actual impact.

Impact:

Unauthorized access to invoices, personal information, documents and other sensitive resources.

Remediation:

Enforce object-level authorization on the server for every resource operation.

61. What is a race condition vulnerability?

Interview-ready answer:

A race condition occurs when multiple operations execute concurrently and the final result depends on their timing or order.

In web applications, race conditions commonly affect operations involving money, inventory, limited-use offers and account state.

Example:

An e-commerce application has only one item remaining.

Two users submit purchase requests simultaneously.

If the backend checks inventory before updating stock without an atomic operation or appropriate transaction control, both requests may succeed.

This may result in overselling.

How I test:

1. Identify sensitive state-changing functionality.
2. Establish normal behavior.
3. Capture the relevant request.
4. Send controlled concurrent requests.
5. Compare the resulting application state.
6. Verify whether duplicate or inconsistent operations occurred.

Tools:

- Burp Suite Repeater with supported parallel request capabilities.
- Controlled concurrency scripts.
- Application logs where available.

Impact:

Duplicate payments, inventory manipulation, coupon abuse and inconsistent financial records.

Remediation:

Use atomic operations, database transactions, appropriate locking and idempotency controls.

62. How do you identify race conditions using Burp Suite?

Interview-ready answer:

I use Burp Suite to send multiple requests to the same endpoint with carefully controlled timing.

I first identify an operation where concurrent requests could affect shared application state.

Examples include:

- Payment submission.
- Coupon redemption.
- Account balance changes.
- Inventory purchase.

- OTP verification.

Testing process:

1. Capture the original HTTP request.
2. Send it to Repeater.
3. Establish a normal single-request baseline.
4. Prepare multiple identical or appropriately varied requests.
5. Use the available parallel request functionality in Repeater.
6. Send requests concurrently.
7. Observe response codes, response times and application state.
8. Verify whether multiple operations succeeded unexpectedly.

For example, if a coupon is intended to be used once, concurrent redemption requests may reveal whether the application correctly enforces the single-use rule.

Important point:

Multiple successful responses alone do not establish a vulnerability. I verify whether the resulting business state violates the intended rules.

Remediation:

Use database transactions, atomic state changes and server-side idempotency.

63. What is HTTP Request Smuggling?

Interview-ready answer:

HTTP Request Smuggling occurs when front-end and back-end HTTP components interpret request boundaries differently.

For example, an application may use a reverse proxy in front of a backend web server.

If both components interpret Content-Length and Transfer-Encoding differently, an attacker may manipulate how subsequent request data is processed.

Important headers:

Common variants:

- CL.TE.

- TE.CL.
- TE.TE.

How I test:

I identify the proxy and backend architecture and use controlled request-smuggling tests to investigate parsing discrepancies.

I examine:

- Response discrepancies.
- Connection behavior.
- Request queue interference.
- Proxy/backend normalization.
- HTTP protocol translation.

Impact:

- Request queue poisoning.
- Access-control bypass.
- Cache poisoning.
- Response disclosure.
- Interference with other users' requests.

Remediation:

Use consistent HTTP parsing, reject ambiguous framing, normalize requests and patch affected intermediaries.

64. Explain CL.TE, TE.CL and TE.TE request smuggling.

Interview-ready answer:

These terms describe different request-smuggling scenarios based on how the front-end proxy and backend server interpret HTTP message framing.

1. CL.TE

The front-end server uses Content-Length to determine the request body boundary.

The backend server uses Transfer-Encoding.

If both interpret the request differently, request desynchronization may occur.

2. TE.CL

The front-end server uses Transfer-Encoding.

The backend server uses Content-Length.

Again, different interpretations may cause the components to disagree about where one request ends and another begins.

3. TE.TE

Both components support Transfer-Encoding, but one may interpret or process the header differently because of implementation inconsistencies.

For example, one component may ignore a malformed or obfuscated Transfer-Encoding header while another processes it.

Security perspective:

I investigate the actual proxy/backend combination rather than assuming that a specific variant is exploitable.

I use safe testing techniques and avoid disrupting production traffic.

Remediation:

Ensure consistent message framing, reject malformed or ambiguous requests and keep reverse proxies and backend servers patched.

65. What is HTTP Host Header Injection?

Interview-ready answer:

Host Header Injection occurs when an application trusts an attacker-controlled Host or related forwarding header without appropriate validation.

The Host header identifies the host targeted by an HTTP request.

Example:

Some applications use host information to generate absolute URLs, password-reset links or redirects.

If the application trusts arbitrary Host values, an attacker may manipulate those operations.

Example scenario:

A password reset email contains a link generated from the incoming Host header.

If the backend uses an attacker-controlled host value, the generated reset link may point to an attacker-controlled domain.

How I test:

1. Capture requests.
2. Modify the Host header.
3. Test related forwarding headers where applicable.
4. Examine generated redirects.
5. Inspect password reset links in an authorized test environment.
6. Check whether caching or routing behavior changes.

Impact:

- Password reset poisoning.
- Open redirects.
- Cache poisoning.
- Incorrect application routing.

Remediation:

Use a strict allowlist of trusted hosts and trusted proxy configurations. Generate security-sensitive URLs from configured canonical origins rather than untrusted request headers.

66. What is Web Cache Poisoning?

Interview-ready answer:

Web Cache Poisoning occurs when an attacker manipulates a request so that a cache stores a malicious or unintended response and subsequently serves it to other users.

A caching system generally uses selected request attributes to create a cache key.

If the backend response depends on an input that is not included in the cache key, an attacker may be able to poison the cached response.

Example:

A website generates content based on a request header but does not include that header in its cache key.

An attacker manipulates the header, causing the cache to store an altered response.

Subsequent users may receive that response.

How I test:

1. Identify cacheable endpoints.
2. Examine Cache-Control and Vary headers.
3. Identify request inputs influencing responses.
4. Check which inputs are included in cache keys.
5. Use harmless controlled markers.
6. Verify whether subsequent requests receive the altered response.

Impact:

- Stored malicious content.
- Cross-user response manipulation.
- Information disclosure.
- Potential XSS through poisoned cached content.

Remediation:

Configure correct cache keys, avoid caching personalized sensitive content and ensure all response-affecting inputs are handled appropriately.

67. What is Web Cache Deception?

Interview-ready answer:

Web Cache Deception is a vulnerability in which an attacker tricks a caching system into storing a response that contains sensitive user-specific information.

It often involves inconsistencies between how the cache and backend interpret URL paths.

For example, an authenticated application may expose:

The attacker may attempt to construct a URL with an additional path segment or file-like extension.

If the cache considers the URL a static resource but the backend routes it to the authenticated profile handler, the cache may store the sensitive response.

How I test:

1. Identify authenticated endpoints returning sensitive information.
2. Understand cache behavior.
3. Examine URL normalization and routing.
4. Use controlled test accounts.
5. Check whether the response is cached.
6. Verify whether another request can retrieve the cached response.

Impact:

Sensitive information belonging to one user may become accessible to another user through shared caching infrastructure.

Remediation:

Configure cache behavior consistently with backend routing, prevent caching of sensitive authenticated responses and use appropriate cache-control directives.

68. What is Prototype Pollution?

Interview-ready answer:

Prototype Pollution is a JavaScript vulnerability in which an attacker modifies an object's prototype, potentially affecting the properties and behavior of other objects.

JavaScript objects can inherit properties through their prototype chain.

If an application processes untrusted object properties without appropriate safeguards, attacker-controlled properties may pollute shared prototypes.

Common dangerous property names include:

A vulnerable recursive merge or object-processing function may allow these properties to modify inherited object behavior.

Example impact:

If an application checks an inherited property such as an authorization flag, polluted prototype values may influence application logic.

Depending on the application, Prototype Pollution may lead to:

- Unexpected application behavior.
- Authorization weaknesses.
- Denial of service.
- Client-side XSS.
- Server-side code execution in certain vulnerable Node.js environments and dependency combinations.

How I test:

1. Identify JavaScript object-merging functionality.
2. Examine JSON parsing and recursive merge operations.
3. Test controlled prototype-related properties.
4. Check whether shared objects are affected.
5. Identify exploitable gadgets or unsafe property checks.
6. Validate actual impact.

Remediation:

Use safe object creation and merge functions, reject dangerous property names where appropriate and avoid unsafe prototype mutation.

69. How can Prototype Pollution lead to XSS or privilege escalation?

Interview-ready answer:

Prototype Pollution can affect security when polluted inherited properties influence sensitive application behavior.

XSS scenario:

Suppose a JavaScript application has a function that creates DOM elements based on object properties.

If an attacker can pollute a property used to determine an HTML attribute or script-related value, the application may insert attacker-controlled content into an unsafe DOM context.

If the application uses unsafe sinks such as `innerHTML`, this may result in XSS.

Privilege escalation scenario:

Suppose application code relies on an inherited property to determine whether a user is an administrator.

If prototype pollution changes that property and the application incorrectly trusts it, the application may display or execute privileged functionality.

However, a secure backend should still enforce authorization independently.

How I investigate:

1. Identify the vulnerable object-processing function.
2. Confirm prototype modification.
3. Identify affected application properties.
4. Trace the polluted values into sensitive sinks or authorization logic.
5. Determine whether exploitation is actually possible.
6. Validate the impact in a controlled environment.

Remediation:

Use safe property handling, avoid relying on inherited values for security decisions, use `Object.create(null)` where appropriate and enforce authorization server-side.

70. What is DOM Clobbering?

Interview-ready answer:

DOM Clobbering is a browser-side technique in which attacker-controlled HTML elements create named properties that interfere with JavaScript variables or DOM properties.

Certain HTML elements can expose named properties through browser DOM APIs.

If application JavaScript relies on these properties without validating their types or origins, an attacker may alter application behavior.

For example, an application might access a global property using a name that can also be created by attacker-controlled HTML.

If the application assumes that the property always refers to a trusted JavaScript object, DOM Clobbering may interfere with that assumption.

How I test:

1. Identify JavaScript code relying on global DOM properties.
2. Find attacker-controlled HTML insertion points.
3. Examine named element behavior.
4. Determine whether DOM properties can be shadowed or replaced.
5. Trace the effect into sensitive application logic.
6. Assess whether the behavior leads to XSS or another security impact.

Remediation:

Avoid relying on named global DOM properties, use explicit element references and validate object types and values.

71. What is WebSocket security, and how do you test WebSocket connections?

Interview-ready answer:

WebSocket is a communication protocol that provides a persistent, bidirectional connection between a client and server.

Unlike traditional HTTP request-response communication, WebSocket allows both parties to send messages over an established connection.

It is commonly used in:

- Chat applications.
- Live notifications.
- Trading platforms.
- Collaborative applications.
- Real-time dashboards.

A WebSocket connection usually begins with an HTTP upgrade request.

Example:

If accepted, the server responds with HTTP 101 Switching Protocols.

How I test WebSocket security:

1. Identify WebSocket endpoints.
2. Capture the handshake through Burp Suite.
3. Check authentication requirements.
4. Inspect Origin validation.
5. Examine message authorization.
6. Test unauthorized room or channel access.
7. Test input validation.
8. Check message replay and manipulation.
9. Test connection lifecycle and session expiration.

Common vulnerabilities:

- Cross-Site WebSocket Hijacking.
- Missing authentication.
- Missing message-level authorization.
- Sensitive data exposure.
- Injection through messages.
- Improper Origin validation.

Remediation:

Use WSS, validate Origin where relevant, enforce authentication and authorization for every sensitive message and validate all message inputs.

72. What is WebSocket Cross-Site Hijacking?

Interview-ready answer:

WebSocket Cross-Site Hijacking is an attack in which an attacker-controlled website attempts to establish a WebSocket connection to a vulnerable application using the victim's authenticated browser session.

It is also commonly called Cross-Site WebSocket Hijacking.

The issue can occur when:

- Authentication relies on automatically attached cookies.
- The WebSocket handshake does not properly validate the Origin.
- The server permits unauthorized cross-origin connections.

Example scenario:

1. A victim logs into a chat application.
2. Their browser stores a session cookie.
3. The victim visits an attacker-controlled page.
4. That page attempts to establish a WebSocket connection to the chat application.
5. The browser may attach the relevant cookies.
6. If the server fails to validate the origin and authorization, the attacker-controlled page may interact with the victim's authenticated connection.

How I test:

1. Identify the WebSocket endpoint.
2. Understand its authentication mechanism.
3. Inspect the handshake request.
4. Test Origin validation.
5. Check whether unauthorized cross-origin connections are accepted.
6. Determine whether sensitive messages can be read or actions triggered.

Remediation:

Validate the Origin header against a strict allowlist, implement appropriate authentication and enforce message-level authorization.

73. What is CORS misconfiguration, and how can it lead to sensitive data exposure?

Interview-ready answer:

CORS misconfiguration occurs when an application permits cross-origin access more broadly than intended.

This may allow an untrusted website to read sensitive API responses through a victim's authenticated browser.

Consider a server response:

If the victim's browser sends credentials to the target application and the server permits the attacker's origin, the attacker's JavaScript may be able to read the response, subject to browser cookie and CORS rules.

Common misconfigurations:

- Arbitrary origin reflection.
- Trusting untrusted subdomains.
- Allowing the null origin unnecessarily.
- Credentialed requests with unsafe origin policies.
- Incorrect origin matching.
- Excessively broad trusted-origin lists.

How I test:

1. Identify endpoints returning sensitive data.
2. Send requests with different Origin values.
3. Inspect CORS response headers.
4. Examine credential handling.
5. Test browser behavior using a controlled external origin.
6. Verify whether sensitive responses can actually be read.

Impact:

Unauthorized disclosure of authenticated user data.

Remediation:

Use an explicit origin allowlist, validate origins carefully and enable credentialed cross-origin access only when necessary.

74. How do you identify and exploit SSTI in different template engines?

Interview-ready answer:

I identify Server-Side Template Injection by locating user-controlled input that is processed by a server-side template engine.

The objective is to determine whether the application treats the input as template code rather than ordinary data.

Step 1: Identify injection points

I examine:

- Search results.
- Error pages.
- Email templates.
- Report generation.
- User-generated content.
- Dynamic page rendering.

Step 2: Test harmless template expressions

For example, a mathematical expression such as:

Some template engines evaluate this expression and return:

However, the syntax differs between engines, and the same input may simply be displayed as text.

Step 3: Identify the template engine

I investigate the technology stack and error behavior.

Common engines include:

- Jinja2.
- Twig.
- FreeMarker.
- Velocity.
- Thymeleaf.

Step 4: Assess execution context

I determine whether the input is evaluated in a template context and what capabilities are exposed.

Step 5: Validate impact

I use harmless expressions first and assess access to objects or server functionality only within the authorized scope.

Potential impact:

- Sensitive information disclosure.
- Access to server-side objects.
- File access.
- Remote code execution in vulnerable configurations.

Remediation:

Never dynamically evaluate untrusted template content. Separate data from template logic and apply appropriate sandboxing.

75. How do you test file upload functionality for security vulnerabilities

You said:

Please continue and complete after 74th question of web

Worked for 1m 34s

ChatGPT said:

Web Application Security Interview Questions (75–100)

Continuing directly from **Question 74: Server-Side Template Injection (SSTI)**. These are the final 26 questions in the Web Security section, with technical explanations, practical testing approaches, impact, remediation, and likely interviewer follow-ups.

Advanced Web VAPT and Real-World Scenarios

75. How do you test file upload functionality for security vulnerabilities?

Answer:

File upload testing involves checking whether an application properly validates uploaded files, stores them securely, and prevents uploaded content from being executed or accessed in an unauthorized way.

I test the following areas:

1. File extension validation

- Upload allowed extensions such as `.jpg`, `.png`, or `.pdf`.
- Try disallowed extensions such as `.php`, `.jsp`, `.aspx`, and `.html`.

- Test double extensions, such as `image.jpg.php`.
- Test case variations, such as `.PHP` or `.pHp`.
- Test trailing characters, spaces, and alternate extensions.

2. MIME type validation

Intercept the upload request in Burp Suite and modify the `Content-Type` header.

For example:

```
Content-Disposition: form-data; name="file"; filename="test.php"
Content-Type: image/jpeg
```

Check whether the server trusts the client-supplied MIME type instead of validating the actual file content.

3. File signature validation

Change the file extension and MIME type while keeping the actual file content unchanged. Check whether the server validates file signatures, also called magic bytes.

4. Path traversal

Try filenames containing traversal sequences, such as:

```
../../test.php
```

Check whether the application allows files to be stored outside the intended upload directory.

5. File size and resource exhaustion

Test whether the application enforces file size limits, upload quotas, and appropriate rate limits.

6. Active content and execution

Check whether uploaded HTML, SVG, or other active content can execute JavaScript when opened. Also verify whether uploaded files are served from a separate domain and whether server-side execution is disabled.

Impact:

- Remote code execution, depending on server configuration.
- Stored XSS.
- Arbitrary file overwrite.
- Sensitive information disclosure.
- Denial of service through oversized uploads.

Remediation:

- Use allowlisted extensions.
- Validate actual file content and signatures.
- Generate random server-side filenames.
- Store uploads outside the webroot or on a separate storage domain.
- Disable execution permissions in upload directories.
- Enforce size limits and authorization checks.
- Scan files where appropriate.

Interviewer follow-up: What is the difference between MIME type validation and file signature validation?

MIME type validation checks the declared content type, which can be manipulated by the client. File signature validation examines the actual binary structure or magic bytes of the file. Neither alone is sufficient, so both should be combined with extension allowlisting and safe storage.

76. How do you identify and exploit IDOR vulnerabilities?

Answer:

IDOR stands for Insecure Direct Object Reference. It occurs when an application exposes an object identifier and fails to verify whether the current user is authorized to access that particular object.

Consider an API request:

```
GET /api/user/1001/profile
Authorization: Bearer USER_A_TOKEN
```

If I change the identifier:

```
GET /api/user/1002/profile
Authorization: Bearer USER_A_TOKEN
```

and receive User B's profile, it indicates a possible IDOR vulnerability.

Testing methodology:

1. Create two accounts, User A and User B.
2. Log in using both accounts.
3. Capture requests involving user-specific objects.
4. Identify object references such as user IDs, account IDs, invoice IDs, document IDs, and order IDs.
5. Replay User A's request after replacing the object identifier with User B's identifier.
6. Check whether the application returns, modifies, or deletes User B's information.
7. Test different HTTP methods, including GET, POST, PUT, PATCH, and DELETE.
8. Repeat the test with identifiers in the URL, request body, headers, and GraphQL variables.

Types of IDOR:

- Horizontal privilege escalation: User A accesses another user's resources at the same privilege level.
- Vertical privilege escalation: A lower-privileged user accesses resources belonging to a higher-privileged user.
- IDOR through modification: A user modifies another user's information.
- IDOR through deletion: A user deletes another user's records.

Impact:

Unauthorized access to personal information, financial records, documents, orders, or other sensitive resources.

Remediation:

- Perform server-side authorization checks for every object.
- Verify ownership and tenant boundaries.
- Do not rely on unpredictable IDs as an authorization mechanism.
- Apply authorization checks consistently across all HTTP methods.

Important interview point: Changing a sequential ID is not automatically a vulnerability. The vulnerability exists when the server fails to enforce authorization.

77. How do you test CSRF when an application uses anti-CSRF tokens?

Answer:

CSRF occurs when an attacker tricks an authenticated victim's browser into sending an unwanted request to an application where the victim is already logged in.

When anti-CSRF tokens are implemented, I check whether they are properly generated, validated, and bound to the user's session.

Testing methodology:

1. Capture a state-changing request, such as changing an email address.
2. Identify the anti-CSRF token in the request.
3. Remove the token and replay the request.
4. Modify the token and replay it.
5. Use another user's token.
6. Check whether the token is reusable or session-bound.
7. Check whether the token is validated on all relevant HTTP methods.
8. Test whether the application accepts requests with missing or malformed tokens.
9. Check whether the application relies only on client-side validation.

Example:

```
POST /account/change-email HTTP/1.1
Cookie: session=abc123
Content-Type: application/x-www-form-urlencoded

email=attacker@example.com&csrf_token=xyz123
```

If the application processes the request after the token is removed or replaced, investigate whether CSRF protection is missing or incorrectly implemented.

What if the token is present but CSRF is still possible?

Possible reasons include:

- Token validation is not enforced on every state-changing endpoint.
- Tokens are not bound to the user's session.
- The token is predictable.
- The application has a token leakage issue.
- An attacker can exploit XSS to obtain the token.
- The application has an alternate endpoint that does not validate the token.

Remediation:

Use unpredictable, session-bound anti-CSRF tokens, validate them server-side, and apply appropriate `SameSite` cookie settings. Origin or Referer validation can provide additional protection.

78. How do you test Content Security Policy (CSP), and can CSP be bypassed?

Answer:

Content Security Policy is a browser security mechanism that restricts which sources can load scripts, styles, images, and other resources.

It helps reduce the impact of XSS, but it is not a replacement for proper input handling and output encoding.

Example:

```
Content-Security-Policy:
default-src 'self';
script-src 'self';
object-src 'none';
base-uri 'self';
```

Testing methodology:

1. Inspect the response headers.
2. Identify the `Content-Security-Policy` directive.
3. Review `script-src`, `object-src`, `base-uri`, `connect-src`, and `frame-ancestors`.

4. Check whether unsafe directives such as `'unsafe-inline'` or `'unsafe-eval'` are present.
5. Check whether wildcard sources are allowed.
6. Check whether trusted script hosts allow attacker-controlled content.
7. Review nonce and hash implementation.
8. Test whether the application has any XSS vulnerability that can bypass the existing policy.

Common CSP weaknesses:

- `script-src 'unsafe-inline'`
- `script-src *`
- Overly broad trusted domains.
- Unsafe use of JSONP endpoints.
- Missing `object-src 'none'`.
- Missing or weak nonce implementation.
- Reusable or predictable nonces.
- Weak `base-uri` restrictions.

What is a nonce in CSP?

A nonce is a cryptographically random value generated for a response and included in the CSP header and authorized script element.

Example:

```
Content-Security-Policy: script-src 'nonce-random123'
```

The corresponding script element must contain the matching nonce.

Remediation:

Use a restrictive CSP, avoid unsafe inline scripts, generate unpredictable nonces per response, and minimize trusted script sources.

79. How do you test rate-limiting vulnerabilities, and how can rate limits be bypassed?

Answer:

Rate limiting restricts how frequently a user, IP address, account, or client can perform a particular action.

It is important for login endpoints, password resets, OTP verification, registration, payment operations, and APIs.

Testing methodology:

1. Identify sensitive endpoints.
2. Send repeated requests using Burp Intruder or an appropriate automated testing tool.
3. Observe whether the application returns HTTP `429 Too Many Requests`.
4. Check whether the application introduces delays, CAPTCHA, account lockout, or temporary blocking.
5. Test whether rate limits are applied per IP, account, session, device, or API key.
6. Check whether different HTTP methods or endpoint variations have separate limits.
7. Verify whether limits are consistently enforced across distributed application instances.

Common weaknesses:

- Rate limiting only by IP address.
- No account-level limits.
- Different endpoints have inconsistent limits.
- Limits can be reset by changing sessions.
- OTP verification permits excessive attempts.
- Password reset requests can be abused.

Impact:

- Brute-force attacks.
- Credential stuffing.
- OTP guessing.
- Account enumeration.
- Resource exhaustion.
- Abuse of business functionality.

Remediation:

Implement layered rate limiting using account, IP, session, device, and endpoint context. Use progressive delays, monitoring, anomaly detection, and appropriate lockout or challenge mechanisms.

Interviewer follow-up: Is HTTP 429 mandatory for rate limiting?

No. It is a standard and useful response, but applications may implement other responses or controls. I verify actual enforcement rather than relying only on the response status code.

80. Is storing JWT tokens in localStorage secure?

Answer:

Storing JWTs in browser `localStorage` is generally risky because JavaScript running in the same origin can access them.

If an attacker exploits an XSS vulnerability, malicious JavaScript may steal the token and use it to impersonate the victim.

Example:

```
const token = localStorage.getItem("access_token");
```

Why is this risky?

- JavaScript can access localStorage.
- XSS can expose stored tokens.
- Tokens may remain available after the browser is closed.
- Token theft can lead to account takeover until expiration or revocation.

What about sessionStorage?

SessionStorage is also accessible to JavaScript and therefore vulnerable to token theft through XSS. It generally has a shorter lifetime tied to the browser tab, but it does not eliminate the XSS risk.

What is a safer approach?

For many browser-based applications, a common approach is to store session credentials in cookies configured with:

```
Set-Cookie: session=abc123;
HttpOnly;
Secure;
SameSite=Lax
```

- `HttpOnly` : prevents JavaScript from reading the cookie.
- `Secure` : sends it only over HTTPS.
- `SameSite` : helps reduce CSRF risk.

However, cookie-based authentication still requires appropriate CSRF protection and session management.

Important: There is no storage option that makes an application automatically secure. The choice depends on the architecture and threat model.

81. Explain cookie security attributes and how you test them.

Answer:

Cookies often store session identifiers, authentication state, and other sensitive information. Incorrect cookie configuration can expose users to session theft, CSRF, or session fixation.

Important attributes:

Attribute	Purpose
Secure	Sends cookie only over HTTPS
HttpOnly	Prevents JavaScript access
SameSite	Controls cross-site cookie sending
Domain	Defines which hosts receive the cookie
Path	Restricts the URL path where the cookie is sent
Expires	Defines a fixed expiration time
Max-Age	Defines cookie lifetime

Testing methodology:

1. Log in and capture the `Set-Cookie` response header.
2. Check whether the session cookie has `Secure`.
3. Check whether it has `HttpOnly`.
4. Review the `SameSite` setting.
5. Check whether the cookie has an unnecessarily broad `Domain` attribute.
6. Verify whether session IDs change after authentication.
7. Check session expiration and logout invalidation.
8. Test whether cookies are transmitted over HTTP.

What is session fixation?

Session fixation occurs when an attacker causes a victim to authenticate using a session identifier already known to the attacker.

A secure application should regenerate the session identifier after login and other privilege-changing events.

Remediation:

Use HTTPS, secure cookie attributes, strong session identifiers, session rotation, and server-side session invalidation.

82. How do you test for web shell upload vulnerabilities?

Answer:

A web shell is a server-side script that allows an attacker to interact with the server through a web-accessible interface.

A vulnerable file upload feature can potentially allow an attacker to upload a server-executable file and execute commands remotely.

Testing methodology:

1. Identify upload functionality.
2. Review extension, MIME type, and signature validation.
3. Check where uploaded files are stored.
4. Determine whether the upload directory permits server-side execution.

5. Check whether the uploaded file is accessible through a public URL.
6. Verify whether the server interprets uploaded files as executable content.
7. Use only an authorized, harmless proof of concept to establish whether execution is possible.

Example of a risky configuration:

An application accepts `.php` files and stores them in a publicly accessible directory where PHP execution is enabled.

This may result in remote code execution.

Impact:

- Remote command execution.
- Server compromise.
- Data theft.
- Lateral movement.
- Persistence.

Remediation:

Store uploads outside the webroot, use random filenames, enforce strict allowlists, validate file content, and disable execution in upload directories.

Interview distinction: Arbitrary file upload and remote code execution are not automatically the same finding. You need to demonstrate whether the uploaded content can actually execute or otherwise cause security impact.

83. How do you test Open Redirect vulnerabilities?

Answer:

An Open Redirect vulnerability occurs when an application redirects users to an attacker-controlled destination without properly validating the target.

Example:

```
GET /redirect?url=https://trusted.example
```

I would test whether changing the URL parameter redirects the browser to an external domain.

Example test:

```
GET /redirect?url=https://attacker.example
```

Testing methodology:

- Identify parameters such as `url`, `redirect`, `returnUrl`, `next`, and `continue`.
- Replace the destination with an external domain.
- Test absolute URLs, relative URLs, and protocol-relative URLs.
- Test URL encoding and double encoding.
- Test alternate hostname representations.
- Review redirect behavior after login, logout, and OAuth authentication.

Impact:

- Phishing.
- Credential theft through convincing trusted-domain links.
- OAuth-related attacks in poorly configured flows.
- Abuse of trusted application URLs.

Remediation:

Use server-side allowlists of permitted redirect destinations. Prefer fixed internal destinations or validated relative paths instead of accepting arbitrary URLs.

Interviewer follow-up: Is Open Redirect always high severity?

No. Severity depends on exploitability and impact. An isolated redirect may be low severity, while one that enables OAuth token leakage or a convincing authentication attack may be more serious.

84. How do you test Local File Inclusion (LFI)?

Answer:

Local File Inclusion occurs when an application uses user-controlled input to include or read files from the local server without proper validation.

Example:

```
GET /view?file=report.php
```

I would check whether modifying the parameter allows access to unintended files.

Example traversal test:

```
../../../../etc/passwd
```

Testing methodology:

1. Identify file-related parameters.
2. Test directory traversal sequences.
3. Try URL encoding and double encoding.
4. Test absolute and relative paths.
5. Check whether application errors reveal filesystem paths.
6. Verify whether sensitive files can be accessed.
7. Check whether file inclusion can lead to code execution in the application's environment.

Common impact:

- Disclosure of configuration files.
- Exposure of application source code.
- Leakage of credentials.
- Potential code execution under certain conditions.

Remediation:

Avoid passing user-controlled values directly to file inclusion functions. Use strict allowlists, canonicalize paths, and enforce access restrictions.

Difference between LFI and Path Traversal:

Path traversal allows access outside an intended directory. LFI occurs when an application includes a local file based on user-controlled input. They can overlap, but they are not identical.

85. Explain Server-Side Request Forgery (SSRF) and how you test it.

Answer:

SSRF occurs when an attacker manipulates a server-side component into making requests to unintended destinations.

For example, an application might fetch a URL supplied by a user:

```
POST /api/fetch-image
Content-Type: application/json

{
  "url": "https://example.com/image.png"
}
```

If the server does not validate the destination, an attacker may attempt to make it request internal resources.

Testing methodology:

1. Identify URL-fetching functionality.
2. Look for parameters such as `url`, `callback`, `webhook`, `image`, and `endpoint`.
3. Submit a URL pointing to an authorized external test server.
4. Monitor whether the application makes a server-side request.
5. Test whether internal IP ranges or localhost are reachable, only within the authorized scope.
6. Check redirect handling and DNS resolution behavior.
7. Review whether non-HTTP protocols are accepted.
8. Assess whether responses are returned to the attacker or whether the behavior is blind SSRF.

Types:

- Basic SSRF: The response from the internal or external destination is returned.
- Blind SSRF: The request is triggered, but the response is not directly visible.
- Second-order SSRF: A supplied URL is stored and fetched later by another process.

Potential impact:

- Access to internal services.
- Cloud metadata exposure.
- Internal network reconnaissance.

- Access to sensitive endpoints.

Remediation:

- Allowlist permitted destinations.
- Block private, loopback, link-local, and reserved address ranges.
- Validate DNS resolution and redirects.
- Restrict outbound network access.
- Use network segmentation.
- Avoid exposing sensitive internal services.

86. Explain SQL Injection and its major types.

Answer:

SQL Injection occurs when an application constructs database queries using untrusted user input without proper parameterization.

For example, vulnerable code might construct a query like:

```
SELECT * FROM users
WHERE username = 'USER_INPUT';
```

If input is concatenated directly into the query, an attacker may manipulate its logic.

Major types:

1. In-band SQL Injection

The attacker uses the same communication channel to submit the payload and receive results.

- Error-based SQLi.
- Union-based SQLi.

2. Blind SQL Injection

The application does not directly return database results.

- Boolean-based blind SQLi.
- Time-based blind SQLi.

3. Out-of-band SQL Injection

The database communicates through a separate channel, such as a DNS or HTTP interaction, where supported.

Testing methodology:

1. Identify database-backed input points.
2. Test for unexpected SQL errors.
3. Use controlled input variations to observe response differences.
4. Test Boolean-based conditions.
5. Test time-based behavior where appropriate.
6. Use Burp Suite and SQLmap to support validation.
7. Confirm the finding with a safe proof of concept.

Remediation:

- Use prepared statements and parameterized queries.
- Avoid dynamic query concatenation.
- Apply least-privileged database accounts.
- Validate input according to business requirements.
- Avoid exposing detailed database errors.

Interviewer follow-up: Does input validation prevent SQL Injection?

Input validation helps, but it is not the primary defense. Parameterized queries ensure that user input is treated as data rather than executable SQL syntax.

87. How do you test whether a Web Application Firewall (WAF) can be bypassed?

Answer:

A WAF monitors HTTP traffic and attempts to block malicious requests based on configured rules and detection logic.

During an authorized assessment, I first determine whether a WAF is present and then assess whether its rules consistently protect the application.

Testing methodology:

1. Identify possible WAF indicators in HTTP headers and responses.
2. Compare normal and suspicious request behavior.
3. Test whether different input locations receive consistent protection.
4. Review encoding and normalization behavior.
5. Check whether the WAF handles alternate HTTP methods consistently.
6. Assess whether the application remains vulnerable despite the WAF.
7. Validate the underlying issue directly rather than treating a WAF response as proof of security.

Common weaknesses:

- Inconsistent request normalization.
- Different parsing behavior between WAF and backend.
- Incomplete endpoint coverage.
- Overly broad exclusions.
- Misconfigured rules.
- Unprotected alternate application paths.

Important interview point:

A WAF is a defense-in-depth control. It does not replace secure coding, authorization checks, input handling, or patching.

Remediation:

Keep WAF rules updated, configure consistent request normalization, monitor false negatives, and fix vulnerabilities in the application itself.

88. What is HTTP request smuggling, and how do you test it?

Answer:

HTTP request smuggling occurs when two HTTP components, typically a frontend proxy and a backend server, interpret request boundaries differently.

This can happen when they disagree about how to interpret headers such as:

- `Content-Length`
- `Transfer-Encoding`

Common types:

- CL.TE: Frontend uses Content-Length, backend uses Transfer-Encoding.
- TE.CL: Frontend uses Transfer-Encoding, backend uses Content-Length.
- TE.TE: Both interpret Transfer-Encoding differently due to parsing inconsistencies.

Why is it dangerous?

An attacker may cause one component to interpret part of a request differently from another, potentially leading to:

- Request desynchronization.
- Unauthorized request routing.
- Cache poisoning.
- Access-control bypass.
- Exposure of other users' requests.

Testing methodology:

1. Understand the architecture and identify proxies or load balancers.
2. Use a specialized tool such as Burp Suite's HTTP Request Smuggler extension.
3. Test only within an authorized environment.
4. Examine frontend/backend parsing differences.
5. Confirm behavior using controlled, non-destructive requests.
6. Avoid tests that could affect other users or shared infrastructure.

Remediation:

Use consistent HTTP parsing, reject ambiguous requests, normalize headers, and keep frontend and backend components updated.

89. How do you test business logic vulnerabilities?

Answer:

Business logic vulnerabilities occur when an application correctly processes technically valid requests but fails to enforce the intended business rules.

These issues are often missed by automated scanners because the requests may be syntactically valid.

Examples:

- Applying a discount multiple times.
- Purchasing an item with a negative quantity.
- Modifying a price parameter.
- Reusing a one-time coupon.
- Skipping mandatory workflow steps.
- Approving one's own request.
- Bypassing transaction limits.
- Reusing an expired or completed transaction.

Testing methodology:

1. Understand the application's normal workflow.
2. Identify business rules and assumptions.
3. Capture requests at every important stage.
4. Modify quantities, prices, IDs, status values, and workflow parameters.
5. Repeat requests to identify replay issues.
6. Test whether steps can be skipped or performed out of order.
7. Use multiple accounts to test ownership and role boundaries.
8. Check whether backend validation enforces the actual business rules.

Example:

An e-commerce application calculates a product price on the client side and accepts the submitted price without independently verifying it.

An attacker could modify the price before checkout.

Remediation:

Enforce business rules server-side, calculate prices using trusted backend data, validate transaction state, and use idempotency controls where appropriate.

Interviewer follow-up: Why are business logic vulnerabilities difficult to automate?

Because the tester needs to understand the application's intended behavior, user roles, transaction states, and business rules. A scanner may identify technical anomalies but usually cannot determine whether a valid-looking transaction violates the business process.

90. How do you test payment gateway vulnerabilities?

Answer:

Payment testing focuses on whether the application correctly validates transaction details and verifies payment status with the trusted payment provider.

Testing scenarios:

1. Modify the payment amount.
2. Modify the order ID.
3. Attempt to pay for another user's order.
4. Replay a payment request.
5. Test whether the same transaction can be completed multiple times.
6. Modify currency or product identifiers.
7. Check whether the application trusts a client-side success response.
8. Check whether an order is marked as paid without verified confirmation from the payment gateway.
9. Test payment callback or webhook authentication.
10. Check whether failed, cancelled, and pending transactions are handled correctly.

Example:

A user initiates a payment for ₹5,000. The application must not trust a client-supplied amount or a browser redirect indicating success. It should verify the transaction with the payment provider.

Important security controls:

- Server-side amount verification.
- Unique transaction identifiers.
- Authenticated payment callbacks.
- Signature verification.
- Idempotency protection.
- Transaction-state validation.
- Proper authorization checks.

Impact:

Financial loss, unauthorized purchases, duplicate transactions, and incorrect order fulfillment.

Interviewer follow-up: What is an idempotency key?

It is a unique identifier associated with an operation. It helps ensure that retrying the same request does not unintentionally execute the transaction multiple times.

91. How do you test privilege escalation in a web application?

Answer:

Privilege escalation occurs when a user gains access to permissions or functionality beyond what their assigned role should allow.

Types:

- Horizontal privilege escalation: Accessing another user's resources at the same privilege level.
- Vertical privilege escalation: Gaining access to a higher privilege level, such as an administrator.

Testing methodology:

1. Create accounts with different roles, such as normal user, manager, and administrator.
2. Capture requests made by each role.
3. Identify administrative endpoints and sensitive operations.
4. Replay privileged requests using a lower-privileged account.
5. Modify role, permission, and user identifiers in requests.
6. Test direct access to restricted URLs.
7. Test HTTP methods and alternate API endpoints.
8. Verify whether the backend enforces authorization.

Example:

A normal user sends:

```
POST /api/users/1001/role
Content-Type: application/json

{
  "role": "admin"
}
```

If the server accepts this request without verifying the user's authorization, it may result in vertical privilege escalation.

Remediation:

Implement server-side role-based or attribute-based access control, deny access by default, and enforce authorization on every sensitive operation.

92. How do you approach security testing of microservices?

Answer:

Microservices architecture divides an application into multiple independently deployable services that communicate through APIs, message queues, or other internal mechanisms.

The increased number of service-to-service interactions creates additional security considerations.

Key testing areas:

1. Authentication between services.
2. Authorization at each service.
3. API gateway configuration.
4. Internal service exposure.
5. Service-to-service communication encryption.
6. Token validation and trust boundaries.
7. Message queue security.
8. Secrets management.
9. Rate limiting and resource controls.
10. Logging and monitoring.

Example scenario:

A public API gateway authenticates a user and forwards the request to an internal service. If the internal service trusts every request from the gateway without validating the required identity or authorization context, an attacker who can reach that service directly may bypass the gateway's controls.

Testing methodology:

- Map the services and communication flows.
- Identify public and internal endpoints.
- Test direct access to internal services within scope.
- Review JWT validation between services.
- Check for missing authorization at individual services.
- Assess message queue permissions and data exposure.
- Review secrets and configuration handling.

Remediation:

Use strong service authentication, mutual TLS where appropriate, least-privilege service permissions, centralized identity management, network segmentation, and independent authorization enforcement.

93. How do you perform GraphQL security testing?

Answer:

GraphQL is an API query language that allows clients to request specific data through a flexible query structure.

Unlike traditional REST APIs, GraphQL commonly exposes a single endpoint, such as:

```
POST /graphql
```

Key testing areas:

1. Introspection

Check whether the schema can be queried to reveal available types, fields, and operations.

2. Broken object-level authorization

Try accessing another user's data by changing object identifiers in GraphQL variables.

3. Excessive data exposure

Check whether the API returns sensitive fields that the client does not actually need.

4. Query depth and complexity

Test whether deeply nested or expensive queries can consume excessive resources.

5. Mutation authorization

Check whether users can execute unauthorized mutations, such as changing another user's details.

6. Batching abuse

Check whether multiple operations can be submitted in one request to bypass controls or amplify resource consumption.

7. Authentication and authorization

Verify that each resolver enforces the correct permissions.

Example:

```
query {
  user(id: "1001") {
    id
    name
    email
  }
}
```

Change the ID to another user's identifier and verify whether authorization is correctly enforced.

Tools:

- Burp Suite.
- GraphQL Voyager.
- InQL.
- GraphQL-specific testing utilities.

Remediation:

Enforce authorization in resolvers, limit query depth and complexity, restrict introspection where appropriate, and apply rate limiting and response-size controls.

94. How do you identify hardcoded secrets in JavaScript files?

Answer:

Modern web applications often load multiple JavaScript bundles containing application logic, API endpoints, configuration values, and sometimes accidentally exposed secrets.

Testing methodology:

1. Inspect the page source and identify loaded JavaScript files.
2. Download relevant bundles.
3. Search for sensitive keywords and patterns.
4. Review API endpoints and environment configuration.
5. Identify exposed credentials, tokens, private keys, or cloud storage references.
6. Determine whether discovered values are actually sensitive and usable.
7. Check source maps, such as `.map` files, for source code disclosure.

Example search terms:

```
api_key
secret
access_token
client_secret
password
private_key
```

Important distinction:

Not every key found in JavaScript is a vulnerability. Some keys are designed to be public, such as certain publishable API keys. The security impact depends on the permissions and restrictions attached to the key.

Impact:

- Unauthorized API access.
- Data exposure.
- Abuse of third-party services.
- Potential cloud resource compromise.

Remediation:

Never embed confidential server-side secrets in client-side code. Rotate exposed credentials, restrict API keys, and enforce server-side authorization.

95. How do you test web cache poisoning and cache deception?

Answer:

Both vulnerabilities involve incorrect caching behavior, but they work differently.

Web cache poisoning

An attacker manipulates a request so that a shared cache stores a malicious or unintended response. Other users may then receive that response.

Potentially relevant inputs include:

- Host headers.
- Forwarded headers.
- Query parameters.
- Unkeyed request headers.
- Cache-control behavior.

Web cache deception

An attacker tricks a caching system into storing a response that should not be publicly cached, potentially exposing sensitive user-specific content.

Testing methodology:

1. Identify whether a CDN or reverse proxy is present.
2. Observe cache-related headers such as `Age`, `X-Cache`, and `Cache-Control`.
3. Send repeated requests and compare responses.
4. Identify whether certain headers or parameters affect the response but are excluded from the cache key.
5. Check whether authenticated pages are cached.
6. Use separate test accounts to determine whether one user's content can be served to another.
7. Avoid poisoning shared production caches without explicit authorization and safeguards.

Impact:

- Stored malicious responses.
- Sensitive data disclosure.
- Cross-user content exposure.
- Persistent client-side attacks.

Remediation:

Configure cache keys correctly, avoid caching sensitive authenticated responses, and use appropriate cache-control directives.

96. What is insecure deserialization, and how do you test it?

Answer:

Insecure deserialization occurs when an application processes serialized data from an untrusted source without adequate validation.

Serialization converts objects into a format that can be stored or transmitted. Deserialization reconstructs those objects.

Some technologies that may be relevant include:

- Java serialization.
- PHP serialization.
- Python pickle.
- .NET object serialization.
- YAML and other object-capable formats.

Testing methodology:

1. Identify serialized data in requests, cookies, or application storage.
2. Look for recognizable serialization formats.
3. Determine whether user-controlled data is deserialized server-side.
4. Assess whether integrity protection or validation is implemented.
5. Review the libraries and frameworks involved.
6. Use safe, authorized test payloads to determine whether untrusted object processing can trigger unexpected behavior.

Potential impact:

- Remote code execution in vulnerable environments.
- Authentication bypass.
- Privilege escalation.
- Denial of service.
- Application logic manipulation.

Remediation:

Avoid deserializing untrusted data. Prefer safer data formats, enforce strict schemas, validate integrity, and use secure serialization libraries.

Interviewer follow-up: Is JSON always safe from deserialization vulnerabilities?

No. JSON itself is a data format, but application logic that converts JSON into objects or invokes unsafe constructors can still introduce security issues.

97. How do you test OAuth 2.0 and SAML implementations?

Answer:

OAuth 2.0 is an authorization framework that allows an application to obtain limited access to resources. OpenID Connect adds an authentication layer on top of OAuth 2.0.

SAML is an XML-based standard commonly used for enterprise Single Sign-On.

OAuth testing areas:

1. Validate the `redirect_uri`.
2. Check the `state` parameter to prevent CSRF in the authorization flow.
3. Verify PKCE where appropriate, especially for public clients.
4. Check authorization code reuse.
5. Check token expiration and revocation behavior.
6. Review scopes and excessive permissions.
7. Verify issuer and audience validation.
8. Check whether access tokens leak through URLs, logs, or browser history.

SAML testing areas:

1. Review signature validation.

2. Check assertion signature and response signature handling.
3. Verify issuer and audience.
4. Validate recipient and destination.
5. Check assertion expiration and conditions.
6. Test replay protection.
7. Check whether the application properly binds the authenticated identity to the assertion.
8. Assess XML Signature Wrapping risks.

Important distinction:

OAuth is primarily an authorization framework. OpenID Connect is used for authentication. SAML is a separate federation protocol that carries authentication and authorization-related assertions.

Remediation:

Follow the relevant protocol specifications, validate all security-critical fields, use secure signing and key management, and implement replay protection.

98. How do you perform a complete Web Application VAPT?

Answer:

I follow a structured methodology rather than randomly testing endpoints. The assessment generally includes planning, reconnaissance, attack-surface mapping, vulnerability testing, validation, and reporting.

Phase 1: Pre-engagement

- Confirm scope and authorization.
- Identify target domains, IPs, applications, and environments.
- Understand testing restrictions.
- Confirm test accounts and user roles.

Phase 2: Reconnaissance

- Identify application technologies.
- Review DNS and subdomains within scope.
- Identify exposed endpoints.
- Review robots.txt and sitemap.xml.

- Identify application entry points.

Phase 3: Application mapping

- Browse the application.
- Capture requests through Burp Suite.
- Identify parameters, cookies, headers, and API calls.
- Map user roles and workflows.
- Identify sensitive business functions.

Phase 4: Authentication and session testing

- Login and logout behavior.
- Password reset.
- MFA implementation.
- Session management.
- Cookie attributes.
- Authentication bypass.

Phase 5: Authorization testing

- IDOR and BOLA.
- Horizontal privilege escalation.
- Vertical privilege escalation.
- Function-level authorization.
- Multi-tenant isolation.

Phase 6: Input validation and injection testing

- SQL Injection.
- XSS.
- Command Injection.
- SSTI.
- XXE.
- SSRF.
- File upload.
- Path traversal.

Phase 7: Business logic testing

- Workflow bypass.

- Price manipulation.
- Coupon abuse.
- Transaction replay.
- Rate-limit weaknesses.
- Payment logic.

Phase 8: Configuration and security controls

- TLS configuration.
- Security headers.
- CORS.
- CSP.
- HTTP methods.
- Error handling.
- Cache behavior.

Phase 9: Validation and reporting

- Confirm vulnerabilities.
- Assess business impact.
- Assign severity using an appropriate risk methodology.
- Capture reproducible evidence.
- Provide remediation recommendations.
- Retest after fixes.

Tools I commonly use:

- Burp Suite for interception and manual testing.
- Postman for API validation.
- Nmap for authorized service discovery.
- Nuclei for template-based checks.
- OWASP ZAP for automated scanning.
- Browser developer tools for client-side analysis.

Interview tip: Explain that automated scanning supports the assessment, but manual testing is essential for authorization and business logic vulnerabilities.

99. How do you identify and exploit vulnerabilities through attack chaining?

Answer:

Attack chaining means combining multiple vulnerabilities or weaknesses to achieve a greater impact than each issue would produce individually.

A vulnerability that appears low severity in isolation may become critical when combined with another weakness.

Example scenario:

1. The application exposes detailed error messages.
2. The error reveals internal API endpoints.
3. One endpoint has an IDOR vulnerability.
4. The IDOR exposes sensitive user information.
5. A separate weak password-reset workflow allows account takeover.

The combined weaknesses can create a much more serious attack path.

Another example:

1. An application contains a stored XSS vulnerability.
2. An administrator views the affected page.
3. The attack executes in the administrator's browser context.
4. Weak authorization or exposed administrative functionality allows further impact.

Testing methodology:

- Map the application's attack surface.
- Identify individual vulnerabilities and weaknesses.
- Understand trust boundaries and user roles.
- Determine whether one weakness can enable another.
- Validate the full chain safely.
- Document each step and the final business impact.

How do you report an attack chain?

I document:

- Initial weakness.
- Preconditions.

- Exploitation steps.
- Intermediate impact.
- Final impact.
- Affected users or systems.
- Recommended fixes for each contributing weakness.

Important: Do not exaggerate severity based on a hypothetical chain. Demonstrate the dependencies and realistic exploitability.

100. You discover a critical vulnerability in production. What would you do?

Answer:

My first priority would be to validate the vulnerability safely, understand its impact, and communicate it through the agreed escalation process.

I would follow these steps:

1. Validate the finding

- Reproduce the issue with minimal impact.
- Confirm that it is not a false positive.
- Identify affected endpoints, accounts, and components.

2. Assess business impact

- Determine whether sensitive information is exposed.
- Check whether authentication or authorization can be bypassed.
- Understand whether the vulnerability enables account takeover, data manipulation, or remote code execution.

3. Avoid destructive testing

I would not perform activities that could damage production data, disrupt services, or affect other users.

4. Notify the appropriate stakeholders

I would immediately inform the designated security contact, project manager, or incident-response team according to the engagement's escalation procedure.

5. Provide evidence

I would share:

- Vulnerability description.
- Affected endpoint or component.
- Reproduction steps.
- Safe proof of concept.
- Business impact.
- Recommended remediation.
- Relevant request and response evidence.

6. Support remediation

I would help developers understand the root cause and suggest appropriate fixes.

7. Retest

After the fix is deployed, I would validate the original vulnerability and check for related weaknesses.

Interviewer follow-up: What if the client asks you to continue testing even after finding a critical production issue?

I would follow the agreed rules of engagement and escalation process. If continued testing could cause harm, I would pause the affected test activity and obtain explicit authorization before proceeding.

Web Security Section Completed

100 / 100 completed

You now have the complete Web Application Security section, Questions 1–100, continuing from SSTI at Question 74 through production incident handling at Question 100.

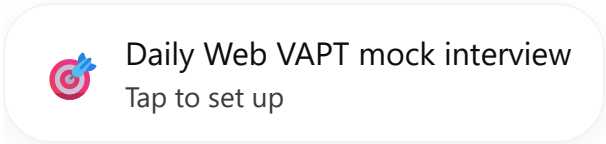
My recommendation for interview preparation: Pay particular attention to Questions 76, 78, 85, 88, 89, 90, 93, 97, and 99. These topics often require you to explain not just what a vulnerability is, but how you would discover it, prove it, assess its impact, and recommend a fix.

The strongest interview answer follows this pattern:

1. Explain the vulnerability.
2. Describe how it works technically.
3. Explain your testing methodology.
4. Give a practical example.
5. Discuss business impact.
6. Explain remediation.

That is how you move from knowing vulnerability names to answering like an experienced AppSec tester.

You can also practise with a scheduled mock Web VAPT interview:



You said:

Similarly please also answer the questions of the Mobile.

Worked for 3m 18s

ChatGPT said:

Absolutely. We'll cover the **100 Mobile Application Security interview questions** with the same depth as the Web section: technical explanations, Android-focused testing methodology, practical examples, impact, remediation, and likely interviewer follow-ups.

I'll keep the progression from fundamentals to advanced exploitation and real-world assessment scenarios, with particular attention to Android, since that is especially relevant to your mobile pentesting preparation.

Mobile Application Security Interview Questions and Answers (1–100)

This section focuses primarily on **Android application penetration testing**, while also covering the Android security architecture, reverse engineering, dynamic instrumentation, data storage, network security, and real-world assessment scenarios.

The answers are structured for someone with around four years of AppSec/VAPT experience. In an interview, explain not only what a vulnerability means, but also how you would identify it, validate it, assess its impact, and recommend a fix.

Section 1: Basic Mobile Security (1–25)

1. What is mobile application penetration testing?

Answer:

Mobile application penetration testing is the process of identifying security vulnerabilities in mobile applications running on platforms such as Android and iOS.

The objective is to determine whether an attacker can compromise the application's confidentiality, integrity, or availability, or abuse its functionality.

I generally divide mobile testing into four major areas:

1. **Client-side testing:** Local storage, application permissions, authentication, source code, and application configuration.
2. **Network testing:** API communication, TLS validation, certificate pinning, and sensitive data transmission.
3. **Backend testing:** APIs, authentication, authorization, business logic, and server-side vulnerabilities.
4. **Runtime testing:** Root detection, anti-debugging, tampering, dynamic instrumentation, and runtime behavior.

Typical testing workflow:

- Obtain the APK or application from an authorized source.
- Perform static analysis.
- Install the application on an emulator or test device.
- Configure traffic interception through Burp Suite.
- Perform dynamic analysis.
- Test local storage and application components.

- Assess APIs and backend functionality.
- Validate vulnerabilities and prepare a report.

Tools: MobSF, JADX, APKTool, ADB, Burp Suite, Frida, Objection, and Android Studio.

Interviewer follow-up: Is mobile application testing the same as API testing?

No. Mobile testing includes the application itself, its local data storage, runtime behavior, platform integrations, and network communication. API testing focuses on the backend interfaces. Both are connected because mobile applications commonly communicate with APIs.

2. What is the difference between Android and iOS application security testing?

Answer:

Android and iOS have different operating-system architectures, application packaging formats, security controls, and testing approaches.

Area	Android	iOS
Application package	APK / AAB	IPA
Programming languages	Java, Kotlin, C/C++	Swift, Objective-C, C/C++
Application sandbox	UID-based sandbox	Entitlement and sandbox-based controls
Application signing	APK signing	Code signing
Local storage	SharedPreferences, SQLite, files	Keychain, Core Data, files
Reverse engineering	JADX, APKTool	Ghidra, Hopper, class-dump
Dynamic instrumentation	Frida, Objection	Frida, Objection
Debugging	ADB	LLDB and Apple development tools

Android testing considerations:

- AndroidManifest.xml configuration.
- Exported Activities, Services, and Content Providers.
- Intent handling.
- SharedPreferences and SQLite.

- Root detection.
- WebView configuration.
- APK reverse engineering.

iOS testing considerations:

- Keychain storage.
- Entitlements.
- URL schemes and Universal Links.
- App Transport Security.
- Jailbreak detection.
- Mach-O binary analysis.
- iOS sandbox and provisioning profiles.

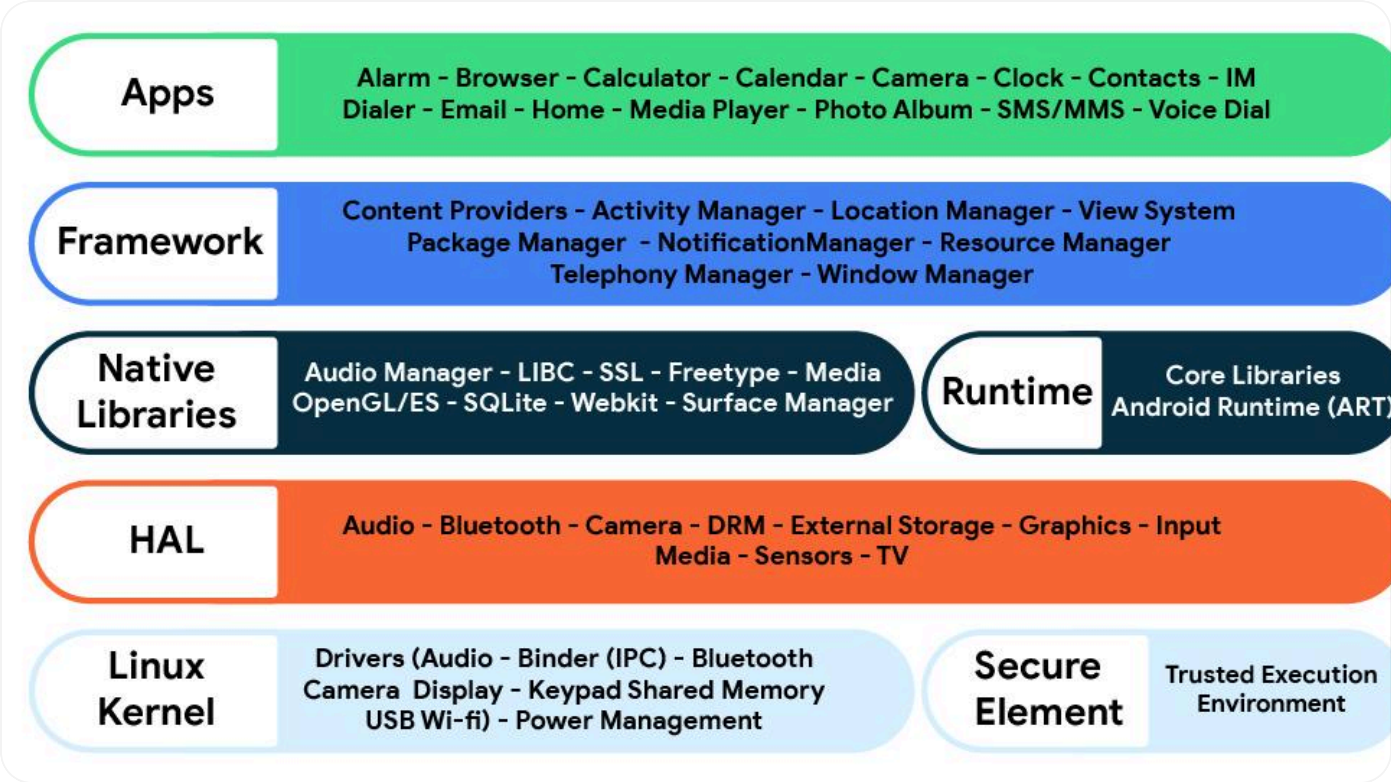
Interviewer follow-up: Which platform is easier to test?

It depends on the application, security controls, device access, and testing environment. Android often provides more flexibility for APK extraction, installation, and component inspection. iOS has different restrictions around application installation, signing, and runtime access.

3. Explain the complete Android application architecture.

Answer:

Android architecture is generally explained through its major software layers.



1. Linux Kernel

The foundation of Android. It provides:

- Process management.
- Memory management.
- Device drivers.
- Networking.
- Security isolation.
- UID-based application separation.

2. Hardware Abstraction Layer (HAL)

HAL provides standard interfaces between Android framework services and hardware components such as cameras, sensors, and Bluetooth.

3. Native Libraries

Android includes native libraries such as:

- SQLite.
- OpenSSL/BoringSSL components.
- Media libraries.

- Graphics libraries.
- Web rendering components.

4. Android Runtime (ART)

ART executes Android application bytecode. It manages application execution, memory, and garbage collection.

5. Application Framework

Provides higher-level services and APIs, including:

- Activity Manager.
- Package Manager.
- Window Manager.
- Location Manager.
- Notification Manager.
- Content Providers.

6. Applications

This layer contains system applications and third-party applications installed by users.

Security relevance:

As a mobile pentester, understanding the architecture helps identify where a vulnerability exists. For example, an exported Activity issue relates to application components, while insecure native code may involve the native library layer.

4. What is an APK file?

Answer:

APK stands for Android Package Kit. It is the installable package format used to distribute Android applications.

An APK is essentially an archive containing the application's code, resources, manifest, and other required files.

Important APK contents:

File or directory	Purpose
`AndroidManifest.xml`	Application configuration and component declarations
`classes.dex`	Compiled Dalvik bytecode
`resources.arsc`	Compiled resource table
`res/`	Application resources
`assets/`	Raw application assets
`lib/`	Native libraries
`META-INF/`	Package metadata and, depending on package format, signing-related information

How do you analyze an APK?

```
unzip application.apk -d extracted_apk
```

For source-code analysis:

```
jadx -d jadx_output application.apk
```

For resource and manifest analysis:

```
apktool d application.apk
```

Security relevance:

An APK can be reverse-engineered to identify hardcoded credentials, API endpoints, insecure configurations, cryptographic weaknesses, and sensitive business logic.

5. What are the important components of an Android application?

Answer:

Android applications have four primary application components:

1. Activities.
2. Services.
3. Broadcast Receivers.
4. Content Providers.

Activity: Represents a user-facing screen or interaction point.

Service: Performs background operations without requiring a user interface.

Broadcast Receiver: Receives and responds to broadcast messages from the system or other applications.

Content Provider: Manages and exposes structured application data through a standard interface.

Security relevance:

During penetration testing, I inspect whether these components are exported unnecessarily, whether permissions protect them, and whether untrusted applications can interact with them.

For example, an exported Activity that performs an administrative operation without checking authorization may allow privilege escalation.

6. What is AndroidManifest.xml?

Answer:

`AndroidManifest.xml` is an important configuration file inside an Android application package.

It declares application components, permissions, application metadata, and other configuration details.

Example:

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <uses-permission android:name="android.permission.INTERNET"/>

    <application
        android:allowBackup="true"
        android:debuggable="false">
```

```
        <activity
            android:name=".MainActivity"
            android:exported="true">

        </activity>

    </application>
</manifest>
```

What do I check during an assessment?

- Application permissions.
- Exported Activities.
- Exported Services.
- Exported Broadcast Receivers.
- Exported Content Providers.
- Debuggable configuration.
- Backup configuration.
- Network security configuration.
- Deep-link declarations.
- Minimum and target SDK versions.

Example security concern:

If a sensitive Activity is exported and does not perform authorization checks, another application may be able to invoke it.

Tools: JADX and APKTool.

7. What is the purpose of minSdkVersion and targetSdkVersion?

Answer:

These values control Android platform compatibility and indicate the Android API level an application supports or targets.

minSdkVersion

Defines the minimum Android API level on which the application can run.

targetSdkVersion

Indicates the Android API level the application is designed and tested against. Android uses this value to determine certain compatibility behaviors and security requirements.

Example:

```
<uses-sdk
    android:minSdkVersion="23"
    android:targetSdkVersion="35"/>
```

Security relevance:

An application targeting an older API level may not benefit from newer platform security protections or may be subject to different compatibility behaviors.

During testing, I inspect these values and determine whether the application uses outdated platform behaviors or insecure compatibility settings.

Important: A low target SDK is a risk indicator, not automatic proof of an exploitable vulnerability.

8. What are Activities, Services, Broadcast Receivers, and Content Providers?

Answer:

These are Android's four major application components.

Activities

Provide the user interface.

Security test: Check whether sensitive Activities are exported and whether they enforce authorization.

Services

Handle background operations.

Security test: Check whether exported Services allow unauthorized applications to invoke sensitive functionality.

Broadcast Receivers

Listen for broadcast messages.

Security test: Check whether untrusted applications can send crafted broadcasts that trigger sensitive operations.

Content Providers

Manage and share structured data.

Security test: Check whether another application can read or modify sensitive records through an exported provider.

Practical testing:

Inspect the manifest and then use ADB or Drozer, where appropriate, to examine exposed components.

9. What is an Intent in Android?

Answer:

An Intent is a messaging object used by Android components to request actions from other components.

For example, an application may use an Intent to:

- Open another Activity.
- Start a Service.
- Send a broadcast.
- Share data with another application.
- Open a browser URL.

Example:

```
Intent intent = new Intent(this, ProfileActivity.class);
startActivity(intent);
```

Security relevance:

Intents can carry sensitive information and trigger application functionality.

During testing, I check:

- Whether untrusted applications can invoke sensitive components.
- Whether Intent extras are validated.
- Whether sensitive data is exposed.
- Whether URI parameters can be manipulated.
- Whether the application trusts externally supplied Intent data.

10. What is the difference between explicit and implicit Intents?

Answer:

Explicit Intent

Specifies the exact component that should receive the request.

```
Intent intent = new Intent(this, DashboardActivity.class);
startActivity(intent);
```

Implicit Intent

Specifies an action and possibly data, allowing Android to identify a suitable component.

```
Intent intent = new Intent(Intent.ACTION_VIEW);
intent.setData(Uri.parse("https://example.com"));
startActivity(intent);
```

Security difference:

Explicit Intents are generally more predictable because the target component is specified.

Implicit Intents can potentially expose data to unintended applications if sensitive information is shared without carefully specifying the intended recipient.

Testing example:

Check whether an application sends sensitive information through an implicit Intent that another installed application could intercept.

Remediation:

Use explicit Intents for sensitive internal communication and carefully validate recipients and shared data.

11. What is an exported Android component?

Answer:

An exported component is an Android application component that can be accessed by other applications, subject to applicable permission and platform rules.

For example:

```
<activity
    android:name=".AdminActivity"
    android:exported="true"/>
```

This declaration makes the Activity externally accessible.

Why can it be dangerous?

If the Activity performs sensitive operations without proper authorization, another application may invoke it.

Testing methodology:

1. Inspect the manifest.
2. Identify exported components.
3. Determine whether they perform sensitive operations.
4. Check associated permissions.

5. Attempt authorized interaction using ADB.
6. Verify whether authorization is enforced.

Example:

```
adb shell am start -n com.example.app/.AdminActivity
```

Remediation:

Set components to `android:exported="false"` unless external access is required. If exposure is necessary, enforce permissions and authorization checks.

12. What is ADB, and how do you use it during mobile penetration testing?

Answer:

ADB stands for Android Debug Bridge. It is a command-line tool that allows communication with Android devices and emulators.

It is useful for installing applications, accessing logs, inspecting package information, executing shell commands, and debugging.

Common commands:

Check connected devices:

```
adb devices
```

Install an APK:

```
adb install application.apk
```

List installed packages:

```
adb shell pm list packages
```

Launch an Activity:

```
adb shell am start -n com.example.app/.MainActivity
```

View application logs:

```
adb logcat
```

Open an application shell:

```
adb shell
```

Pull a file:

```
adb pull /sdcard/example.txt .
```

Inspect package information:

```
adb shell dumpsys package com.example.app
```

Security relevance:

ADB helps inspect application behavior, identify exposed components, examine logs, investigate local storage, and support dynamic testing.

Interviewer follow-up: Does ADB require root access?

No. Many ADB operations work without root. Access to protected application directories and certain system-level operations generally requires elevated privileges.

13. What is the difference between static and dynamic analysis?

Answer:

Static Analysis

Examines an application without executing it.

Examples:

- Inspecting the manifest.
- Reviewing decompiled source code.
- Identifying hardcoded secrets.
- Reviewing cryptographic implementations.
- Identifying insecure configurations.

Tools: JADX, APKTool, MobSF.

Dynamic Analysis

Examines the application while it is running.

Examples:

- Intercepting network traffic.
- Monitoring runtime behavior.
- Testing authentication.
- Inspecting local files.
- Hooking methods.
- Testing root detection and certificate pinning.

Tools: Burp Suite, Frida, Objection, ADB.

Static analysis	Dynamic analysis
Examines code and configuration	Examines runtime behavior
Does not require normal execution	Requires application execution
Useful for hardcoded secrets	Useful for runtime secrets
May identify insecure code paths	Can validate actual exploitability

Best approach: Combine both. Static analysis identifies suspicious implementation, while dynamic analysis helps confirm whether it is exploitable.

14. What is reverse engineering in mobile application security?

Answer:

Reverse engineering is the process of analyzing an application's compiled package to understand its internal implementation and behavior.

In Android, an APK contains DEX bytecode and resources that can often be decompiled or disassembled for analysis.

Why do we reverse-engineer applications?

- Identify hardcoded credentials.
- Discover API endpoints.
- Understand authentication logic.
- Find insecure cryptographic implementations.
- Identify hidden functionality.
- Analyze root detection.
- Review network security configuration.
- Investigate sensitive data handling.

Typical workflow:

```
APK
 ↓
JADX / APKTool
 ↓
Manifest and source-code analysis
 ↓
Identify suspicious methods
 ↓
Dynamic validation using Frida
 ↓
Document findings
```

Important: Decompiled code is a reconstructed representation, not necessarily the original source code. Names and structure may be lost through obfuscation or compilation.

15. What is JADX, and why is it used?

Answer:

JADX is an open-source tool used to decompile Android DEX bytecode into Java-like source code.

It helps testers understand application logic.

Common uses:

- Searching for API endpoints.
- Identifying hardcoded credentials.
- Reviewing authentication logic.
- Inspecting cryptographic methods.
- Reviewing root detection.
- Finding WebView configurations.
- Identifying insecure storage mechanisms.

Command-line example:

```
jadx -d output application.apk
```

GUI usage is also available through JADX-GUI.

Limitations:

- Obfuscated code may be difficult to understand.
- Decompiled output may not perfectly represent original source code.
- Native C/C++ code is not decompiled into Java by JADX.

For native libraries, I would use tools such as Ghidra.

16. What is APKTool?

Answer:

APKTool is a tool used to decode and rebuild Android APK resources and application structure.

It is particularly useful for inspecting:

- AndroidManifest.xml.

- Resource files.
- Smali code.
- Application configuration.

Decode an APK:

```
apktool d application.apk
```

Rebuild an APK:

```
apktool b application_folder
```

Difference between JADX and APKTool:

JADX focuses on producing Java-like source code from DEX bytecode.

APKTool decodes resources and disassembles DEX bytecode into Smali.

When would you use APKTool?

I use it when I need to inspect manifest declarations, resource configurations, or Smali code, or when authorized testing requires modification and rebuilding of an application.

17. What is MobSF, and how does it help in mobile application security testing?

Answer:

MobSF stands for Mobile Security Framework. It is an automated mobile application security testing framework that supports static and dynamic analysis.

It can analyze Android and iOS applications.

Static analysis can identify:

- Insecure permissions.
- Hardcoded secrets.
- Weak cryptographic algorithms.
- Insecure configurations.

- Exported components.
- Potentially insecure network settings.
- Suspicious code patterns.

Dynamic analysis can help assess:

- Runtime behavior.
- Network communication.
- Application data storage.
- API interactions.
- Runtime security issues.

Typical workflow:

1. Upload the APK to MobSF.
2. Review the static analysis report.
3. Identify suspicious findings.
4. Validate findings manually using JADX or APKTool.
5. Install the application in a test environment.
6. Perform dynamic testing.
7. Document confirmed vulnerabilities.

Important interview point:

MobSF is an excellent starting point, but automated findings require manual validation. A reported insecure configuration does not always mean an exploitable vulnerability.

18. What is the difference between APK and AAB?

Answer:

APK is the Android installable package format, whereas AAB stands for Android App Bundle.

An AAB is a publishing format used to provide the required application resources and compiled code to distribution systems such as Google Play.

APK	AAB
Installable package	Publishing format

Can be installed directly	Usually needs to be processed into APKs
Contains package resources	Allows optimized APK generation
Commonly used for testing and distribution	Commonly used for Play Store publishing

Security relevance:

A pentester may receive an APK generated from an App Bundle or a set of split APKs. The tester needs to ensure that all relevant modules and dynamic features are considered during assessment.

19. What is application signing in Android?

Answer:

Android application signing uses a cryptographic key to establish the application's identity and verify package integrity.

An APK must be signed before it can be installed on modern Android versions.

Why is signing important?

- Establishes application identity.
- Helps detect unauthorized package modification.
- Supports application update verification.
- Enables signature-based trust relationships between applications.

Signing schemes include:

- v1: JAR signing.
- v2: APK Signature Scheme v2.
- v3: APK Signature Scheme v3.
- v4: Streaming-install-related signature support.

Inspect APK signing information:

```
apksigner verify --verbose --print-certs application.apk
```

Security relevance:

During reverse engineering, if an APK is modified, the original signature is no longer valid. The modified APK must be signed again for installation, and it generally cannot update the original application unless signing identity requirements are met.

20. What is Android Keystore?

Answer:

Android Keystore is a system facility that allows applications to create and use cryptographic keys with protection provided by the Android platform.

Instead of storing raw cryptographic keys directly in application files, an application can generate or import keys into the Keystore and request cryptographic operations through it.

Benefits:

- Key material can be non-exportable.
- Keys can be protected by hardware-backed security where supported.
- Key usage can be restricted.
- User authentication can be required for certain operations.

Testing methodology:

1. Review the application's cryptographic implementation.
2. Identify whether keys are stored directly in files or preferences.
3. Inspect how Keystore keys are generated and used.
4. Check whether authentication requirements are enforced.
5. Examine whether sensitive plaintext is exposed before encryption or after decryption.
6. Assess whether the application uses appropriate algorithms and modes.

Important: Android Keystore does not automatically make an application secure. Poor key usage, insecure cryptographic design, or exposed plaintext can still create vulnerabilities.

21. What is the difference between internal and external storage in Android?

Answer:

Internal storage is application-specific storage in the application's private sandbox. Other applications generally cannot access it directly under normal Android security controls.

External storage refers to shared or app-specific storage on external/shared storage volumes. Its access rules depend on Android version, storage type, and application permissions.

Testing methodology:

- Inspect application files under `/data/data/<package-name>/` on an authorized rooted test device.
- Review files stored under shared storage.
- Check whether sensitive information is written in plaintext.
- Check whether backups or exports expose data.
- Verify whether the application uses appropriate storage APIs and access restrictions.

Example risk:

An application storing authentication tokens in a publicly accessible shared directory may expose those tokens to other applications or users with access to the file.

Remediation:

Store sensitive data in private app storage, use suitable encryption and key management, and avoid storing secrets in shared storage.

22. What is SharedPreferences?

Answer:

SharedPreferences is an Android mechanism for storing small key-value pairs, commonly used for application settings and simple state information.

Example:

```
SharedPreferences prefs =
    getSharedPreferences("app_config", MODE_PRIVATE);

prefs.edit()
```

```
    .putString("username", "testuser")
    .apply();
```

What do I check during penetration testing?

- Whether access tokens are stored in plaintext.
- Whether passwords or PINs are stored.
- Whether sensitive personal information is exposed.
- Whether encryption keys are stored alongside encrypted data.
- Whether preferences are incorrectly shared.
- Whether backup mechanisms expose the files.

Important distinction:

`MODE_PRIVATE` restricts normal cross-application access. It does not encrypt the contents. On a rooted device, a tester may be able to inspect the application's private files.

Remediation:

Avoid storing passwords or unnecessary secrets. Use Android Keystore-backed encryption for sensitive data where local persistence is required.

23. What is SQLite, and how is it used in Android applications?

Answer:

SQLite is a lightweight relational database engine frequently used by Android applications to store structured local data.

Examples of data stored in SQLite:

- User profiles.
- Application settings.
- Cached records.
- Transaction history.
- Offline application data.

Testing methodology:

1. Identify database files through static analysis.

2. Inspect the application's private storage on an authorized test device.
3. Pull accessible database files.
4. Open them using SQLite Browser or the SQLite command-line tool.
5. Search for sensitive data.
6. Check whether sensitive records are stored in plaintext.
7. Review database access and encryption mechanisms.

Example:

```
SELECT name FROM sqlite_master
WHERE type='table';
```

Security risks:

- Plaintext storage of sensitive information.
- Exposure of authentication tokens.
- Insecure backups.
- Weak encryption.
- Sensitive data in database logs or temporary files.

Remediation:

Minimize sensitive local storage, apply appropriate encryption, protect keys using Android Keystore, and enforce secure backup policies.

24. What is WebView in Android?

Answer:

WebView is an Android component that allows applications to display web content inside the application without opening a separate browser.

Example:

```
WebView webView = findViewById(R.id.webview);
webView.loadUrl("https://example.com");
```

Security testing areas:

- JavaScript enabled unnecessarily.
- Unsafe JavaScript interfaces.
- Loading untrusted URLs.
- SSL error handling.
- File access configuration.
- Universal access from file URLs.
- Mixed content.
- Insecure navigation handling.

Example risk:

If an application loads attacker-controlled content in a WebView with an exposed JavaScript bridge, the attacker may be able to invoke sensitive native application functionality.

Remediation:

Load only trusted content, disable unnecessary JavaScript and file access, validate URLs, and expose only carefully designed JavaScript interfaces.

25. Explain the complete Android application penetration testing methodology.

Answer:

I follow a structured methodology that combines static, dynamic, network, and backend testing.

Phase 1: Scope and preparation

- Confirm authorization and scope.
- Obtain the APK or test build.
- Identify supported Android versions.
- Obtain test accounts and required roles.
- Prepare a test device or emulator.

Phase 2: Static analysis

Use MobSF, JADX, and APKTool.

Review:

- Manifest configuration.
- Application permissions.
- Exported components.
- Hardcoded secrets.
- API endpoints.
- Cryptographic implementation.
- Network security configuration.
- Debug and backup settings.

Phase 3: Dynamic analysis

Install the application and observe its behavior.

- Test authentication.
- Monitor application logs.
- Inspect local storage.
- Test component exposure.
- Examine runtime behavior.
- Test root detection and anti-debugging controls.

Phase 4: Network testing

Use Burp Suite to intercept traffic.

- Check TLS configuration.
- Test certificate validation.
- Assess certificate pinning.
- Inspect API requests and responses.
- Test authentication and authorization.
- Review sensitive data transmission.

Phase 5: Runtime instrumentation

Use Frida and Objection where appropriate.

- Inspect runtime methods.
- Observe sensitive function calls.
- Assess certificate pinning.
- Evaluate root detection.
- Investigate application tampering controls.

Phase 6: Backend and API testing

Test the application's APIs for:

- Broken object-level authorization.
- Broken authentication.
- Excessive data exposure.
- Injection.
- Rate limiting.
- Business logic issues.

Phase 7: Reporting and retesting

- Validate findings.
- Capture evidence.
- Assess severity and business impact.
- Provide remediation.
- Retest after fixes.

Interview-ready summary: I combine static analysis, dynamic analysis, network interception, runtime instrumentation, and backend API testing to assess the complete mobile application attack surface.

Section 2: Intermediate Android Security (26–50)

26. What is insecure local data storage?

Answer:

Insecure local data storage occurs when an application stores sensitive information in a way that allows unauthorized access or disclosure.

Examples include:

- Passwords.
- Authentication tokens.
- Personal information.
- Payment details.
- Session identifiers.

- Encryption keys.

Common locations:

- SharedPreferences.
- SQLite databases.
- Internal files.
- External/shared storage.
- Application logs.
- Cached data.
- Backup files.

Testing methodology:

1. Identify where the application stores data.
2. Use JADX to locate storage-related methods.
3. Inspect application files on an authorized device.
4. Search for sensitive values.
5. Check whether data is encrypted.
6. Review encryption key management.
7. Check backup and export behavior.

Impact: Account takeover, sensitive information disclosure, and session compromise.

Remediation: Minimize local sensitive data, use private storage, protect encryption keys with Android Keystore, and implement appropriate backup controls.

27. How do you identify sensitive information stored in SharedPreferences?

Answer:

I first use static analysis to identify SharedPreferences usage.

Search for methods such as:

```
getSharedPreferences()
getPreferences()
putString()
putBoolean()
```

Then I inspect the application's preference files on an authorized rooted device.

Typical location:

```
/data/data/<package-name>/shared_prefs/
```

I review whether the files contain:

- Access tokens.
- Refresh tokens.
- Passwords.
- User identifiers.
- Session information.
- Sensitive personal information.

Example risk:

If the application stores a long-lived access token in plaintext, an attacker with access to the application's private data may steal the token and impersonate the user.

Remediation:

Avoid storing unnecessary sensitive values. Use Keystore-backed encryption and appropriate token lifetime and revocation controls.

28. How do you identify sensitive information stored in SQLite databases?

Answer:

I identify database usage through static analysis by searching for:

```
SQLiteDatabase
SQLiteOpenHelper
rawQuery()
execSQL()
```

Next, I inspect database files on the authorized test device.

Common location:

```
/data/data/<package-name>/databases/
```

I pull accessible databases and inspect them using SQLite Browser or the SQLite CLI.

Example:

```
.tables
SELECT * FROM users;
```

I look for plaintext passwords, tokens, personal information, financial records, and sensitive application data.

Further checks:

- Database encryption.
- Encryption key storage.
- Temporary database files.
- Journal and WAL files.
- Backup exposure.

Remediation: Encrypt sensitive data where required, secure key management, restrict access, and avoid retaining unnecessary records.

29. What is insecure logging in Android applications?

Answer:

Insecure logging occurs when applications write sensitive information to system or application logs.

For example:

```
Log.d("AUTH", "Access token: " + token);
```

This is dangerous because logs may expose sensitive data to debugging tools, diagnostic systems, or other parties with sufficient access.

Testing methodology:

1. Run the application.
2. Monitor logs using ADB.
3. Exercise login, payment, password-reset, and other sensitive workflows.
4. Search for tokens, passwords, personal information, and session IDs.

Example:

```
adb logcat
```

Impact:

- Credential disclosure.
- Token theft.
- Exposure of personal information.
- Sensitive implementation details leakage.

Remediation:

Remove sensitive logging from production builds, sanitize log messages, and use controlled logging levels.

30. How do you identify hardcoded API keys and credentials in Android applications?

Answer:

Hardcoded secrets are credentials or sensitive values embedded directly in application code or resources.

I inspect:

- Decompiled Java/Kotlin code.
- XML resources.
- Configuration files.
- Assets.

- Native libraries.
- Build configuration.
- JavaScript or embedded web resources.

Search terms include:

```
api_key
client_secret
password
access_token
private_key
authorization
```

I also review API endpoints and third-party service configurations.

Important distinction:

A public API key is not automatically a vulnerability. I verify whether the key grants sensitive access or has insufficient restrictions.

Impact:

Unauthorized API access, service abuse, data exposure, or cloud resource compromise.

Remediation:

Move confidential secrets to server-side systems, rotate exposed credentials, restrict public keys, and enforce authorization on backend operations.

31. What is insecure communication in Android applications?

Answer:

Insecure communication occurs when an application transmits sensitive data without adequate transport security or fails to validate the identity of the remote server.

Examples:

- Using HTTP instead of HTTPS.
- Accepting invalid certificates.

- Trusting all certificates.
- Ignoring hostname verification.
- Sending tokens in URLs.
- Using weak TLS configurations.

Testing methodology:

1. Configure Burp Suite as an authorized proxy.
2. Install the proxy CA certificate where appropriate.
3. Capture application traffic.
4. Inspect URLs, headers, requests, and responses.
5. Check TLS validation and certificate handling.
6. Look for sensitive information in cleartext.
7. Review Network Security Configuration.

Impact: Credential theft, session hijacking, and interception or modification of sensitive traffic.

Remediation: Enforce HTTPS, validate certificates and hostnames, use modern TLS configurations, and avoid sending secrets in URLs.

32. How do you intercept Android application traffic using Burp Suite?

Answer:

I configure Burp Suite as a proxy and route the Android device's network traffic through it.

Steps:

1. Open Burp Suite.
2. Configure a proxy listener on an accessible interface and port.
3. Identify the host machine's IP address.
4. Configure the Android device's Wi-Fi proxy to point to that IP and port.
5. Install the Burp CA certificate using an appropriate test setup.
6. Open the application and generate traffic.
7. Inspect HTTP requests and responses in Burp Proxy.

Example:

```
Android device
      |
      v
Burp Suite proxy
      |
      v
Application backend
```

What if traffic does not appear?

I check:

- Device proxy settings.
- Firewall configuration.
- Network connectivity.
- Whether the app uses certificate pinning.
- Whether it uses a separate networking stack.
- Whether it uses QUIC or another protocol not handled by the configured proxy.

Important: Installing the Burp CA certificate does not guarantee that every application will trust it. Android's trust configuration and application-level pinning may affect interception.

33. What is SSL/TLS certificate validation?

Answer:

TLS certificate validation is the process through which a client verifies the identity of a server before establishing a secure connection.

The application should validate:

1. The certificate chain.
2. The certificate's validity period.
3. The issuing certificate authority.
4. The server hostname.
5. Relevant certificate constraints and trust policies.

Why is it important?

Without proper validation, an attacker may impersonate the server and intercept communication.

Testing methodology:

- Inspect the application's trust configuration.
- Use Burp Suite to observe whether a proxy certificate is accepted.
- Test invalid and untrusted certificates in an authorized environment.
- Review certificate validation code.
- Check whether hostname verification is disabled.

Remediation:

Use standard platform TLS APIs and never implement trust-all certificate managers or hostname verifiers.

34. What is SSL pinning?

Answer:

Certificate pinning is a security technique in which an application restricts which certificates or public keys it accepts for a particular server.

Normally, a client trusts certificates issued by trusted Certificate Authorities. With pinning, the application additionally checks that the server's certificate or public key matches an expected value.

Why is it used?

It reduces the risk of accepting an otherwise valid but unexpected certificate issued by a trusted CA.

Example:

If an application pins a server's public key, a Burp proxy certificate will not match the expected key and the connection may be rejected.

Testing relevance:

I assess whether pinning is implemented correctly, whether backup pins are available, and whether the application handles certificate rotation securely.

Important: Pinning is an additional control. It does not replace correct TLS certificate and hostname validation.

35. What is the difference between SSL pinning and certificate pinning?

Answer:

The term "SSL pinning" is commonly used, but modern secure connections generally use TLS rather than the obsolete SSL protocols.

Certificate pinning can refer to pinning a specific certificate or a public key associated with the server.

Certificate pinning

The application checks whether the server certificate matches an expected certificate.

Public-key pinning

The application checks whether the certificate's public key matches an expected public key.

Practical distinction:

If a server renews its certificate but retains the same public key, public-key pinning may continue to work, while exact certificate pinning may require an updated pin.

Security consideration:

A well-designed implementation should include a secure certificate rotation strategy and backup pins where appropriate.

36. How do you identify whether an Android application uses certificate pinning?

Answer:

I use both static and dynamic analysis.

Static analysis:

Search for:

- `CertificatePinner`
- `X509TrustManager`
- `checkServerTrusted`
- `HostnameVerifier`
- `network_security_config`
- `TrustManager`
- Certificate or public-key hashes.

Dynamic analysis:

1. Configure Burp Suite.
2. Install the proxy CA certificate.
3. Launch the application.
4. Observe whether requests fail during TLS negotiation.
5. Check application logs and network errors.
6. Compare behavior when the application connects directly versus through the proxy.

Important: A failed interception does not prove certificate pinning. It could be caused by proxy configuration, unsupported protocols, or network restrictions. I confirm using code analysis or controlled runtime testing.

37. How do you bypass SSL pinning using Frida?

Answer:

Frida is a dynamic instrumentation framework that allows a tester to inspect and modify application behavior at runtime.

In an authorized test environment, I use Frida to identify and instrument the application's certificate-validation logic.

General methodology:

1. Prepare a rooted test device or supported instrumentation environment.
2. Install and configure Frida.
3. Identify the application's networking libraries.
4. Review the implementation using JADX.

5. Identify relevant certificate-validation methods.
6. Use appropriate Frida hooks to observe or modify validation behavior.
7. Route traffic through Burp Suite and verify interception.

Common targets may include:

- Java `TrustManager` implementations.
- OkHttp `CertificatePinner`.
- Custom hostname verifiers.
- Native TLS validation routines.

Example commands:

```
frida-ps -U
```

```
frida -U -f com.example.app -l script.js
```

Important interview point:

There is no universal pinning-bypass script. The technique depends on the Android version, networking library, pinning implementation, and whether validation occurs in Java or native code.

38. What is Objection, and how is it used in mobile penetration testing?

Answer:

Objection is a runtime mobile exploration toolkit built on top of Frida.

It simplifies several common mobile security testing activities.

Common uses:

- Inspecting application files.
- Exploring application storage.
- Reviewing Android components.
- Investigating runtime behavior.
- Testing certain root detection controls.
- Assessing SSL pinning.

- Inspecting application configuration.

Example:

```
objection -g com.example.app explore
```

Useful commands depend on the Objection version and platform, but may include exploration of files, activities, preferences, and runtime security controls.

Difference between Frida and Objection:

Frida is the underlying instrumentation framework that provides flexible runtime hooking.

Objection provides a higher-level interface and prebuilt mobile exploration functionality.

Interview tip: I use Objection for quick checks and Frida when I need custom instrumentation or more precise control.

39. What is Frida, and how does it work?

Answer:

Frida is a dynamic instrumentation toolkit used to inspect and modify application behavior while the application is running.

It supports JavaScript-based scripts that can hook methods, inspect arguments, modify return values, and observe runtime operations.

How it works at a high level:

1. Frida connects to the target process.
2. It injects its instrumentation runtime.
3. A script identifies methods or functions.
4. Hooks are installed.
5. The tester observes or modifies behavior.

Example:

```
Java.perform(function () {
    var Activity = Java.use("android.app.Activity");

    Activity.onResume.implementation = function () {
        console.log("Activity resumed");
        return this.onResume();
    };
});
```

This illustrates hooking a Java method. In practice, the hook must be written carefully to avoid recursion and preserve the original behavior.

Security uses:

- Runtime method analysis.
- Certificate pinning assessment.
- Root detection analysis.
- Authentication logic investigation.
- Cryptographic method inspection.
- Native function instrumentation.

40. What is root detection in Android applications?

Answer:

Root detection is a security mechanism used by applications to identify whether an Android device has elevated privileges or has been modified in ways that may weaken platform security.

Root access can allow a tester or attacker to inspect private application files, modify runtime behavior, and access protected system resources.

Common root detection techniques:

- Checking for `su` binaries.
- Checking known root management packages.
- Inspecting system properties.
- Looking for modified build tags.
- Checking writable system locations.

- Detecting known root management artifacts.
- Detecting unusual processes or loaded libraries.

Why do applications implement it?

Financial, banking, and enterprise applications may use root detection as a risk signal because rooted devices can have a reduced security boundary.

Important: Root detection is not a substitute for server-side security. Sensitive operations and authorization must still be enforced by the backend.

41. How do you identify root detection mechanisms in an Android application?

Answer:

I use static and dynamic analysis.

Static analysis:

Search decompiled code for:

```
su
Superuser
Magisk
test-keys
which
isRooted
Build.TAGS
```

Review methods that inspect filesystem paths, installed packages, system properties, or process information.

Dynamic analysis:

1. Run the application on a rooted test device.
2. Observe whether it displays a warning or terminates.
3. Monitor logs.
4. Use Frida to observe root-check methods.
5. Compare behavior on rooted and non-rooted devices.

Tools:

- JADX.
- MobSF.
- Frida.
- Objection.
- ADB.

Interview point: I distinguish between detecting a rooted device and enforcing a security decision. Detection may be bypassed, so backend authorization must remain independent.

42. How do you bypass root detection using Frida?

Answer:

I first identify the exact root detection implementation instead of applying a generic bypass blindly.

Methodology:

1. Decompile the APK.
2. Identify root-check methods.
3. Review filesystem, package, property, and process checks.
4. Run the application on the test device.
5. Hook the identified methods with Frida.
6. Observe their return values.
7. In an authorized test, modify the result to assess whether the application proceeds.
8. Verify whether sensitive backend functionality remains protected.

Possible hook targets:

- Methods returning whether the device is rooted.
- File existence checks.
- Package manager queries.
- System property checks.
- Native root detection routines.

Important: Bypassing a root warning does not automatically mean the application is vulnerable. The security impact depends on what security boundary or sensitive operation the root detection was intended to protect.

43. What is emulator detection in Android applications?

Answer:

Emulator detection is the process of identifying whether an application is running inside an Android emulator rather than on a physical device.

Applications may use emulator detection to reduce automated abuse or detect potentially manipulated testing environments.

Common detection methods:

- Checking device model and manufacturer.
- Inspecting build properties.
- Checking hardware identifiers.
- Detecting emulator-specific files.
- Checking telephony characteristics.
- Looking for emulator-related packages or processes.

Security relevance:

Some applications restrict sensitive features on emulators. During testing, I assess whether these checks are implemented correctly and whether they can be bypassed.

Important: Emulator detection is not a reliable security boundary. An attacker may use a physical device or manipulate the checks.

44. How do you identify and bypass emulator detection?

Answer:

I first identify the checks through static analysis.

Search for:

- `Build.MODEL`
- `Build.MANUFACTURER`
- `Build.FINGERPRINT`
- Emulator-specific properties.
- Hardware and telephony checks.

Then I observe runtime behavior.

Testing methodology:

1. Install the application on an emulator.
2. Observe whether it detects the environment.
3. Review relevant checks in JADX.
4. Use Frida to monitor the methods.
5. In an authorized assessment, modify the relevant return values.
6. Confirm whether the application proceeds.
7. Check whether backend controls still protect sensitive operations.

Remediation:

Treat emulator detection as a risk signal rather than a primary security control. Enforce authorization and transaction integrity on the backend.

45. What are deep links in Android?

Answer:

Deep links allow an application to open a specific screen or resource when a user clicks a URL or invokes a supported URI.

Example:

```
myapp://profile/1001
```

A deep link can direct the application to the profile associated with identifier `1001`.

Security testing areas:

- Unvalidated URI parameters.

- Authentication bypass.
- IDOR through object identifiers.
- Open redirects.
- Intent injection.
- Sensitive data in URI parameters.
- Unsafe WebView navigation.

Testing methodology:

1. Inspect manifest intent filters.
2. Identify supported URI schemes and paths.
3. Trigger deep links using ADB.
4. Modify path and query parameters.
5. Test authenticated and unauthenticated behavior.
6. Check whether the application validates the destination and user permissions.

Example:

```
adb shell am start -a android.intent.action.VIEW \
-d "myapp://profile/1001"
```

Remediation:

Validate incoming URI data, enforce authorization, and avoid placing sensitive credentials in deep links.

46. What are Android App Links?

Answer:

Android App Links are verified web links that allow Android to open a specific application directly when a user visits a website URL associated with that application.

They are based on HTTP or HTTPS URLs and use domain verification.

A website publishes an `assetlinks.json` file under:

```
https://example.com/.well-known/assetlinks.json
```

This file establishes an association between the website and the Android application.

Difference from custom deep links:

- Custom deep links use custom URI schemes, such as `myapp://`.
- Android App Links use verified HTTP or HTTPS domains.

Security testing:

- Check domain verification.
- Test URL path validation.
- Test unauthorized object access.
- Check whether sensitive data is exposed through parameters.
- Verify that application routing enforces authorization.

47. How do you test deep-link vulnerabilities?

Answer:

I start by identifying deep-link intent filters in the manifest.

Then I trigger the links and manipulate their parameters.

Test cases:

1. Unauthenticated access to sensitive screens.
2. Changing user or transaction IDs.
3. Open redirects.
4. Unsafe URL handling.
5. Intent injection.
6. WebView loading of attacker-controlled URLs.
7. Exposure of sensitive information in query parameters.
8. Bypassing expected workflow steps.

Example:

```
myapp://payment/success?transaction_id=1001
```

I would verify whether the application trusts the `success` parameter or independently confirms the transaction status with the backend.

Remediation:

Validate URI schemes, hosts, paths, and parameters. Enforce authorization and transaction-state validation on the server.

48. What is an insecure Content Provider?

Answer:

A Content Provider is an Android component that manages and exposes application data through a standard interface.

It can allow other applications to query, insert, update, or delete data.

A Content Provider becomes insecure when it exposes sensitive information or allows unauthorized operations.

Testing methodology:

1. Inspect the manifest for Content Provider declarations.
2. Check whether the provider is exported.
3. Review read and write permissions.
4. Inspect URI patterns.
5. Attempt authorized queries and modifications.
6. Check for SQL injection or path traversal in provider logic.

Example risk:

An exported Content Provider exposes user records without requiring appropriate permissions.

Impact:

- Sensitive information disclosure.
- Unauthorized modification.
- Data deletion.
- Potential privilege escalation.

Remediation:

Disable unnecessary exports, enforce permissions, validate URI inputs, and apply proper authorization to every operation.

49. How do you test exported Activities and Services?

Answer:

I inspect the manifest to identify exported Activities and Services.

For each component, I determine whether it performs sensitive functionality and whether appropriate permissions and authorization checks are enforced.

Testing methodology:

1. Identify exported components.
2. Review their implementation using JADX.
3. Identify expected Intent actions and extras.
4. Attempt to invoke components using ADB.
5. Modify Intent data and extras.
6. Test with different user authentication states.
7. Check whether sensitive operations can be triggered by another application.

Example Activity invocation:

```
adb shell am start -n com.example.app/.AdminActivity
```

Example Service invocation:

```
adb shell am startservice \
-n com.example.app/.MyService
```

Impact:

Unauthorized access, sensitive operations, data exposure, or privilege escalation.

Remediation:

Disable unnecessary exports, use component permissions, validate all incoming data, and enforce authorization within the component.

50. What is an insecure Broadcast Receiver?

Answer:

A Broadcast Receiver listens for broadcast messages sent by Android or other applications.

An insecure receiver may accept untrusted broadcasts and perform sensitive operations without validating the sender or message.

Testing methodology:

1. Identify receivers in the manifest.
2. Check whether they are exported.
3. Review required permissions.
4. Inspect expected broadcast actions.
5. Send controlled broadcasts from an authorized test environment.
6. Modify Intent extras.
7. Check whether unauthorized actions can be triggered.

Example:

An exported receiver processes a broadcast that triggers a sensitive account operation without validating the sender.

Impact:

Unauthorized functionality execution, data disclosure, or state manipulation.

Remediation:

Avoid unnecessary exports, use appropriate permissions, validate broadcast data, and avoid relying on implicit broadcasts for sensitive internal operations.

Section 3: Advanced Android Security (51–75)

51. Explain Android application sandboxing.

Answer:

Android sandboxing isolates applications from one another using operating-system security mechanisms.

Each application normally runs under its own Linux UID and has a private data directory. This prevents one application from directly accessing another application's private files under normal conditions.

Security controls include:

- Linux UID-based isolation.
- Application-specific data directories.
- SELinux enforcement.
- Application permissions.
- Process isolation.
- Restricted access to system resources.

Example:

Application A normally cannot read files stored in:

```
/data/data/com.example.applicationB/
```

because the directory belongs to another application's UID.

Testing relevance:

I check whether the application unnecessarily exposes its data through exported components, shared storage, backups, logs, or insecure IPC mechanisms.

Interviewer follow-up: Does rooting bypass Android sandboxing?

Root access can weaken or bypass many of the operating system's normal access restrictions. This is why private application data can become accessible on a rooted test device. However, rooting does not automatically mean every application or device security control has been defeated.

52. Explain the Android permission model.

Answer:

Android permissions control an application's access to sensitive device capabilities and data.

Examples include:

- Camera.
- Microphone.
- Location.
- Contacts.
- SMS.
- Storage-related access.
- Bluetooth.

An application declares required permissions in its manifest and requests applicable runtime permissions from the user.

Major permission categories:

1. Normal permissions.
2. Dangerous permissions.
3. Signature permissions.
4. Special permissions.

Testing methodology:

1. Review permissions declared in the manifest.
2. Determine whether each permission is necessary.
3. Check when permissions are requested.
4. Test whether sensitive features work without the permission.
5. Check whether the application handles permission denial securely.
6. Verify whether sensitive data is transmitted or stored unnecessarily.

Example:

A simple calculator application requesting access to contacts and microphone may indicate excessive permissions that require investigation.

Remediation:

Follow the principle of least privilege. Request permissions only when needed and explain their purpose to the user.

53. What is the difference between normal, dangerous, and signature permissions?

Answer:

Normal permissions

Grant access to resources considered to have relatively low privacy or security impact. They are generally granted automatically when requested.

Dangerous permissions

Provide access to sensitive information or capabilities. They generally require runtime user approval.

Examples include camera, microphone, and location permissions.

Signature permissions

Are granted based on the signing identity relationship between applications, subject to the permission definition and platform rules.

Security relevance:

During testing, I review whether applications request excessive permissions and whether privileged application components are properly protected.

Important: A permission being granted does not automatically mean an application is vulnerable. The concern is whether it is unnecessarily requested or misused.

54. What are exported components, and why are they dangerous?

Answer:

Exported components allow other applications to interact with an application's Activities, Services, Broadcast Receivers, or Content Providers.

They are not inherently vulnerable. They become risky when they expose sensitive functionality without proper access controls.

Example:

An application exposes an administrative Activity:

```
<activity
    android:name=".AdminActivity"
    android:exported="true"/>
```

If the Activity allows a user to change account settings without checking authorization, another application may invoke it.

Testing methodology:

- Review manifest declarations.
- Identify sensitive component functionality.
- Inspect permissions.
- Trigger components externally.
- Manipulate incoming Intents.
- Verify authorization and input validation.

Remediation:

Set `android:exported="false"` when external access is unnecessary. Otherwise, protect the component with appropriate permissions and internal authorization checks.

55. How do you test unauthorized access to an exported Activity?

Answer:

I first identify exported Activities in the manifest and understand their functionality.

Then I attempt to invoke them externally using ADB.

Example:

```
adb shell am start -n com.example.app/.ProfileActivity
```

I also test whether the Activity accepts untrusted Intent extras.

Example:

```
Intent intent = getIntent();
String userId = intent.getStringExtra("user_id");
```

If the Activity trusts the supplied `user_id` without verifying the logged-in user's authorization, it may expose another user's information.

Test cases:

- Access without authentication.
- Access using a lower-privileged account.
- Modify Intent extras.
- Supply unexpected data types.
- Trigger sensitive operations directly.
- Bypass expected application workflows.

Remediation:

Validate the caller's permissions, validate all Intent data, and enforce authorization independently of the UI workflow.

56. How do you test and exploit an insecure Content Provider?

Answer:

I inspect the provider declaration in the manifest and review its implementation.

Important properties include:

- `android:exported`
- `android:readPermission`

- `android:writePermission`
- URI permissions.
- Path permissions.

I then identify the provider's content URI and determine which operations it exposes.

Testing operations:

- Query.
- Insert.
- Update.
- Delete.

Example ADB interaction:

```
adb shell content query \
--uri content://com.example.app.provider/users
```

I assess whether unauthorized applications can read or modify sensitive records.

Additional checks:

- SQL injection through selection arguments.
- Path traversal.
- Improper URI validation.
- Excessive data exposure.
- Missing row-level authorization.

Remediation:

Restrict access, enforce permissions, validate URIs, use parameterized database operations, and apply authorization to individual records.

57. What is Intent Redirection?

Answer:

Intent Redirection occurs when an application receives an untrusted Intent or Intent-like input and uses it to launch another component without adequately validating the destination or data.

An attacker may manipulate the incoming Intent to cause the application to invoke unintended functionality.

Example scenario:

An application receives an Intent containing a destination component. It forwards that Intent without checking whether the target is trusted.

An attacker may abuse this behavior to access a non-exported component through the vulnerable application's privileges, depending on the implementation and Android protections.

Testing methodology:

1. Identify components that accept nested Intents or Intent data.
2. Review how incoming Intents are parsed.
3. Check whether the application forwards them.
4. Attempt to manipulate the target component, action, URI, or extras.
5. Assess whether sensitive internal functionality can be triggered.

Remediation:

Do not blindly forward untrusted Intents. Validate the destination, action, URI, and extras. Prefer explicit, internally constructed Intents.

58. What is PendingIntent Hijacking?

Answer:

A PendingIntent allows another application or system component to execute a predefined operation using the creator application's identity and permissions, subject to the PendingIntent's configuration and platform controls.

PendingIntent misuse can allow unintended modification or invocation of an operation.

Security concerns:

- Mutable PendingIntents when immutability is appropriate.
- Missing or weak request-code handling.
- Untrusted Intent data.
- Sensitive operations exposed through PendingIntents.

- Incorrect flags.

Example:

An application creates a mutable PendingIntent for a sensitive operation and does not adequately constrain the data that can be supplied when it is invoked.

Testing methodology:

1. Identify PendingIntent creation.
2. Review its flags.
3. Determine whether mutability is required.
4. Inspect the underlying Intent.
5. Check whether an external party can influence sensitive parameters.
6. Assess the operation's authorization requirements.

Remediation:

Use `FLAG_IMMUTABLE` when modification is not required, validate all sensitive data, and ensure that the operation independently enforces authorization.

59. What is Android Task Hijacking?

Answer:

Task Hijacking refers to attacks involving Android task and Activity behavior, where a malicious application may interfere with how an application's activities are presented or managed.

Historically, insecure task configurations and certain Activity launch behaviors have enabled phishing-like scenarios in which an attacker attempts to make a malicious screen appear associated with a trusted application.

Testing areas:

- `taskAffinity` .
- `launchMode` .
- Activity launch behavior.
- Task management.
- Application identity and screen transitions.

Impact:

- UI spoofing.
- Phishing.
- User credential disclosure.
- Confusion between trusted and malicious screens.

Remediation:

Use appropriate task configuration, avoid unnecessary task affinity customization, and carefully validate application navigation behavior.

Interview point: Task hijacking is platform- and configuration-dependent. Do not assume every unusual `taskAffinity` value is exploitable.

60. What is Tapjacking in Android?

Answer:

Tapjacking is a UI redressing attack in which an attacker tricks a user into tapping a hidden or misleading UI element.

A malicious overlay may appear to show a harmless screen while intercepting the user's interaction with an underlying application.

Example:

A malicious application displays a transparent overlay over a legitimate application's permission or sensitive-action screen. The user believes they are tapping one thing but actually activates another.

Testing methodology:

- Review sensitive Activities.
- Check overlay handling.
- Test touch filtering.
- Review whether sensitive operations require clear user confirmation.
- Examine the use of `filterTouchesWhenObscured` .

Example:

```
android:filterTouchesWhenObscured="true"
```

Remediation:

Protect sensitive UI interactions, reject obscured touches where appropriate, and avoid relying solely on visual confirmation for critical actions.

61. What is an insecure WebView configuration?

Answer:

An insecure WebView configuration occurs when an application embeds web content with unsafe settings or fails to validate the content it loads.

Common weaknesses include:

- JavaScript enabled unnecessarily.
- Untrusted URLs loaded.
- Unsafe JavaScript bridges.
- File access enabled unnecessarily.
- Weak SSL error handling.
- Mixed content allowed.
- Inadequate navigation restrictions.

Testing methodology:

1. Identify WebView usage in JADX.
2. Review WebSettings configuration.
3. Identify URLs loaded by the application.
4. Check JavaScript configuration.
5. Inspect JavaScript interfaces.
6. Test whether untrusted content can be loaded.
7. Assess file access and SSL error handling.

Impact:

Depending on the configuration, an attacker may achieve XSS-like behavior, access native functionality, or expose local application data.

Remediation:

Load only trusted content, disable unnecessary features, validate navigation, and implement secure JavaScript bridge controls.

62. What is JavaScript Interface Exposure in Android WebView?

Answer:

Android WebView can expose native application methods to JavaScript through a JavaScript interface.

For example:

```
webView.addJavascriptInterface(new WebAppInterface(), "Android");
```

JavaScript inside the WebView may then invoke exposed methods through the interface.

Why can this be dangerous?

If untrusted web content is loaded, it may interact with exposed native functionality.

The impact depends on the methods exposed, the Android version, and the WebView configuration.

Testing methodology:

1. Search for `addJavascriptInterface`.
2. Identify the exposed interface class.
3. Review methods exposed to JavaScript.
4. Identify URLs loaded into the WebView.
5. Check whether untrusted navigation is possible.
6. Determine whether sensitive native methods can be invoked.

Remediation:

Expose only necessary methods, validate origins and navigation, load trusted content, and avoid exposing sensitive native functionality to untrusted JavaScript.

63. What are the security risks of `addJavascriptInterface()`?

Answer:

`addJavascriptInterface()` allows JavaScript in a WebView to communicate with native Android code.

The main security risk is that untrusted content may be able to invoke native methods that were intended only for trusted application content.

Example:

Suppose an application exposes a method that retrieves a user's authentication token. If attacker-controlled JavaScript can execute inside that WebView, it may call the exposed method and obtain sensitive information.

Testing methodology:

- Identify the interface object.
- Review methods annotated with `@JavascriptInterface`.
- Determine whether sensitive data or functionality is exposed.
- Check whether arbitrary URLs can be loaded.
- Assess origin validation and navigation restrictions.

Remediation:

- Avoid exposing sensitive methods.
- Restrict WebView navigation.
- Use a strict allowlist of trusted origins.
- Avoid loading untrusted content in privileged WebViews.
- Keep Android and WebView components updated.

64. What is cleartext traffic in Android?

Answer:

Cleartext traffic refers to network communication that is not protected by encryption, such as HTTP communication.

Example:

```
http://api.example.com/login
```

If an application sends credentials or session tokens through HTTP, an attacker positioned on the network may be able to intercept or modify the traffic.

Testing methodology:

1. Inspect API endpoints in the application.
2. Monitor traffic.
3. Review Network Security Configuration.
4. Identify HTTP requests.
5. Check whether sensitive information is transmitted in plaintext.
6. Verify whether the application rejects insecure connections.

Remediation:

Use HTTPS for all sensitive communication, disable cleartext traffic where possible, and configure TLS validation correctly.

Example manifest setting:

```
<application
    android:usesCleartextTraffic="false">
</application>
```

The actual behavior can also depend on Network Security Configuration and Android version.

65. What is Android Network Security Configuration?

Answer:

Network Security Configuration is an Android XML configuration mechanism that allows applications to define network security policies.

It can control:

- Trusted certificate authorities.
- Custom trust anchors.
- Cleartext traffic.
- Certificate pinning.
- Debug-only trust settings.

Example:

```
<network-security-config>
    <base-config cleartextTrafficPermitted="false">
        <trust-anchors>
            <certificates src="system"/>
        </trust-anchors>
    </base-config>
</network-security-config>
```

The application can reference it through the manifest:

```
<application
    android:networkSecurityConfig="@xml/network_security_config">
</application>
```

Testing methodology:

1. Locate the configuration file.
2. Review trusted certificate sources.
3. Check cleartext settings.
4. Review domain-specific configurations.
5. Inspect debug overrides.
6. Check whether pinning is configured and correctly implemented.

Remediation:

Use appropriate system trust anchors, disable unnecessary cleartext traffic, and avoid trusting user-installed certificates in production unless there is a specific security requirement.

66. What is weak TLS configuration in Android applications?

Answer:

Weak TLS configuration occurs when an application uses outdated protocols, weak cipher suites, incorrect certificate validation, or insecure trust settings.

Examples:

- TLS 1.0 or other obsolete protocols where still enabled.
- Weak cipher suites.
- Trust-all certificate managers.
- Disabled hostname verification.
- Acceptance of expired or untrusted certificates.
- Incorrect certificate-chain validation.

Testing methodology:

- Inspect the application's TLS implementation.
- Observe network handshakes.
- Review supported protocols and cipher suites.
- Test certificate validation.
- Check for custom insecure trust managers.
- Verify hostname validation.

Impact:

Potential man-in-the-middle attacks, traffic interception, and compromise of sensitive data.

Remediation:

Use platform TLS APIs, modern TLS configurations, strong cipher suites, and correct certificate and hostname validation.

67. Explain cryptographic key management in Android.

Answer:

Cryptographic key management covers how encryption keys are generated, stored, accessed, rotated, and destroyed.

A strong encryption algorithm cannot protect data if its keys are stored insecurely.

Good practices:

- Generate keys using secure random sources.
- Use Android Keystore where appropriate.
- Prefer hardware-backed key protection when available and required.
- Avoid hardcoding encryption keys.
- Avoid storing keys beside encrypted data.
- Restrict key usage.
- Rotate or invalidate keys when necessary.
- Require user authentication for sensitive key operations where appropriate.

Testing methodology:

1. Identify cryptographic operations in source code.
2. Find where keys are generated.
3. Inspect where keys are stored.
4. Determine whether keys are exportable.
5. Check algorithm and mode selection.
6. Assess whether encryption keys can be recovered from application files or runtime memory.

Important: Hardware-backed storage improves key protection, but cannot prevent every attack against a compromised application running on a compromised device.

68. What are common cryptographic mistakes in Android applications?

Answer:

Common mistakes include:

- Hardcoded encryption keys.
- Weak random number generation.
- ECB mode encryption.
- Reusing initialization vectors incorrectly.
- Using outdated hash functions for security-sensitive purposes.
- Storing keys in SharedPreferences.
- Implementing custom cryptographic algorithms.
- Using encryption without authentication.

- Reusing passwords as encryption keys without a proper key derivation function.

Example:

AES-ECB is generally unsuitable for protecting structured sensitive data because identical plaintext blocks produce identical ciphertext blocks.

Testing methodology:

1. Search for cryptographic API usage.
2. Identify algorithms and modes.
3. Review key generation.
4. Check IV and nonce handling.
5. Examine storage of keys.
6. Validate whether ciphertext integrity is protected.

Remediation:

Use well-reviewed platform cryptographic APIs and authenticated encryption modes such as AES-GCM where appropriate. Avoid custom cryptography.

69. How do you test whether encryption keys are stored in SharedPreferences?

Answer:

I first identify cryptographic operations through static analysis.

I search for:

- AES key generation.
- SecretKeySpec.
- KeyGenerator.
- Cipher initialization.
- SharedPreferences writes.
- Base64 encoding and decoding.

Then I trace the key lifecycle.

Testing methodology:

1. Identify where the key is generated.

2. Determine whether it is stored in SharedPreferences.
3. Inspect the preference file on an authorized test device.
4. Check whether the key is stored in plaintext or obfuscated.
5. Determine whether the same key is used to encrypt sensitive application data.

Important distinction:

Base64 encoding and obfuscation are not encryption. If the key can be recovered easily from local storage, encryption may provide little protection.

Remediation:

Use Android Keystore to protect cryptographic keys and avoid storing raw key material in preferences or application resources.

70. How do you test biometric authentication in Android?

Answer:

Biometric authentication allows users to authenticate using supported biometric mechanisms, such as fingerprint or face recognition.

The important security question is whether the application correctly binds sensitive operations to a secure authentication result.

Testing methodology:

1. Identify biometric authentication implementation.
2. Review whether Android's supported biometric APIs are used.
3. Check how authentication results are handled.
4. Test biometric cancellation and failure.
5. Check fallback to PIN or password.
6. Determine whether sensitive operations require fresh authentication.
7. Inspect whether cryptographic keys are protected by user authentication.

Potential weaknesses:

- Treating a UI callback as sufficient proof of authentication.
- Insecure fallback logic.

- Reusing old authentication state.
- Allowing sensitive operations without the required authentication.
- Poorly protected cryptographic keys.

Remediation:

Use Android's supported biometric APIs and bind sensitive cryptographic operations to appropriate authentication requirements.

71. What is the Play Integrity API?

Answer:

The Play Integrity API is a Google service that helps applications assess whether requests are coming from an environment that meets specified integrity criteria.

It can provide signals related to:

- Application recognition.
- Application licensing.
- Device integrity.
- Certain risk indicators.

Why do applications use it?

Financial, gaming, and enterprise applications may use integrity signals to identify potentially modified applications or untrusted environments.

Testing methodology:

1. Identify Play Integrity API integration.
2. Review how the application requests integrity tokens.
3. Determine how the backend validates those tokens.
4. Check whether the backend relies on client-side claims.
5. Review whether sensitive actions are denied when integrity checks fail.
6. Check whether tokens are bound to the relevant request and validated appropriately.

Important: The API provides risk signals, not an absolute guarantee that a device or application is secure.

Remediation:

Validate integrity tokens on the server, bind them to relevant requests, and use them as one layer of a broader risk-based security architecture.

72. What is the difference between root detection and runtime integrity checking?

Answer:

Root detection attempts to identify whether the Android device has elevated privileges or root-related modifications.

Runtime integrity checking examines whether the application or its execution environment has been modified or manipulated.

Examples of runtime integrity checks include:

- APK signature verification.
- Code integrity checks.
- Detection of unexpected loaded libraries.
- Debugger detection.
- Runtime method integrity checks.
- Detection of application tampering.

Difference:

Root detection focuses on the device's privilege state.

Runtime integrity checking focuses on whether the application is executing in an expected and trustworthy state.

Security limitation:

Both can be bypassed by sufficiently capable attackers. Neither should replace backend authorization, transaction validation, or server-side security controls.

73. How do you use Frida to hook application methods?

Answer:

Frida allows testers to intercept method execution and observe or modify application behavior at runtime.

Example:

```
Java.perform(function () {
    var Login = Java.use("com.example.app.LoginManager");

    Login.isAuthenticated.implementation = function () {
        console.log("isAuthenticated() called");

        var result = this.isAuthenticated();

        console.log("Original result: " + result);

        return result;
    };
});
```

This example observes the return value of an authentication-related method. In a real assessment, I would first confirm the method signature and avoid recursive calls.

General workflow:

1. Identify the class and method using JADX.
2. Connect to the running application using Frida.
3. Write a hook for the target method.
4. Observe parameters and return values.
5. Modify behavior only when required to validate a security control.
6. Document the finding and its actual impact.

Common uses:

- Inspecting authentication logic.
- Investigating certificate pinning.
- Testing root detection.
- Observing encryption methods.
- Assessing anti-tampering mechanisms.

74. How do you bypass anti-debugging and anti-tampering controls?

Answer:

Anti-debugging and anti-tampering controls are designed to identify whether an application is being debugged, modified, or instrumented.

Common anti-debugging techniques:

- Checking debugger attachment.
- Using `Debug.isDebuggerConnected()`.
- Inspecting process state.
- Detecting debugging ports.
- Monitoring suspicious runtime behavior.

Common anti-tampering techniques:

- APK signature verification.
- Application checksum verification.
- Integrity checks.
- Detection of modified libraries.
- Detection of instrumentation frameworks.

Testing methodology:

1. Identify checks using JADX.
2. Observe behavior during Frida attachment.
3. Review application logs.
4. Identify methods responsible for terminating or restricting execution.
5. Use Frida to observe those methods.
6. In an authorized test, assess whether bypassing a check changes the application's behavior.
7. Determine whether the control protects a genuinely sensitive operation.

Remediation:

Use multiple integrity signals, protect sensitive backend operations, monitor suspicious activity, and avoid relying on a single client-side check.

75. How do you perform security testing of Android native libraries?

Answer:

Android applications may include native libraries written in C or C++, generally stored in the APK's `lib/` directory.

Examples:

```
lib/arm64-v8a/libnative.so
lib/armeabi-v7a/libnative.so
```

These libraries may contain cryptographic functions, business logic, or performance-sensitive operations.

Testing methodology:

1. Extract native libraries from the APK.
2. Identify architecture and exported symbols.
3. Analyze strings and symbols.
4. Review native functions using Ghidra or IDA.
5. Identify hardcoded secrets and insecure cryptographic logic.
6. Review memory handling and input validation.
7. Investigate JNI communication between Java/Kotlin and native code.
8. Use dynamic instrumentation where appropriate.

Common vulnerabilities:

- Buffer overflows.
- Memory corruption.
- Insecure cryptography.
- Hardcoded secrets.
- Unsafe JNI interfaces.
- Command injection.
- Improper input validation.

Remediation:

Use secure native coding practices, validate all inputs, avoid hardcoded secrets, and apply memory-safety protections and appropriate compiler hardening.

Section 4: Expert-Level and Real-World Scenarios (76–100)

76. You receive an APK but do not have the source code. How would you perform a security assessment?

Answer:

This is a common black-box mobile penetration testing scenario. I would perform reverse engineering and dynamic analysis to understand the application's implementation.

Step 1: Initial analysis

- Identify APK package name and version.
- Inspect AndroidManifest.xml.
- Identify permissions and exported components.
- Check minimum and target SDK versions.
- Identify application architecture and native libraries.

Step 2: Static analysis

Use JADX, APKTool, and MobSF to identify:

- Hardcoded credentials.
- API endpoints.
- Cryptographic implementations.
- Insecure storage.
- Authentication logic.
- Root detection.
- Certificate pinning.
- WebView configurations.

Step 3: Dynamic analysis

Install the APK on an Android test device and inspect:

- Application behavior.
- Local storage.
- Logs.
- Runtime permissions.

- Network traffic.
- Component interactions.

Step 4: Runtime instrumentation

Use Frida and Objection to investigate application behavior and validate suspected security controls.

Step 5: API testing

Use Burp Suite and Postman to assess backend authentication, authorization, and business logic.

Conclusion: Lack of source code does not prevent a comprehensive assessment. APK reverse engineering, runtime instrumentation, and API testing provide substantial visibility.

77. You are unable to intercept application traffic through Burp Suite. How would you troubleshoot it?

Answer:

I would troubleshoot systematically rather than immediately assuming certificate pinning.

Step 1: Verify proxy configuration

- Check Burp listener IP and port.
- Confirm the device and computer are on the same network.
- Verify Android proxy configuration.
- Check firewall rules.

Step 2: Verify connectivity

Try accessing a normal HTTPS website through the configured proxy.

Step 3: Verify certificate installation

Check whether the Burp CA certificate is installed and whether the application trusts user-installed certificates.

Step 4: Investigate certificate pinning

Review the APK using JADX and look for:

- OkHttp CertificatePinner.
- Custom TrustManager.
- HostnameVerifier.
- Native certificate-validation logic.

Step 5: Check alternative networking mechanisms

The application may use:

- Native networking libraries.
- QUIC or HTTP/3.
- WebSockets.
- Separate network stacks.
- VPN or proxy restrictions.

Step 6: Perform runtime analysis

Use Frida to identify the certificate-validation mechanism and assess it in an authorized environment.

Interview point: A failed proxy connection is not sufficient evidence of SSL pinning. I establish the actual reason before reporting or attempting a bypass.

78. You identify certificate pinning in an application. How would you test it?

Answer:

First, I would understand the implementation and determine whether the application uses certificate pinning or public-key pinning.

Testing process:

1. Configure Burp Suite.
2. Install its CA certificate.
3. Launch the application.
4. Observe whether the TLS connection is rejected.
5. Review the certificate-validation implementation through JADX.
6. Identify the networking library.
7. Use Frida or Objection to assess the pinning mechanism in the authorized test environment.

8. Intercept traffic after controlled instrumentation.
9. Verify whether the backend still enforces authentication and authorization.

What do I evaluate?

- Whether pinning is implemented correctly.
- Whether backup pins are configured.
- Whether certificate rotation is supported.
- Whether hostname validation remains enabled.
- Whether sensitive API operations are protected independently.

Important: Successful pinning bypass does not automatically indicate a vulnerability. The finding depends on whether the bypass reveals an actual security weakness, such as sensitive data exposure or inadequate backend controls.

79. You find access tokens stored in SharedPreferences. How would you assess the risk?

Answer:

I would first determine the token type, lifetime, permissions, and accessibility.

Testing methodology:

1. Locate the SharedPreferences file.
2. Identify the stored token.
3. Determine whether it is plaintext or encrypted.
4. Check whether encryption keys are securely managed.
5. Identify token expiration.
6. Determine whether refresh tokens are also stored.
7. Assess whether the token can be reused.
8. Check whether the backend validates token scope and authorization.
9. Test whether logout invalidates the token.

Example:

An access token stored in plaintext could be exposed if an attacker gains access to the application's private storage.

If the token is long-lived and has extensive privileges, the impact may be significant.

Remediation:

- Minimize persistent token storage.
- Use appropriate Keystore-backed protection.
- Apply short access-token lifetimes.
- Rotate refresh tokens.
- Implement server-side revocation.
- Enforce proper token authorization.

Interviewer follow-up: Is encrypting the token enough?

No. If the encryption key is accessible to the attacker or the application can be manipulated at runtime, encryption alone may not provide adequate protection.

80. An application blocks execution on rooted devices. How would you assess this control?

Answer:

I would determine how root detection is implemented and whether the application relies on it as a security boundary.

Testing methodology:

1. Install the application on a rooted Android device.
2. Observe the warning or blocking behavior.
3. Review the APK using JADX.
4. Identify root detection methods.
5. Examine filesystem and package checks.
6. Inspect runtime behavior using Frida.
7. Assess whether the detection can be bypassed.
8. Verify whether backend operations remain properly protected.

What would I report?

I would not automatically report bypassing root detection as a critical vulnerability.

I would assess whether bypassing it allows access to sensitive functionality that should have remained protected.

Remediation:

Use layered integrity checks, backend-side authorization, transaction validation, and risk-based controls. Root detection should be treated as an additional signal.

81. An application detects an emulator and refuses to launch. How would you test it?

Answer:

I would first identify which emulator characteristics the application checks.

Static analysis:

Search for:

```
Build.MODEL
Build.MANUFACTURER
Build.FINGERPRINT
Build.PRODUCT
```

Also inspect emulator-related files, properties, and package checks.

Dynamic analysis:

1. Run the application on an emulator.
2. Observe the blocking behavior.
3. Review logs.
4. Identify the detection methods.
5. Use Frida to monitor relevant checks.
6. Test the application's behavior in an authorized environment after controlled instrumentation.

What is the security impact?

Emulator detection is often used to discourage automated abuse or testing. Bypassing it is not automatically a vulnerability.

I would focus on whether the application has other security weaknesses that allow unauthorized functionality or data access.

82. You discover hardcoded credentials in an APK. What would you do?

Answer:

I would first establish whether the discovered value is actually a secret and whether it grants access to sensitive functionality.

Testing methodology:

1. Identify the credential in decompiled code or resources.
2. Determine its purpose.
3. Identify the associated service or API.
4. Check its privileges and restrictions.
5. Validate it safely using an authorized request.
6. Determine whether it exposes sensitive data or functionality.
7. Document evidence without unnecessarily exposing the secret.

Potential impact:

- Unauthorized API access.
- Third-party service abuse.
- Data disclosure.
- Cloud resource compromise.
- Financial impact.

Remediation:

- Remove secrets from the APK.
- Rotate exposed credentials.
- Restrict API keys.
- Move confidential operations server-side.
- Apply backend authentication and authorization.

Interview point: A hardcoded value is not automatically a critical finding. Its actual privileges and exploitability determine severity.

83. An application stores encrypted SQLite data. How would you assess whether the encryption is secure?

Answer:

I would examine both the encryption implementation and the key-management mechanism.

Testing methodology:

1. Locate the SQLite database.
2. Identify the encryption library or framework.
3. Review the encryption algorithm.
4. Identify how keys are generated.
5. Determine where keys are stored.
6. Check whether keys are protected by Android Keystore.
7. Inspect initialization vectors or nonces.
8. Assess authentication and integrity protection.
9. Observe how the application accesses plaintext at runtime.

Potential weaknesses:

- Hardcoded encryption keys.
- Keys stored beside the database.
- Weak key derivation.
- Insecure encryption modes.
- Static IV or nonce reuse.
- Plaintext temporary files.
- Sensitive information exposed in memory.

Remediation:

Use established encryption libraries, secure key management, authenticated encryption, and appropriate data lifecycle controls.

Important: An encrypted database does not automatically mean secure storage. The key lifecycle is equally important.

84. How do you test Android application backup security?

Answer:

Android applications may support backup mechanisms that allow application data to be transferred or restored.

If sensitive data is included in backups without appropriate protection, it may become exposed.

Testing methodology:

1. Inspect `android:allowBackup` .
2. Review backup-related configuration.
3. Identify sensitive files and preferences.
4. Check whether sensitive information is included in backup rules.
5. Review device-to-device transfer behavior.
6. Assess whether backup data is encrypted and appropriately protected.

Example manifest setting:

```
<application
    android:allowBackup="false">
</application>
```

Important: Backup behavior depends on Android version, target SDK, device implementation, and backup mechanism. The manifest attribute alone does not establish the complete security posture.

Impact:

Exposure of tokens, personal information, application records, or other sensitive local data.

Remediation:

Exclude sensitive files from backup, configure appropriate backup rules, and protect any data that must be backed up.

85. How do you test screenshots, clipboard, and keyboard caching vulnerabilities?

Answer:

Mobile applications can unintentionally expose sensitive information through operating-system features.

1. Screenshot and screen recording

Check whether sensitive screens allow screenshots or appear in recent-app previews.

Android provides:

```
getWindow().setFlags(
    WindowManager.LayoutParams.FLAG_SECURE,
    WindowManager.LayoutParams.FLAG_SECURE
);
```

2. Clipboard

Check whether passwords, OTPs, payment details, or other secrets are copied to the clipboard.

Clipboard content may be accessible to other applications under certain platform and version conditions.

3. Keyboard caching and autofill

Check whether sensitive input fields are configured appropriately.

For example, password fields should use suitable input types and should avoid unnecessary learning or autofill exposure.

Testing methodology:

- Open sensitive screens.
- Take screenshots.
- Check recent-app previews.
- Copy sensitive values.
- Inspect clipboard behavior.
- Review keyboard and autofill configuration.

Remediation:

Use secure window flags where appropriate, avoid copying sensitive values unnecessarily, and configure sensitive input fields securely.

86. How do you test deep-link vulnerabilities in a real-world Android application?

Answer:

I identify deep-link declarations in the manifest and understand the application's routing logic.

Testing methodology:

1. Identify custom URI schemes.
2. Identify verified App Links.
3. Find relevant Activities.
4. Trigger deep links through ADB.
5. Modify path and query parameters.
6. Test access without authentication.
7. Test access with different user roles.
8. Check for IDOR, open redirects, and Intent injection.
9. Review WebView navigation triggered by deep links.

Example:

```
adb shell am start -a android.intent.action.VIEW \
-d "myapp://account/1002"
```

Example vulnerability:

If a deep link accepts a user ID and displays another user's account without verifying ownership, it may result in an IDOR vulnerability.

Remediation:

Validate incoming links and enforce authorization on the backend for every sensitive object.

87. You discover an exported Content Provider. How would you assess its security?

Answer:

I would inspect the provider's implementation and determine what information or operations it exposes.

Testing methodology:

1. Identify the provider in AndroidManifest.xml.
2. Check the exported attribute.
3. Review read and write permissions.
4. Identify accessible content URIs.
5. Test queries.
6. Test insertion, updating, and deletion.
7. Review path and selection validation.
8. Determine whether sensitive data can be accessed by unauthorized applications.

Example:

A provider exposes:

```
content://com.example.app.provider/users
```

If unauthorized applications can query this URI and retrieve user records, it may indicate sensitive information disclosure.

Remediation:

Disable unnecessary exports, use suitable permissions, validate queries, and enforce row-level authorization.

88. What is Intent Spoofing, and how do you test it?

Answer:

Intent Spoofing occurs when an attacker sends a crafted Intent to an application component to trigger unexpected behavior or supply malicious data.

This can affect exported Activities, Services, and Broadcast Receivers.

Testing methodology:

1. Identify exported components.
2. Review their expected Intent actions and extras.
3. Inspect whether input is validated.
4. Send crafted Intents.
5. Modify data fields, URIs, and identifiers.
6. Test whether sensitive functionality can be triggered.
7. Check whether authorization is enforced.

Example:

A receiver trusts a broadcast containing:

```
user_id=1001
action=approve
```

If it performs an approval operation without validating the sender and user's permissions, an attacker may abuse the functionality.

Remediation:

Restrict component exposure, validate incoming Intents, use permissions, and enforce authorization on sensitive operations.

89. You discover that an application loads remote URLs inside WebView. What security checks would you perform?

Answer:

I would investigate whether the WebView can load attacker-controlled content and whether native functionality is exposed to the loaded page.

Testing methodology:

1. Identify the WebView implementation.
2. Review `loadUrl()` usage.
3. Identify the source of the URL.
4. Check whether the URL is user-controlled.
5. Review JavaScript configuration.

6. Inspect JavaScript bridges.
7. Check file access settings.
8. Review SSL error handling.
9. Test navigation to untrusted domains.
10. Assess whether sensitive native methods can be invoked.

Example:

If a WebView loads an attacker-controlled page and exposes a privileged JavaScript interface, the attacker may be able to invoke native application functionality.

Remediation:

- Allowlist trusted URLs.
- Validate navigation.
- Disable unnecessary JavaScript.
- Restrict JavaScript interfaces.
- Disable unnecessary file access.
- Never bypass SSL errors.

90. How would you test a JavaScript bridge vulnerability in an Android application?

Answer:

A JavaScript bridge allows web content running inside WebView to communicate with native Android methods.

I would identify the bridge and review the methods exposed to JavaScript.

Testing methodology:

1. Search for `addJavascriptInterface`.
2. Identify the bridge class.
3. Review methods exposed using `@JavascriptInterface`.
4. Identify sensitive functionality.
5. Determine which URLs can access the bridge.
6. Check whether external or attacker-controlled content can be loaded.
7. Validate whether native operations can be invoked without proper authorization.

Example:

A bridge exposes:

```
@JavascriptInterface
public String getSessionToken() {
    return sessionToken;
}
```

If untrusted JavaScript can access this interface, it may expose the session token.

Remediation:

Avoid exposing sensitive methods, restrict content origins, validate navigation, and design bridges with minimal functionality.

91. How do you test biometric authentication when the application also provides a PIN or password fallback?

Answer:

I examine the complete authentication flow, including the fallback mechanism.

Biometric authentication may be strong, but an insecure fallback can undermine the entire control.

Testing methodology:

1. Identify biometric authentication APIs.
2. Test successful biometric authentication.
3. Test failed biometric authentication.
4. Test cancellation.
5. Test repeated failures.
6. Examine PIN or password fallback.
7. Check whether the fallback can be bypassed.
8. Determine whether sensitive operations require fresh authentication.
9. Review cryptographic key authentication requirements.

Example:

If a payment screen requires fingerprint authentication but allows a weak, easily bypassed PIN fallback, the overall authentication mechanism may be weak.

Remediation:

Use secure fallback mechanisms, enforce server-side authorization for sensitive operations, and bind cryptographic operations to appropriate authentication requirements.

92. An application implements SSL pinning, root detection, and anti-debugging. How would you perform a comprehensive assessment?

Answer:

I would assess each control independently and then evaluate whether the controls work together to protect sensitive operations.

Step 1: SSL pinning

- Identify the pinning implementation.
- Test certificate validation.
- Assess whether controlled runtime instrumentation can bypass it.
- Verify that TLS hostname validation remains correct.

Step 2: Root detection

- Identify filesystem and package checks.
- Test behavior on a rooted device.
- Observe the checks with Frida.
- Assess whether bypassing detection exposes sensitive functionality.

Step 3: Anti-debugging

- Identify debugger detection.
- Observe runtime behavior.
- Check process termination and application restrictions.
- Assess whether instrumentation is detected.

Step 4: Anti-tampering

- Review APK signature verification.

- Check integrity validation.
- Assess modified application behavior.

Step 5: Backend security

Even if all client-side controls are bypassed in the test environment, verify that the backend continues enforcing authentication, authorization, transaction validation, and business rules.

Interview point: A mobile application should not depend exclusively on client-side protections to secure critical business operations.

93. You discover API endpoints and secrets inside a native `.so` library. How would you analyze them?

Answer:

Native `.so` files are shared libraries compiled for specific Android architectures.

I would extract the library and perform static and dynamic analysis.

Static analysis:

Use tools such as:

- Ghidra.
- IDA.
- `strings`.
- `readelf`.
- `nm`.

Example:

```
strings libnative.so
```

I would search for:

- API endpoints.
- Hardcoded credentials.
- Cryptographic keys.

- Internal service names.
- Debug messages.
- Sensitive configuration values.

Further analysis:

1. Identify exported symbols.
2. Review native functions.
3. Examine JNI methods.
4. Understand how Java/Kotlin calls native functions.
5. Review cryptographic operations.
6. Use Frida to observe native functions at runtime where appropriate.

Impact:

Exposed secrets may enable unauthorized API access, third-party service abuse, or disclosure of internal infrastructure.

Remediation:

Remove confidential secrets from client-side binaries, rotate exposed credentials, and enforce backend authorization.

94. How do you test native Android code written in C or C++?

Answer:

Native Android code requires analysis of both the native implementation and its interaction with Java/Kotlin code.

Static analysis:

- Extract `.so` libraries.
- Identify architecture.
- Review symbols and strings.
- Analyze functions using Ghidra or IDA.
- Inspect JNI interfaces.
- Review input validation and memory handling.

Dynamic analysis:

- Use Frida to instrument native functions.
- Observe function parameters and return values.
- Monitor sensitive operations.
- Test malformed inputs in an authorized environment.
- Investigate crashes and unexpected behavior.

Common vulnerabilities:

- Buffer overflow.
- Integer overflow.
- Use-after-free.
- Out-of-bounds access.
- Format-string vulnerabilities.
- Unsafe JNI handling.
- Hardcoded secrets.

Remediation:

Use memory-safe programming practices, validate input boundaries, apply compiler hardening, and avoid unsafe native interfaces.

95. What is dynamic native instrumentation, and how do you use Frida for it?

Answer:

Dynamic native instrumentation allows a tester to inspect and modify native functions while an application is running.

Frida supports native instrumentation through APIs such as `Interceptor`.

Example:

```
var address = Module.findExportByName(
    "libnative.so",
    "native_function"
);
```

```
if (address !== null) {
    Interceptor.attach(address, {
        onEnter: function (args) {
            console.log("Native function called");
        },
        onLeave: function (retval) {
            console.log("Function returned: " + retval);
        }
    });
}
```

Testing methodology:

1. Identify the native library.
2. Identify the target function or exported symbol.
3. Determine its address.
4. Attach a Frida hook.
5. Observe parameters and return values.
6. Validate behavior through controlled inputs.
7. Document the security impact.

Uses:

- Investigating native authentication logic.
- Reviewing encryption functions.
- Analyzing root detection.
- Understanding native TLS validation.
- Inspecting sensitive data processing.

96. How do you test the security of IPC between Android applications?

Answer:

IPC stands for Inter-Process Communication. Android applications use IPC mechanisms to communicate with other processes.

Common mechanisms include:

- Intents.
- Binder.

- Bound Services.
- Broadcasts.
- Content Providers.
- Messenger.

Testing methodology:

1. Identify exported components.
2. Review IPC interfaces.
3. Inspect permissions.
4. Test external access.
5. Manipulate incoming parameters.
6. Review authentication and authorization.
7. Check for sensitive information leakage.
8. Test whether untrusted applications can trigger privileged operations.

Example:

An exported bound Service exposes a method that retrieves account information but does not verify the caller's identity or permissions.

Impact:

Unauthorized data access, privilege escalation, and sensitive functionality abuse.

Remediation:

Restrict component exposure, enforce permissions, authenticate callers where appropriate, and validate all incoming data.

97. How do you assess Firebase security in an Android application?

Answer:

Many Android applications use Firebase for authentication, databases, cloud storage, messaging, and analytics.

During testing, I identify Firebase integration and assess whether backend security rules are correctly configured.

Testing areas:

- Firebase Authentication.
- Realtime Database.
- Cloud Firestore.
- Firebase Storage.
- Cloud Functions.
- Security rules.
- Authentication and authorization.
- Exposed configuration and API keys.

Testing methodology:

1. Identify Firebase configuration in the APK.
2. Identify associated Firebase services.
3. Determine which services are accessible.
4. Review database and storage access controls.
5. Test unauthenticated access where authorized.
6. Test whether one user can access another user's records.
7. Review Cloud Function authorization.

Important distinction:

Firebase configuration values, including many API keys, are commonly included in client applications by design. Their presence alone is not a vulnerability. Misconfigured security rules or exposed privileged credentials are the real concerns.

Remediation:

Apply restrictive database and storage rules, validate authorization server-side, and never expose service-account private keys in mobile applications.

98. How do you perform an integrated mobile application and API security assessment?

Answer:

I assess the Android application and its backend APIs together because mobile client security and server-side security are closely connected.

Step 1: Mobile static analysis

- Inspect APK.
- Review manifest.
- Identify endpoints.
- Find hardcoded secrets.
- Examine storage and cryptography.

Step 2: Mobile dynamic analysis

- Install application.
- Monitor logs.
- Inspect local storage.
- Test exported components.
- Assess runtime protections.

Step 3: Network analysis

- Intercept traffic through Burp Suite.
- Review TLS and pinning.
- Inspect authentication tokens.
- Analyze request and response data.

Step 4: API testing

Test:

- Broken object-level authorization.
- Broken authentication.
- Excessive data exposure.
- Injection.
- Rate limiting.
- Business logic.
- Mass assignment.
- Improper function-level authorization.

Step 5: Cross-layer attack scenarios

For example:

1. The mobile application stores a long-lived access token insecurely.

2. The token is recovered from local storage.
3. The token is replayed against the backend.
4. The backend does not properly enforce object-level authorization.
5. Another user's information becomes accessible.

This demonstrates how a client-side weakness and backend weakness can combine into a larger attack path.

Remediation:

Secure local storage, protect tokens, enforce server-side authorization, and apply appropriate API security controls.

99. How do you assess the overall severity and business impact of a mobile application vulnerability?

Answer:

I do not determine severity only from the technical weakness. I evaluate how an attacker could exploit it and what the business consequences would be.

Factors I consider:

1. Exploitability.
2. Required privileges.
3. User interaction.
4. Confidentiality impact.
5. Integrity impact.
6. Availability impact.
7. Number of affected users.
8. Sensitivity of exposed information.
9. Business and regulatory consequences.
10. Possibility of chaining with other vulnerabilities.

Example 1:

An application stores a non-sensitive UI preference in plaintext.

This may have limited impact.

Example 2:

An application stores long-lived authentication tokens in plaintext, and the backend accepts those tokens without adequate authorization checks.

This may create a much more serious account-compromise risk.

Severity assessment:

I use CVSS as a structured technical scoring methodology, while considering the organization's risk context and business impact.

Reporting:

I include:

- Vulnerability description.
- Affected component.
- Reproduction steps.
- Proof of concept.
- Technical impact.
- Business impact.
- CVSS score and rationale.
- Remediation recommendation.

100. You identify a vulnerability that requires a rooted device to exploit. Would you report it?

Answer:

Yes, I would assess and report it if it represents a genuine security weakness within the agreed assessment scope. However, I would not automatically classify it as high or critical severity simply because it is exploitable on a rooted device.

I would evaluate the following:

1. Understand the root requirement

Determine whether root access is required because:

- The vulnerability exists in the application's private storage.

- The application exposes sensitive data at runtime.
- The weakness involves local application files.
- The issue requires bypassing an application security control.

2. Determine the attack scenario

Ask whether an attacker can realistically obtain the required access and what additional conditions are necessary.

3. Assess the impact

For example:

- Exposure of authentication tokens.
- Disclosure of sensitive personal information.
- Unauthorized modification of application data.
- Session compromise.
- Account takeover.

4. Check for alternative attack paths

Determine whether the same weakness can be exploited without root access through:

- Exported components.
- Insecure Content Providers.
- Application backups.
- Logs.
- External storage.
- Intent handling.
- Other application interfaces.

5. Assign appropriate severity

Use CVSS and business context. Root access is an important precondition, but it does not automatically make a finding informational or invalid.

Example:

Suppose a banking application stores authentication tokens in plaintext under its private application directory. Accessing the file requires root privileges.

I would document the storage weakness, demonstrate token recovery in the authorized test environment, assess token privileges and lifetime, and determine the actual account-compromise risk.

Remediation:

- Minimize sensitive local storage.
- Use Android Keystore-backed protection.
- Apply secure token lifecycle controls.
- Enforce server-side authorization.
- Avoid relying exclusively on root detection.

Strong interview closing statement:

"My responsibility as a penetration tester is to demonstrate the actual security impact, not just identify a technical weakness. I consider the attacker's prerequisites, exploitability, affected assets, and business impact before assigning severity."

Mobile Security Section Completed

100 / 100 completed

You now have the full Mobile Application Security question set, Questions 1–100.

What I would prioritize before your next Android AppSec interview

These are especially important because interviewers often move beyond definitions and ask you to demonstrate how you would test them.

Priority	Topics	Questions
Very high	Android architecture, components, manifest	3–12
Very high	Local storage, SharedPreferences, SQLite	21–30
Very high	Burp, TLS, certificate pinning	31–39
Very high	Root detection and Frida	40–44, 73–74
Very high	Exported components, Intent, Content Provider	48–59

High	WebView and JavaScript bridges	61–63, 89–90
High	Cryptography and Keystore	67–69, 83
High	Deep links and IPC	45–47, 96
High	Native libraries and Firebase	75, 93–97
Very high	Complete assessment and attack chaining	25, 76–100

Practical Android toolkit to be comfortable with



MobSF

Automated mobile application analysis



JADX / APKTool

Reverse engineering and manifest analysis



Burp Suite

Network interception and API testing



Frida / Objection

Runtime instrumentation and security-control testing



ADB

Device interaction, logs, and component testing



Ghidra

Native library analysis

Recommended official references

- OWASP MASVS ↗: Mobile Application Security Verification Standard.
- OWASP MASTG ↗: Mobile Application Security Testing Guide.
- Android Security and Privacy ↗: Official Android security documentation.
- Frida Documentation ↗: Runtime instrumentation.

- MobSF ↗: Mobile security testing framework.

My strongest advice: Don't just memorize these 100 answers. Take an APK, perform a complete assessment, and practise explaining each finding while demonstrating it in Burp, JADX, ADB, or Frida. That practical fluency is what turns interview preparation into real confidence.

You said:

Can I donwload this whole chat or threat as pdf. I want the complete thing in one pdf.

ChatGPT said:

Yes! You can save this entire interview-preparation conversation as **one PDF**.

Since you're using ChatGPT on Chrome desktop, here's the easiest method:

Option 1: Save the entire chat as PDF

1. Open this conversation.
2. Scroll through the chat to make sure all the responses have loaded.
3. Press **Ctrl + P**.
4. Under Destination, select **Save as PDF**.
5. Set Layout to Portrait.
6. Enable Background graphics if you want the formatting preserved.
7. Click **Save**.

Option 2: I can prepare a polished interview-preparation PDF

I can organise the material into a proper study document with:

- API Security: 100 questions and detailed answers.
- Web Application Security: 100 questions and detailed answers.
- Mobile Application Security: 100 questions and detailed answers.
- Numbered questions and structured headings.
- Code examples and practical testing methodologies.
- A table of contents and revision checklist.

One important point: I don't currently have the complete text of every earlier answer available in this turn, so I don't want to generate an incomplete PDF and call it the full 300-question document.

If you export the conversation and upload the file here, I can turn the complete material into one professionally formatted PDF.